

12

Image Pattern Classification

One of the most interesting aspects of the world is that it can be considered to be made up of patterns.

A pattern is essentially an arrangement. It is characterized by the order of the elements of which it is made, rather than by the intrinsic nature of these elements.

Norbert Wiener

Preview

We conclude our coverage of digital image processing with an introduction to techniques for image pattern classification. The approaches developed in this chapter are divided into three principal categories: classification by *prototype matching*, classification based on an *optimal statistical* formulation, and classification based on *neural networks*. The first two approaches are used extensively in applications in which the nature of the data is well understood, leading to an effective pairing of features and classifier design. These approaches often rely on a great deal of engineering to define features and elements of a classifier. Approaches based on neural networks rely less on such knowledge, and lend themselves well to applications in which pattern class characteristics (e.g., features) are learned by the system, rather than being specified a priori by a human designer. The focus of the material in this chapter is on principles, and on how they apply specifically in image pattern classification.

Upon completion of this chapter, readers should:

- Understand the meaning of patterns and pattern classes, and how they relate to digital image processing.
- Be familiar with the basics of minimum-distance classification.
- Know how to apply image correlation techniques for template matching.
- Understand the concept of string matching.
- Be familiar with Bayes classifiers.
- Understand perceptrons and their history.
- Be familiar with the concept of learning from training samples.
- Understand neural network architectures.
- Be familiar with the concept of deep learning in fully connected and deep convolutional neural networks. In particular, be familiar with the importance of the latter in digital image processing.

12.1 BACKGROUND

Humans possess the most sophisticated pattern recognition capabilities in the known biological world. By contrast, the capabilities of current recognition machines pale in comparison with tasks humans perform routinely, from being able to interpret the meaning of complex images, to our ability for generalizing knowledge stored in our brains. But recognition machines play an important, sometimes even crucial role in everyday life. Imagine what modern life would be like without machines that read barcodes, process bank checks, inspect the quality of manufactured products, read fingerprints, sort mail, and recognize speech.

In image pattern recognition, we think of a *pattern* as a spatial arrangement of features. A *pattern class* is a set of patterns that share some common properties. Pattern recognition by machine encompasses techniques for automatically assigning patterns to their respective classes. That is, given a pattern or sets of patterns whose class is unknown, the job of a pattern recognition system is to assign a class label to each of its input patterns.

There are four main stages involved in recognition: (1) sensing, (2) preprocessing, (3) feature extraction, and (4) classification. In terms of image processing, sensing is concerned with generating signals in a spatial (2-D) or higher-dimensional format. We covered numerous aspects of image sensing in Chapter 1. Preprocessing deals with techniques for tasks such as noise reduction, enhancement, restoration, and segmentation, as discussed in earlier chapters. You learned about feature extraction in Chapter 11. Classification, the focus of this chapter, deals with using a set of features as the basis for assigning class labels to unknown input image patterns.

In the following section, we will discuss three basic approaches used for image pattern classification: (1) classification based on matching unknown patterns against specified prototypes, (2) optimum statistical classifiers, and (3) neural networks. One way to characterize the differences between these approaches is in the level of “engineering” required to transform raw data into formats suitable for computer processing. Ultimately, recognition performance is determined by the discriminative power of the features used.

In classification based on prototypes, the objective is to make the features so unique and easily detectable that classification itself becomes a simple task. A good example of this are bank-check processors, which use stylized font styles to simplify machine processing (we will discuss this application in Section 12.3).

In the second category, classification is cast in decision-theoretic, statistical terms, and the classification approach is based on selecting parameters that can be shown to yield optimum classification performance in a statistical sense. Here, emphasis is placed on both the features used, and the design of the classifier. We will illustrate this approach in Section 12.4 by deriving the Bayes pattern classifier, starting from basic principles.

In the third category, classification is performed using neural networks. As you will learn in Sections 12.5 and 12.6, neural networks can operate using engineered features too, but they have the unique ability of being able to generate, on their own, representations (features) suitable for recognition. These systems can accomplish this using raw data, without the need for engineered features.

One characteristic shared by the preceding three approaches is that they are based on parameters that must be either specified or learned from patterns that represent the recognition problem we want to solve. The patterns can be *labeled*, meaning that we know the class of each pattern, or *unlabeled*, meaning that the data are known to be patterns, but the class of each pattern is unknown. A classic example of labeled data is the character recognition problem, in which a set of character samples is collected and the identity of each character is recorded as a label from the group 0 through 9 and *a* through *z*. An example of unlabeled data is when we are seeking clusters in a data set, with the aim of utilizing the resulting cluster centers as being prototypes of the pattern classes contained in the data.

When working with a labeled data, a given data set generally is subdivided into three subsets: a *training set*, a *validation set*, and a *test set* (a typical subdivision might be 50% training, and 25% each for the validation and test sets). The process by which a training set is used to generate classifier parameters is called *training*. In this mode, a classifier is given the class label of each pattern, the objective being to make adjustments in the parameters if the classifier makes a mistake in identifying the class of the given pattern. At this point, we might be working with several candidate designs. At the end of training, we use the validation set to compare the various designs against a performance objective. Typically, several iterations of training/validation are required to establish the design that comes closest to meeting the desired objective. Once a design has been selected, the final step is to determine how it will perform “in the field.” For this, we use the test set, which consists of patterns that the system has never “seen” before. If the training and validation sets are truly representative of the data the system will encounter in practice, the results of training/validation should be close to the performance using the test set. If training/validation results are acceptable, but test results are not, we say that training/validation “over fit” the system parameters to the available data, in which case further work on the system architecture is required. Of course all this assumes that the given data are truly representative of the problem we want to solve, and that the problem in fact can be solved by available technology.

A system that is designed using training data is said to undergo *supervised learning*. If we are working with unlabeled data, the system learns the pattern classes themselves while in an *unsupervised* learning mode. In this chapter, we deal only with supervised learning. As you will see in this and the next chapter, supervised learning covers a broad range of approaches, from applications in which a system learns parameters of features whose form is fixed by a designer, to systems that utilize *deep learning* and large sets of raw data sets to learn, *on their own*, the features required for classification. These systems accomplish this task without a human designer having to specify the features, *a priori*.

After a brief discussion in the next section of how patterns are formed, and on the nature of patterns classes, we will discuss in Section 12.3 various approaches for prototype-based classification. In Section 12.4, we will start from basic principles and derive the equations of the Bayes classifier, an approach characterized by optimum classification performance on an average basis. We will also discuss supervised training of a Bayes classifier based on the assumption of multivariate Gaussian

Because the examples in this chapter are intended to demonstrate basic principles and are not large scale, we dispense with validation and subdivide the pattern data into training and test sets.

Generally, we associate the concept of deep learning with large sets of data. These ideas are discussed in more detail later in this section and next.

distributions. Starting with Section 12.5, we will spend the rest of the chapter discussing neural networks. We will begin Section 12.5 with a brief introduction to perceptrons and some historical facts about machine learning. Then, we will introduce the concept of deep neural networks and derive the equations of backpropagation, the method of choice for training deep neural nets. These networks are well-suited for applications in which input patterns are vectors. In Section 12.6, we will introduce deep convolutional neural networks, which currently are the preferred approach when the system inputs are digital images. After deriving the backpropagation equations used for training convolutional nets, we will give several examples of applications involving classes of images of various complexities. In addition to working directly with image inputs, deep convolutional nets are capable of learning, on their own, image features suitable for classification. This is accomplished starting with raw image data, as opposed to the other classification methods discussed in Sections 12.3 and 12.4, which rely on “engineered” features whose form, as noted earlier, is specified a priori by a human designer.

12.2 PATTERNS AND PATTERN CLASSES

In image pattern classification, the two principal pattern arrangements are quantitative and structural. *Quantitative patterns* are arranged in the form of *pattern vectors*. *Structural patterns* typically are composed of symbols, arranged in the form of *strings*, *trees*, or, less frequently, as *graphs*. Most of the work in this chapter is based on pattern vectors, but we will discuss structural patterns briefly at the end of this section, and give an example at the end of Section 12.3.

PATTERN VECTORS

Pattern vectors are represented by lowercase letters, such as $\mathbf{x}$, $\mathbf{y}$, and $\mathbf{z}$, and have the form

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} \quad (12-1)$$

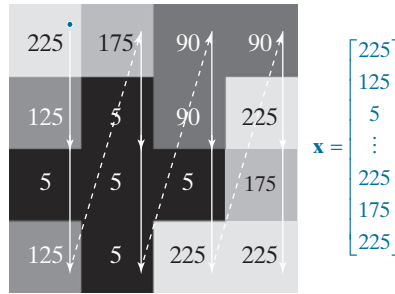
where each component, x_i , represents the i th feature descriptor, and n is the total number of such descriptors. We can express a vector in the form of a column, as in Eq. (12-1), or in the equivalent row form $\mathbf{x} = (x_1, x_2, \dots, x_n)^T$, where T indicates transposition. A pattern vector may be “viewed” as a point in n -dimensional Euclidean space, and a pattern class may be interpreted as a “hypercloud” of points in this *pattern space*. For the purpose of recognition, we like for our pattern classes to be grouped tightly, and as far away from each other as possible.

Pattern vectors can be formed directly from image pixel intensities by vectorizing the image using, for example, linear indexing, as in Fig. 12.1. A more common approach is for pattern elements to be features. An early example is the work of Fisher [1936] who, close to a century ago, reported the use of what then was a new

a b

FIGURE 12.1

Using linear indexing to vectorize a grayscale image.



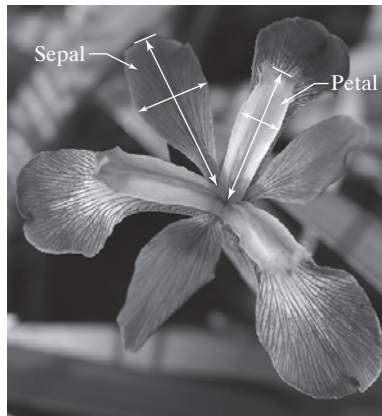
Sepals are the undergrowth beneath the petals.

technique called *discriminant analysis* to recognize three types of iris flowers (*Iris setosa*, *virginica*, and *versicolor*). Fisher described each flower using four features: the length and width of the petals, and similarly for the sepals (see Fig. 12.2). This leads to the 4-D vectors shown in the figure. A set of these vectors, obtained for fifty samples of each flower gender, constitutes the three famous *Fisher iris pattern classes*. Had Fisher been working today, he probably would have added spectral colors and shape features to his measurements, yielding vectors of higher dimensionality. We will be working with the original iris data set later in this chapter.

A higher-level representation of patterns is based on feature descriptors of the types you learned in Chapter 11. For instance, pattern vectors formed from descriptors of boundary shape are well-suited for applications in controlled environments, such as industrial inspection. Figure 12.3 illustrates the concept. Here, we are interested in classifying different types of noisy shapes, a sample of which is shown in the figure. If we represent an object by its signature, we would obtain 1-D signals of the form shown in Fig. 12.3(b). We can express a signature as a vector by sampling its amplitude at increments of θ , then forming a vector by letting $x_i = r(\theta_i)$, for $i = 0, 1, 2, \dots, n$. Instead of using “raw” sampled signatures, a more common approach is to compute some function, $x_i = g(r(\theta_i))$, of the signature samples and use them to form vectors. You learned in Section 11.3 several approaches to do this, such as statistical moments.

FIGURE 12.2

Petal and sepal width and length measurements (see arrows) performed on iris flowers for the purpose of data classification. The image shown is of the *Iris virginica* gender. (Image courtesy of USDA.)

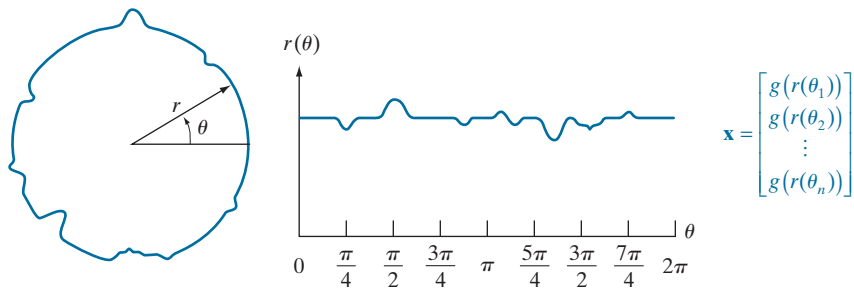


$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix}$$

x_1 = Petal width
 x_2 = Petal length
 x_3 = Sepal width
 x_4 = Sepal length

a b

FIGURE 12.3
(a) A noisy object boundary, and (b) its corresponding signature.



Vectors can be formed also from features of both boundary and regions. For example, the objects in Fig. 12.4 can be represented by 3-D vectors whose components capture shape information related to both boundary and region properties of single binary objects. Pattern vectors can be used also to represent properties of image regions. For example, the elements of the 6-D vector in Fig. 12.5 are texture measures based on the feature descriptors in Table 11.3. Figure 12.6 shows an example in which pattern vector elements are features that are invariant to transformations, such as image rotation and scaling (see Section 11.4).

When working with sequences of registered images, we have the option of using pattern vectors formed from corresponding pixels in those images (see Fig. 12.7). Forming pattern vectors in this way implies that recognition will be based on information extracted from the same spatial location across the images. Although this may seem like a very limiting approach, it is ideally suited for applications such as recognizing regions in multispectral images, as you will see in Section 12.4.

When working with entire images as units, we need the detail afforded by vectors of much-higher dimensionality, such as those we discussed in Section 11.7 in connection with the SIFT algorithm. However, a more powerful approach when working with entire images is to use deep convolutional neural networks. We will discuss neural nets in detail in Sections 12.5 and 12.6.

STRUCTURAL PATTERNS

Pattern vectors are not suitable for applications in which objects are represented by structural features, such as strings of symbols. Although they are used much less than vectors in image processing applications, patterns containing structural descriptions of objects are important in applications where shape is of interest. Figure 12.8 shows an example. The boundaries of the bottles were approximated by a polygon

a b c d

FIGURE 12.4
Pattern vectors whose components capture both boundary and regional characteristics.

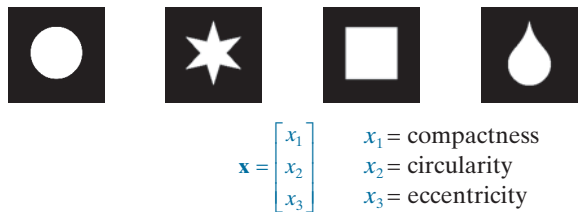


FIGURE 12.5
An example of pattern vectors based on properties of subimages. See Table 11.3 for an explanation of the components of $\mathbf{x}$.

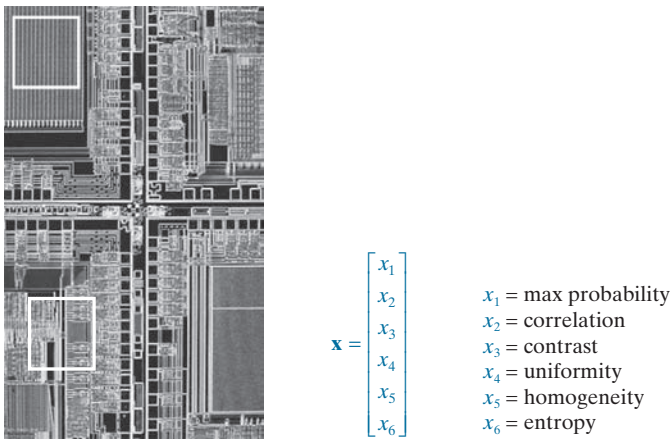


FIGURE 12.6 Feature vectors with components that are invariant to transformations such as rotation, scaling, and translation. The vector components are moment invariants.

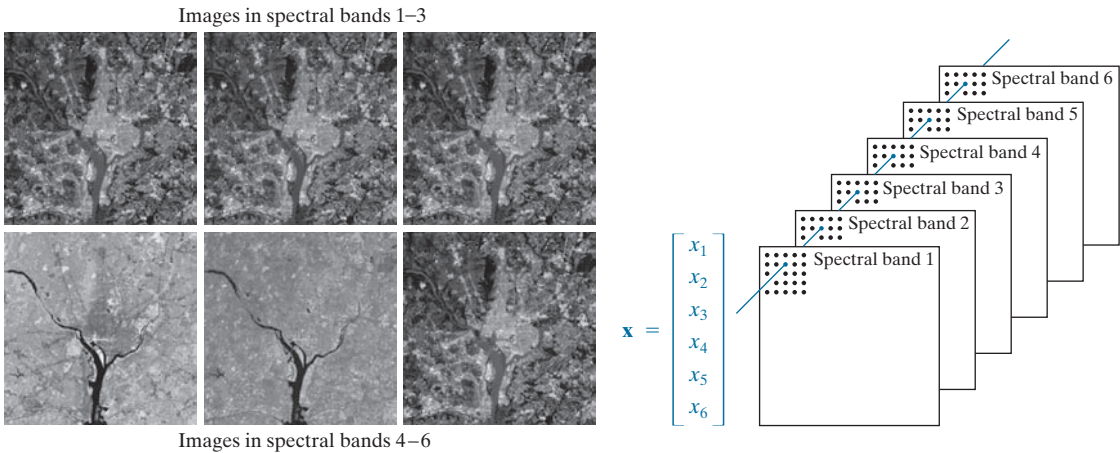
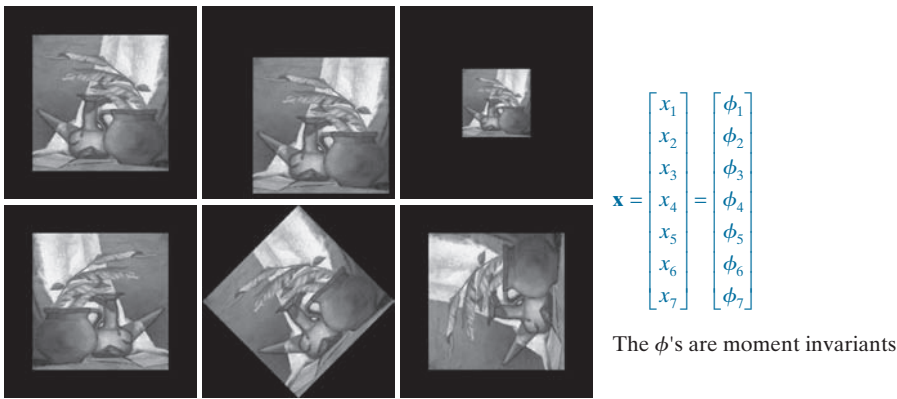
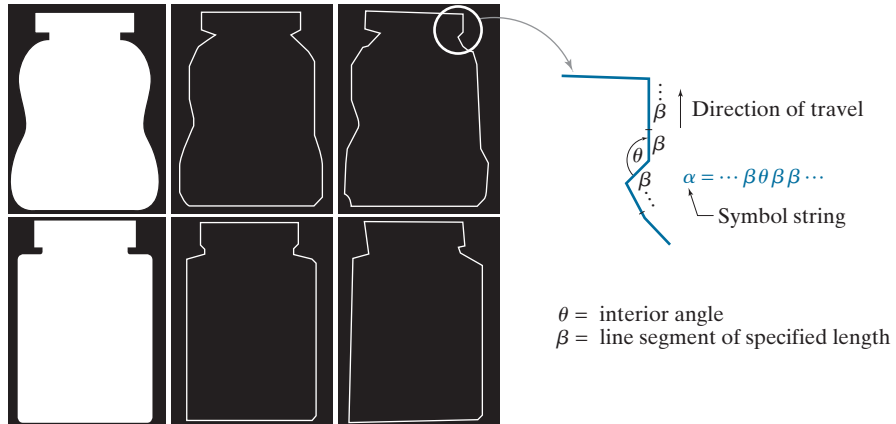


FIGURE 12.7 Pattern (feature) vectors formed by concatenating corresponding pixels from a set of registered images. (Original images courtesy of NASA.)

FIGURE 12.8

Symbol string generated from a polygonal approximation of the boundaries of medicine bottles.



using the approach explained in Section 11.2. The boundary is subdivided into line segments (denoted by β in the figure), and the interior angle, θ , is computed at each intersection of two line segments. A string of sequential symbols is generated as the boundary is traversed in the counterclockwise direction, as the figure shows. *Strings* of this form are structural patterns, and the objective, as you will see in Section 12.3, is to match a given string against stored string prototypes.

A *tree* is another structural representation, suitable for higher-level descriptions of an entire image in terms of its component regions. Basically, most hierarchical ordering schemes lead to tree structures. For example, Fig. 12.9 shows a satellite image of a heavily built downtown area and surrounding residential areas. Let the symbol \$ represent the root of a tree. The (upside down) tree shown in the figure was obtained using the structural relationship “composed of.” Thus, the root of the tree represents the entire image. The next level indicates that the image is composed of a downtown and residential areas. In turn, the residential areas are composed of housing, highways, and shopping malls. The next level down in the tree further describes the housing and highways. We can continue this type of subdivision until we reach the limit of our ability to resolve different regions in the image.

12.3 PATTERN CLASSIFICATION BY PROTOTYPE MATCHING

Prototype matching involves comparing an unknown pattern against a set of prototypes, and assigning to the unknown pattern the class of the prototype that is the most “similar” to the unknown. Each prototype represents a unique pattern class, but there may be more than one prototype for each class. What distinguishes one matching method from another is the measure used to determine similarity.

MINIMUM-DISTANCE CLASSIFIER

One of the simplest and most widely used prototype matching methods is the *minimum-distance classifier* which, as its name implies, computes a distance-based measure between an unknown pattern vector and each of the class prototypes. It then assigns the unknown pattern to the class of its closest prototype. The prototype

The minimum-distance classifier is also referred to as the *nearest-neighbor classifier*.

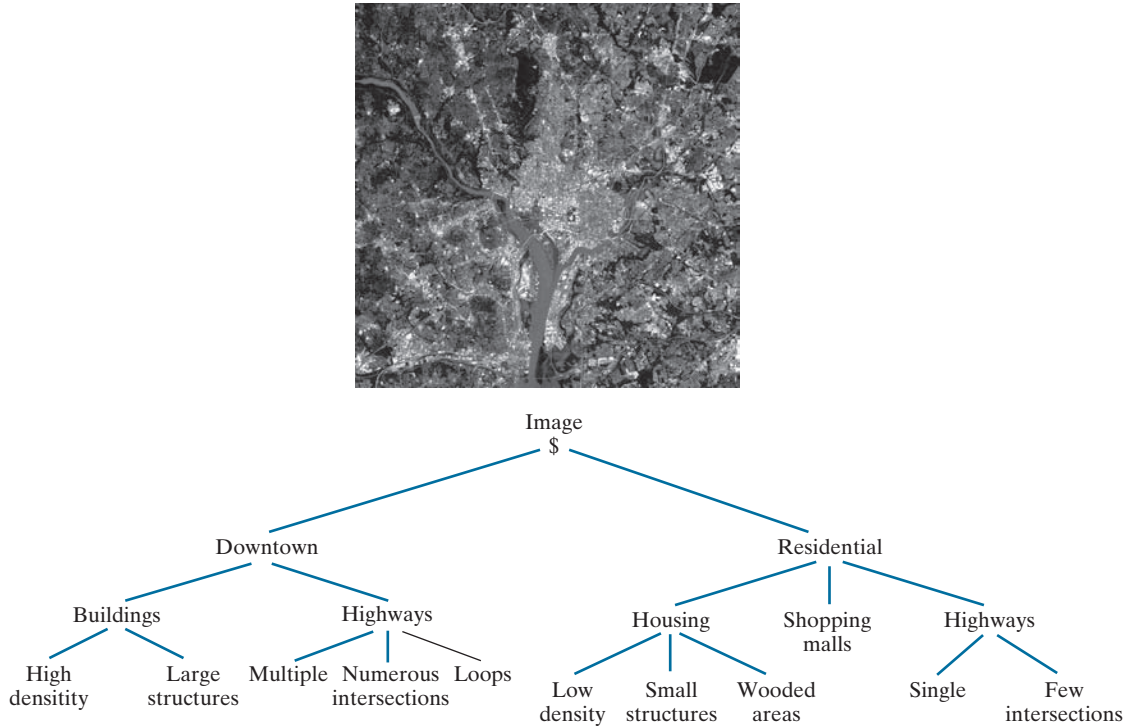


FIGURE 12.9 Tree representation of a satellite image showing a heavily built downtown area (Washington, D.C.) and surrounding residential areas. (Original image courtesy of NASA.)

vectors of the minimum-distance classifier usually are the mean vectors of the various pattern classes:

$$\mathbf{m}_j = \frac{1}{n_j} \sum_{\mathbf{x} \in c_j} \mathbf{x} \quad j = 1, 2, \dots, N_c \quad (12-2)$$

where n_j is the number of pattern vectors used to compute the j th mean vector, c_j is the j th pattern class, and N_c is the number of classes. If we use the Euclidean distance to determine similarity, the minimum-distance classifier computes the distances

$$D_j(\mathbf{x}) = \|\mathbf{x} - \mathbf{m}_j\| \quad j = 1, 2, \dots, N_c \quad (12-3)$$

where $\|\mathbf{a}\| = (\mathbf{a}^T \mathbf{a})^{1/2}$ is the Euclidean norm. The classifier then assigns an unknown pattern $\mathbf{x}$ to class c_i if $D_i(\mathbf{x}) < D_j(\mathbf{x})$ for $j = 1, 2, \dots, N_c$, $j \neq i$. Ties [i.e., $D_i(\mathbf{x}) = D_j(\mathbf{x})$] are resolved arbitrarily.

It is not difficult to show (see Problem 12.2) that selecting the smallest distance is equivalent to evaluating the functions

$$d_j(\mathbf{x}) = \mathbf{m}_j^T \mathbf{x} - \frac{1}{2} \mathbf{m}_j^T \mathbf{m}_j \quad j = 1, 2, \dots, N_c \quad (12-4)$$

and assigning an unknown pattern $\mathbf{x}$ to the class whose prototype yielded the *largest* value of d . That is, $\mathbf{x}$ is assigned to class c_i , if

$$d_i(\mathbf{x}) > d_j(\mathbf{x}) \quad j = 1, 2, \dots, N_c; \quad j \neq i \quad (12-5)$$

When used for recognition, functions of this form are referred to as *decision* or *discriminant functions*.

The *decision boundary* separating class c_i from c_j is given by the values of $\mathbf{x}$ for which

$$d_i(\mathbf{x}) = d_j(\mathbf{x}) \quad (12-6)$$

or, equivalently, by values of $\mathbf{x}$ for which

$$d_i(\mathbf{x}) - d_j(\mathbf{x}) = 0 \quad (12-7)$$

The decision boundaries for a minimum-distance classifier follow directly from this equation and Eq. (12-4):

$$\begin{aligned} d_{ij}(\mathbf{x}) &= d_i(\mathbf{x}) - d_j(\mathbf{x}) \\ &= (\mathbf{m}_i - \mathbf{m}_j)^T \mathbf{x} - \frac{1}{2}(\mathbf{m}_i - \mathbf{m}_j)^T (\mathbf{m}_i + \mathbf{m}_j) = 0 \end{aligned} \quad (12-8)$$

The boundary given by Eq. (12-8) is the perpendicular bisector of the line segment joining $\mathbf{m}_i$ and $\mathbf{m}_j$ (see Problem 12.3). In 2-D (i.e., $n = 2$), the perpendicular bisector is a line, for $n = 3$ it is a plane, and for $n > 3$ it is called a *hyperplane*.

EXAMPLE 12.1: Illustration of the minimum-distance classifier for two classes in 2-D.

Figure 12.10 shows scatter plots of petal width and length values for the classes Iris versicolor and Iris setosa. As mentioned in the previous section, pattern vectors in the iris database consists of four measurements for each flower. We show only two here so that you can visualize the pattern classes and the decision boundary between them. We will work with the complete database later in this chapter.

We denote the Iris versicolor and setosa data as classes c_1 and c_2 , respectively. The means of the two classes are $\mathbf{m}_1 = (4.3, 1.3)^T$ and $\mathbf{m}_2 = (1.5, 0.3)^T$. It then follows from Eq. (12-4) that

$$\begin{aligned} d_1(\mathbf{x}) &= \mathbf{m}_1^T \mathbf{x} - \frac{1}{2} \mathbf{m}_1^T \mathbf{m}_1 \\ &= 4.3x_1 + 1.3x_2 - 10.1 \end{aligned}$$

and

$$\begin{aligned} d_2(\mathbf{x}) &= \mathbf{m}_2^T \mathbf{x} - \frac{1}{2} \mathbf{m}_2^T \mathbf{m}_2 \\ &= 1.5x_1 + 0.3x_2 - 1.17 \end{aligned}$$

From Eq. (12-8), the equation of the boundary is

$$\begin{aligned} d_{12}(\mathbf{x}) &= d_1(\mathbf{x}) - d_2(\mathbf{x}) \\ &= 2.8x_1 + 1.0x_2 - 8.9 = 0 \end{aligned}$$

FIGURE 12.10

Decision boundary of a minimum distance classifier (based on two measurements) for the classes of *Iris versicolor* and *Iris setosa*. The dark dot and square are the means of the two classes.

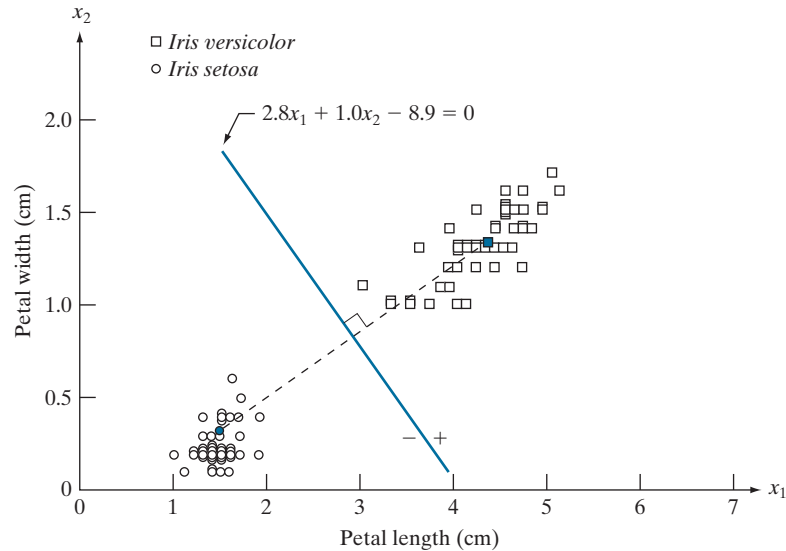


Figure 12.10 shows a plot of this boundary. Substituting any pattern vector from class c_1 into this equation would yield $d_{12}(\mathbf{x}) > 0$. Conversely, any pattern from class c_2 would give $d_{12}(\mathbf{x}) < 0$. Thus, given an unknown pattern $\mathbf{x}$ belonging to one of these two classes, the sign of $d_{12}(\mathbf{x})$ would be sufficient to determine the class to which that pattern belongs.

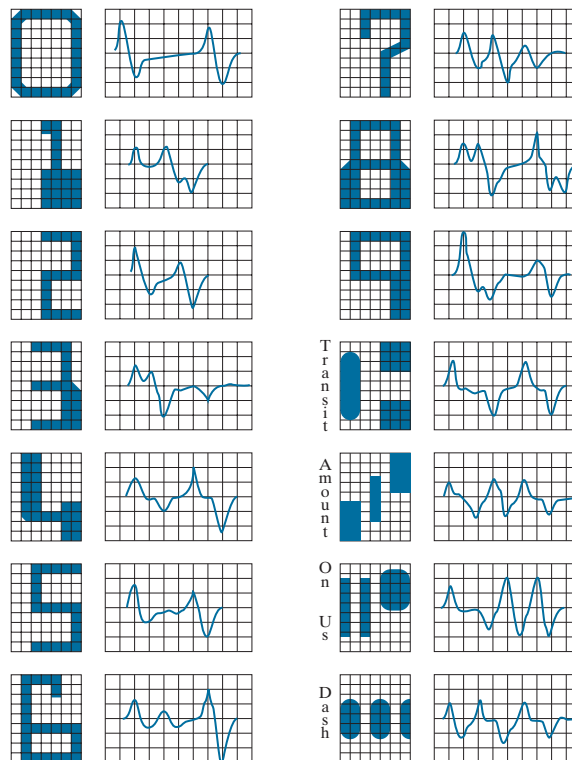
The minimum-distance classifier works well when the distance between means is large compared to the spread or randomness of each class with respect to its mean. In Section 12.4 we will show that the minimum-distance classifier yields optimum performance (in terms of minimizing the average loss of misclassification) when the distribution of each class about its mean is in the form of a spherical “hypercloud” in n -dimensional pattern space.

As noted earlier, one of the keys to accurate recognition performance is to specify features that are effective discriminators between classes. As a rule, the better the features are at meeting this objective, the better the recognition performance will be. In the case of the minimum-distance classifier this implies wide separation between means and tight grouping of the classes.

Systems based on the Banker’s Association E-13B font character are a classic example of how highly engineered features can be used in conjunction with a simple classifier to achieve superior results. In the mid-1940s, bank checks were processed manually, which was a laborious, costly process prone to mistakes. As the volume of check writing increased in the early 1950s, banks became keenly interested in automating this task. In the middle 1950s, the E-13B font and the system that reads it became the standard solution to the problem. As Fig. 12.11 shows, this font set consists of 14 characters laid out on a 9×7 grid. The characters are stylized to maximize the difference between them. The font was designed to be compact and readable by humans, but the overriding purpose was that the characters should be readable by machine, quickly, and with very high accuracy.

FIGURE 12.11

The American Bankers Association E-13B font character set and corresponding waveforms.



In addition to a stylized font design, the operation of the reading system is further enhanced by printing each character using an ink that contains finely ground magnetic material. To improve character detectability in a check being read, the ink is subjected to a magnetic field that accentuates each character against the background. The stylized design further enhances character detectability. The characters are scanned in a horizontal direction with a single-slit reading head that is narrower but taller than the characters. As a check passes through the head, the sensor produces a 1-D electrical signal (a signature) that is conditioned to be proportional to the rate of increase or decrease of the character area under the head. For example, consider the waveform of the number 0 in Fig. 12.11. As a check moves to the right past the head, the character area seen by the sensor begins to increase, producing a positive derivative (a positive rate of change). As the right leg of the character begins to pass under the head, the character area seen by the sensor begins to decrease, producing a negative derivative. When the head is in the middle zone of the character, the area remains nearly constant, producing a zero derivative. This waveform repeats itself as the other leg of the character enters the head. The design of the font ensures that the waveform of each character is distinct from all others. It also ensures that the peaks and zeros of each waveform occur approximately on the vertical lines of the background grid on which these waveforms are displayed, as the figure shows. The E-13B font has the property that sampling the waveforms only at these (nine)

Appropriately, recognition of magnetized characters is referred to as *Magnetic Ink Character Recognition (MICR)*.

points yields enough information for their accurate classification. The effectiveness of these highly engineered features is further refined by the magnetized ink, which results in clean waveforms with almost no scatter.

Designing a minimum-distance classifier for this application is straightforward. We simply store the sample values of each waveform at the vertical lines of the grid, and let each set of the resulting samples be represented as a 9-D prototype vector, $\mathbf{m}_j$, $j = 1, 2, \dots, 14$. When an unknown character is to be classified, the approach is to scan it in the manner just described, express the grid samples of the waveform as a 9-D vector, $\mathbf{x}$, and identify its class by selecting the class of the prototype vector that yields the highest value in Eq. (12-4). We do not even need a computer to do this. Very high classification speeds can be achieved with analog circuits composed of resistor banks (see Problem 12.4).

The most important lesson in this example is that a recognition problem often can be made trivial if we can control the environment in which the patterns are generated. The development and implementation of the E13-B font reading system is a striking example of this fact. On the other hand, this system would be inadequate if we added the requirement that it has to recognize the textual content and signature written on each check. For this, we need systems that are significantly more complex, such as the convolutional neural networks we will discuss in Section 12.6.

USING CORRELATION FOR 2-D PROTOTYPE MATCHING

We introduced the basic idea of spatial correlation and convolution in Section 3.4, and used these concepts extensively in Chapter 3 for spatial filtering. From Eq. (3-34), we know that correlation of a kernel w with an image $f(x, y)$ is given by

$$(w \star f)(x, y) = \sum_s \sum_t w(s, t) f(x + s, y + t) \quad (12-9)$$

where the limits of summation are taken over the region shared by w and f . This equation is evaluated for all values of the displacement variables x and y so all elements of w visit every pixel of f . As you know, correlation has its highest value(s) in the region(s) where f and w are equal or nearly equal. In other words, Eq. (12-9) finds locations where w matches a region of f . But this equation has the drawback that the result is sensitive to changes in the amplitude of either function. In order to normalize correlation to amplitude changes in one or both functions, we perform matching using the *correlation coefficient* instead:

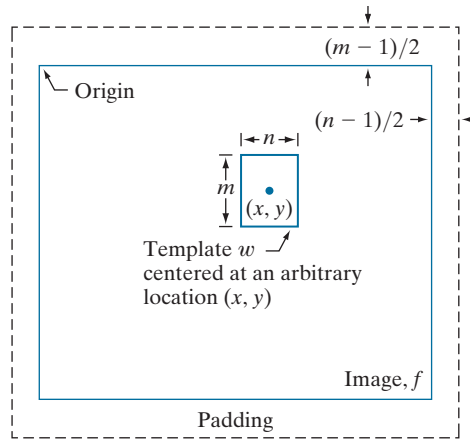
$$\gamma(x, y) = \frac{\sum_s \sum_t [w(s, t) - \bar{w}] [f(x + s, y + t) - \bar{f}_{xy}]}{\left\{ \sum_s \sum_t [w(s, t) - \bar{w}]^2 \sum_s \sum_t [f(x + s, y + t) - \bar{f}_{xy}]^2 \right\}^{\frac{1}{2}}} \quad (12-10)$$

where the limits of summation are taken over the region shared by w and f , $\bar{w}$ is the average value of the kernel (computed only once), and $\bar{f}_{xy}$ is the average value of f in the region coincident with w . In image correlation work, w is often referred to as a *template* (i.e., a prototype subimage) and correlation is referred to as *template matching*.

To be formal, we should refer to correlation (and the correlation coefficient) as *cross-correlation* when the functions are different, and as *autocorrelation* when they are the same. However, it is customary to use the generic term *correlation* and *correlation coefficient*, except when the distinction is important (as in deriving equations, in which it makes a difference which is being applied).

FIGURE 12.12

The mechanics of template matching.



It can be shown (see Problem 12.5) that $\gamma(x, y)$ has values in the range $[-1, 1]$ and is thus normalized to changes in the amplitudes of w and f . The maximum value of γ occurs when the normalized w and the corresponding normalized region in f are identical. This indicates *maximum correlation* (the best possible match). The *minimum* occurs when the two normalized functions exhibit the least similarity in the sense of Eq. (12-10).

Figure 12.12 illustrates the mechanics of the procedure just described. The border around image f is padding, as explained in Section 3.4. In template matching, values of correlation when the center of the template is past the border of the image generally are of no interest, so the padding is limited to half the kernel width.

The template in Fig. 12.12 is of size $m \times n$, and it is shown with its center at an arbitrary location (x, y) . The value of the correlation coefficient at that point is computed using Eq. (12-10). Then, the center of the template is incremented to an adjacent location and the procedure is repeated. Values of the correlation coefficient $\gamma(x, y)$ are obtained by moving the center of the template (i.e., by incrementing x and y) so the center of w visits every pixel in f . At the end of the procedure, we look for the maximum in $\gamma(x, y)$ to find where the best match occurred. It is possible to have multiple locations in $\gamma(x, y)$ with the same maximum value, indicating several matches between w and f .

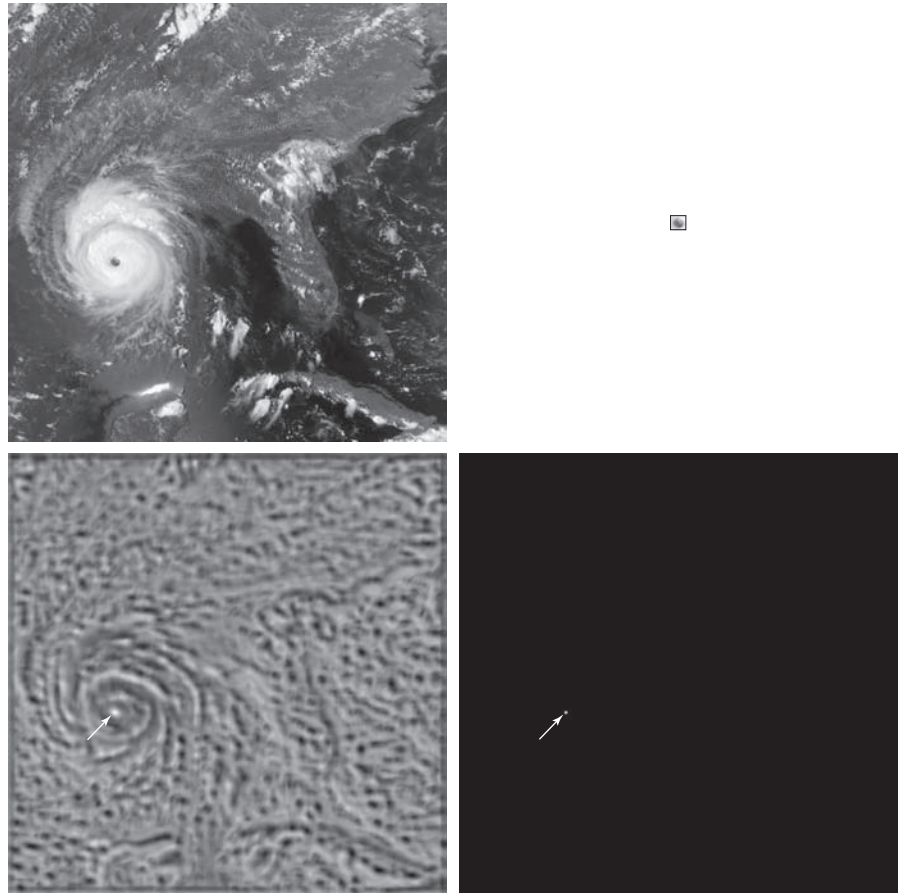
EXAMPLE 12.2: Matching by correlation.

Figure 12.13(a) shows a 913×913 satellite image of 1992 Hurricane Andrew, in which the eye of the storm is clearly visible. We want to use correlation to find the location of the best match in Fig. 12.13(a) of the template in Fig. 12.13(b), which is a 31×31 subimage of the eye of the storm. Figure 12.13(c) shows the result of computing the correlation coefficient in Eq. (12-10) for all values of x and y in the original image. The size of this image was 943×943 pixels due to padding (see Fig. 12.12), but we cropped it to the size of the original image for display. The intensity in this image is proportional to the correlation values, and all negative correlations were clipped at 0 (black) to simplify the visual analysis of the image. The area of highest correlation values appears as a small white region in this image. The brightest point in this region matches with the center of the eye of the storm. Figure 12.13(d) shows as a

a	b
c	d

FIGURE 12.13

(a) 913×913 satellite image of Hurricane Andrew.
 (b) 31×31 template of the eye of the storm.
 (c) Correlation coefficient shown as an image (note the brightest point, indicated by an arrow).
 (d) Location of the best match (identified by the arrow). This point is a single pixel, but its size was enlarged to make it easier to see. (Original image courtesy of NOAA.)



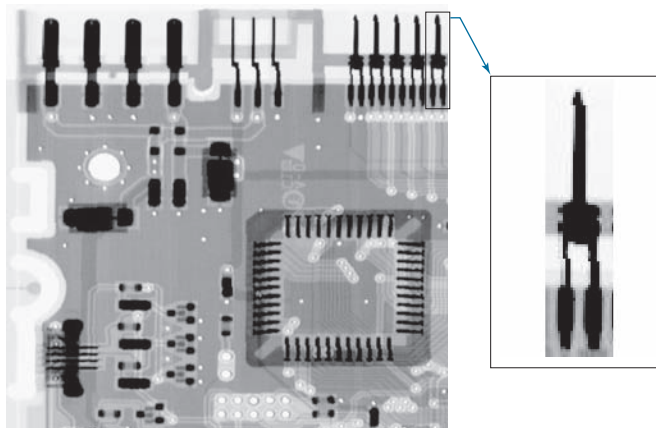
white dot the location of this maximum correlation value (in this case there was a unique match whose maximum value was 1), which we see corresponds closely with the location of the eye in Fig. 12.13(a).

MATCHING SIFT FEATURES

We discussed the scale-invariant feature transform (SIFT) in Section 11.7. SIFT computes a set of invariant features that can be used for matching between known (prototype) and unknown images. The SIFT implementation in Section 11.7 yields 128-dimensional feature vectors for each local region in an image. SIFT performs matching by looking for correspondences between sets of stored feature vector prototypes and feature vectors computed for an unknown image. Because of the large number of features involved, searching for exact matches is computationally intensive. Instead, the approach is to use a best-bin-first method that can identify the nearest neighbors with high probability using only a limited amount of computation (see Lowe [1999], [2004]). The search is further simplified by looking for clusters of potential solutions using the generalized Hough transform proposed by Ballard [1981]. We

FIGURE 12.14

Circuit board image of size 948×915 pixels, and a subimage of one of the connectors. The subimage is of size 212×128 pixels, shown zoomed on the right for clarity. (Original image courtesy of Mr. Joseph E. Pascente, Lixi, Inc.)



know from the discussion in Section 10.2 that the Hough transform simplifies looking for data patterns by utilizing bins that reduce the level of detail with which we look at a data set. We already discussed the SIFT algorithm in Section 11.7. The focus in this section is to further illustrate the capabilities of SIFT for prototype matching.

Figure 12.14 shows the circuit board image we have used several times before. The small rectangle enclosing the rightmost connector on the top of the large image identifies an area from which an image of the connector was extracted. The small image is shown zoomed for clarity. The sizes of the large and small images are shown in the figure caption. Figure 12.15 shows the keypoints found by SIFT, as explained in Section 11.7. They are visible as faint gray lines on both images. The zoomed view of the subimage shows them a little clearer. It is important to note that the keypoints for the image and subimage were found independently by SIFT. The large image had 2714 keypoints, and the small image had 35.

Figure 12.16 shows the matches between keypoints found by SIFT. A total of 41 matches were found between the two images. Because there are only 35 keypoints

FIGURE 12.15

Keypoints found by SIFT. The large image has 2714 keypoints (visible as faint gray lines). The subimage has 35 keypoints. This is a separate image, and SIFT found its keypoints independently of the large image. The zoomed section is shown for clarity.

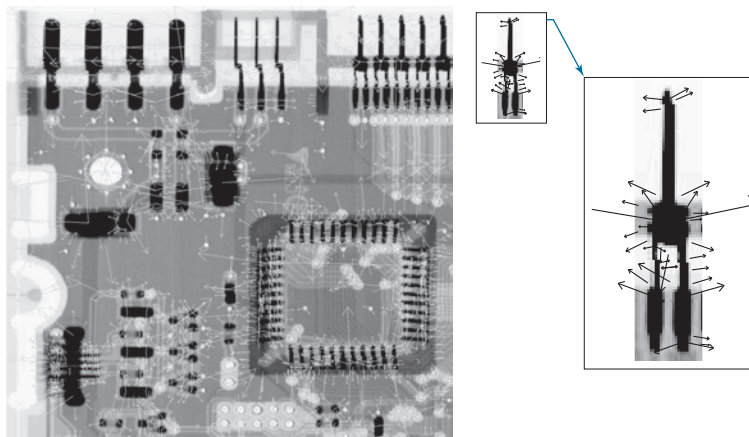
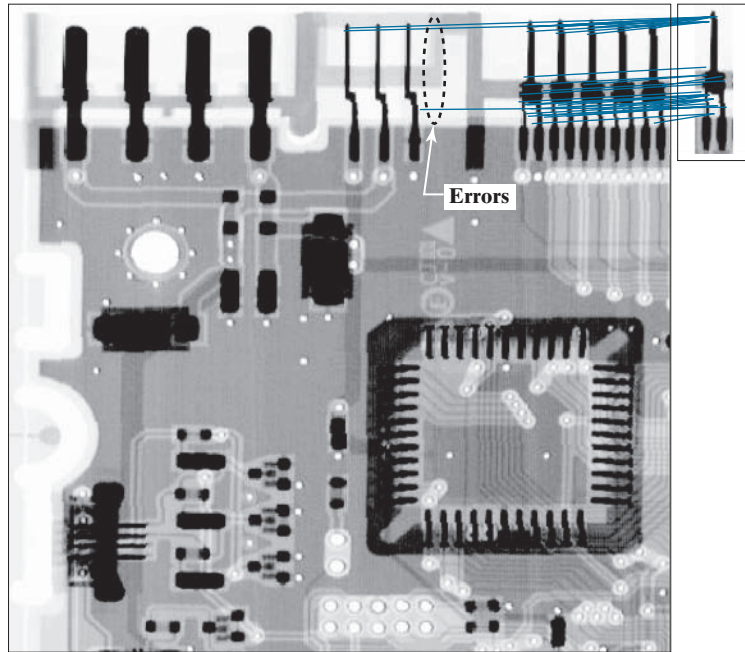


FIGURE 12.16

Matches found by SIFT between the large and small images. A total of 41 matching pairs were found. They are shown connected by straight lines. Only three of the matches were “real” errors (labeled “Errors” in the figure).



in the small image, obviously at least six matches are either incorrect, or there are multiple matches. Three of the errors are clearly visible as matches with connectors in the middle of the large image. However, if you compare the shape of the connectors in the middle of the large image, you can see that they are virtually identical to *parts* of the connectors on the right. Therefore, these errors can be explained on that basis. The other three extra matches are easier to explain. All connectors on the top right of the circuit board are identical, and we are comparing one of them against the rest. There is no way for a system to tell the difference between them. In fact, by looking at the connecting lines, we can see that the matches are between the subimage and all five connectors. These in fact are correct matches between the subimage and other connectors that are identical to it.

MATCHING STRUCTURAL PROTOTYPES

The techniques discussed up to this point deal with patterns quantitatively, and largely ignore any structural relationships inherent in pattern shapes. The methods discussed in this section seek to achieve pattern recognition by capitalizing precisely on these types of relationships. In this section, we introduce two basic approaches for the recognition of boundary shapes based on string representations, which are the most practical approach in structural pattern recognition.

Matching Shape Numbers

A procedure similar in concept to the minimum-distance classifier introduced earlier for pattern vectors can be formulated for comparing region boundaries that are

described by shape numbers. With reference to the discussion in Section 11.3, the *degree of similarity*, k , between two region boundaries, is defined as the largest order for which their shape numbers still coincide. For example, let a and b denote shape numbers of closed boundaries represented by 4-directional chain codes. These two shapes have a degree of similarity k if

Parameter j starts at 4 and is always even because we are working with 4-connectivity, and we require that boundaries be closed.

$$\begin{aligned} s_j(a) &= s_j(b) & \text{for } j = 4, 6, 8, \dots, k; \text{ and} \\ s_j(a) &\neq s_j(b) & \text{for } j = k + 2, k + 4, \dots \end{aligned} \quad (12-11)$$

where s indicates shape number, and the subscript indicates shape order. The *distance* between two shapes a and b is defined as the inverse of their degree of similarity:

$$D(a, b) = \frac{1}{k} \quad (12-12)$$

This expression satisfies the following properties:

$$\begin{aligned} D(a, b) &\geq 0 \\ D(a, b) &= 0 \quad \text{if and only if } a = b \\ D(a, c) &\leq \max[D(a, b), D(b, c)] \end{aligned} \quad (12-13)$$

Either k or D may be used to compare two shapes. If the degree of similarity is used, the larger k is, the more similar the shapes are (note that k is infinite for identical shapes). The reverse is true when Eq. (12-12) is used.

EXAMPLE 12.3: Matching shape numbers.

Suppose we have a shape, f , and want to find its closest match in a set of five shape prototypes, denoted by a, b, c, d , and e , as shown in Fig. 12.17(a). The search may be visualized with the aid of the *similarity tree* in Fig. 12.17(b). The root of the tree corresponds to the lowest possible degree of similarity, which is 4. Suppose shapes are identical up to degree 8, with the exception of shape a , whose degree of similarity with respect to all other shapes is 6. Proceeding down the tree, we find that shape d has degree of similarity 8 with respect to all others, and so on. Shapes f and c match uniquely, having a higher degree of similarity than any other two shapes. Conversely, if a had been an unknown shape, all we could have said using this method is that a was similar to the other five shapes with degree of similarity 6. The same information can be summarized in the form of the *similarity matrix* in Fig. 12.17(c).

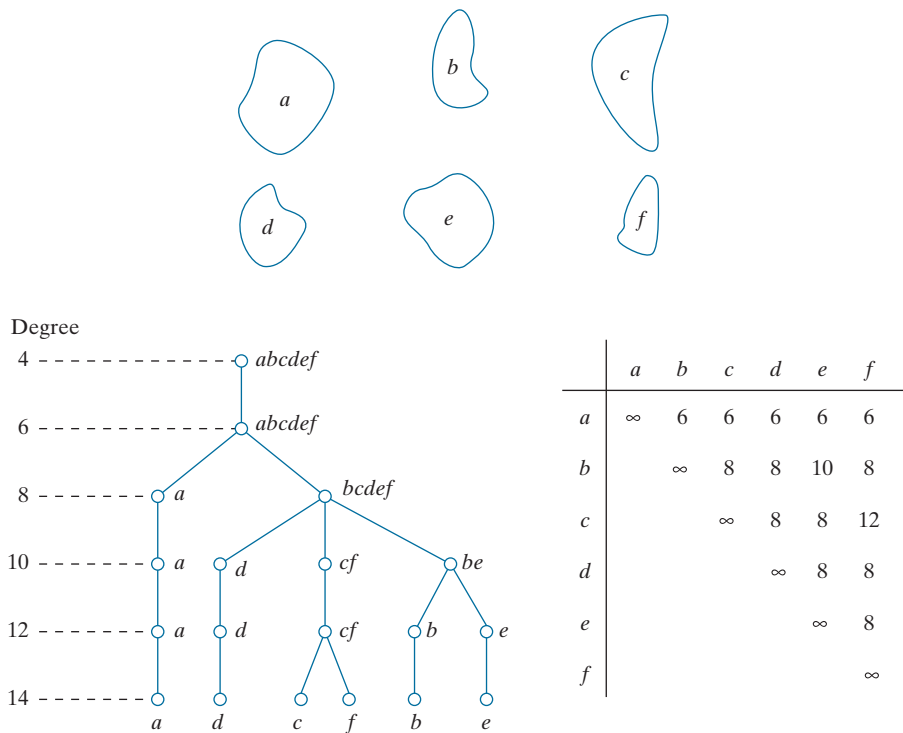
String Matching

Suppose two region boundaries, a and b , are coded into strings of symbols, denoted as $a_1 a_2 \dots a_n$ and $b_1 b_2 \dots b_m$, respectively. Let α represent the number of matches between the two strings, where a match occurs in the k th position if $a_k = b_k$. The number of symbols that do not match is

$$\beta = \max(|a|, |b|) - \alpha \quad (12-14)$$

a
b c

FIGURE 12.17
(a) Shapes.
(b) Similarity
tree. (c) Similarity
matrix.
(Bribiesca and
Guzman.)



where $|\arg|$ is the length (number of symbols) of string in the argument. It can be shown that $\beta = 0$ if and only if a and b are identical (see Problem 12.7).

An effective measure of similarity is the ratio

$$R = \frac{\alpha}{\beta} = \frac{\alpha}{\max(|a|, |b|) - \alpha} \quad (12-15)$$

We see that R is infinite for a perfect match and 0 when none of the corresponding symbols in a and b match ($\alpha = 0$ in this case). Because matching is done symbol by symbol, the starting point on each boundary is important in terms of reducing the amount of computation required to perform a match. Any method that normalizes to, or near, the same starting point is helpful if it provides a computational advantage over brute-force matching, which consists of starting at arbitrary points on each string, then shifting one of the strings (with wraparound) and computing Eq. (12-15) for each shift. The largest value of R gives the best match.

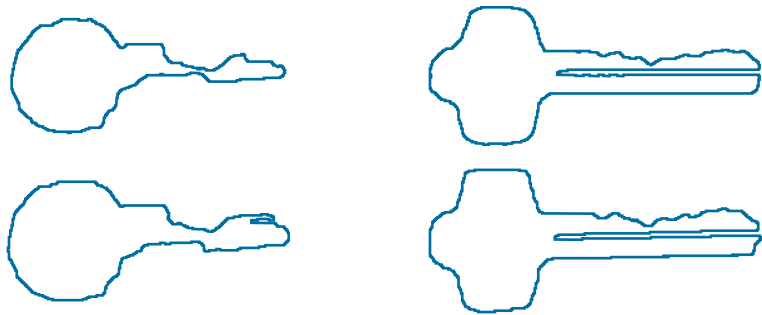
Refer to Section 11.2
for examples of how the
starting point of a curve
can be normalized.

EXAMPLE 12.4: String matching.

Figures 12.18(a) and (b) show sample boundaries from each of two object classes, which were approximated by a polygonal fit (see Section 11.2). Figures 12.18(c) and (d) show the polygonal approximations

a b
c d
e f
g

FIGURE 12.18
(a) and (b) sample boundaries of two different object classes; (c) and (d) their corresponding polygonal approximations; (e)–(g) tabulations of R .
(Sze and Yang.)



R	1.a	1.b	1.c	1.d	1.e	1.f
1.a	∞					
1.b	16.0	∞				
1.c	9.6	26.3	∞			
1.d	5.1	8.1	10.3	∞		
1.e	4.7	7.2	10.3	14.2	∞	
1.f	4.7	7.2	10.3	8.4	23.7	∞

R	2.a	2.b	2.c	2.d	2.e	2.f
2.a	∞					
2.b	33.5	∞				
2.c	4.8	5.8	∞			
2.d	3.6	4.2	19.3	∞		
2.e	2.8	3.3	9.2	18.3	∞	
2.f	2.6	3.0	7.7	13.5	27.0	∞

R	1.a	1.b	1.c	1.d	1.e	1.f
2.a	1.24	1.50	1.32	1.47	1.55	1.48
2.b	1.18	1.43	1.32	1.47	1.55	1.48
2.c	1.02	1.18	1.19	1.32	1.39	1.48
2.d	1.02	1.18	1.19	1.32	1.29	1.40
2.e	0.93	1.07	1.08	1.19	1.24	1.25
2.f	0.89	1.02	1.02	1.24	1.22	1.18

corresponding to the boundaries in Figs. 12.18(a) and (b), respectively. Strings were formed from the polygons by computing the interior angle, θ , between segments as each polygon was traversed clockwise. Angles were coded into one of eight possible symbols, corresponding to multiples of 45° ; that is, $\alpha_1 : 0^\circ < \theta \leq 45^\circ$; $\alpha_2 : 45^\circ < \theta \leq 90^\circ$; ...; $\alpha_8 : 315^\circ < \theta \leq 360^\circ$.

Figure 12.18(e) shows the results of computing the measure R for six samples of object 1 against themselves. The entries are values of R and, for example, the notation 1.c refers to the third string from object class 1. Figure 12.18(f) shows the results of comparing the strings of the second object class against themselves. Finally, Fig. 12.18(g) shows the R values obtained by comparing strings of one class against the other. These values of R are significantly smaller than any entry in the two preceding tabulations. This indicates that the R measure achieved a high degree of discrimination between the two classes of objects. For example, if the class of string 1.a had been unknown, the *smallest* value of R resulting from comparing this string against sample (prototype) strings of class 1 would have been 4.7 [see Fig. 12.18(e)]. By contrast, the *largest* value in comparing it against strings of class 2 would have been 1.24 [see Fig. 12.18(g)]. This result would have led to the conclusion that string 1.a is a member of object class 1. This approach to classification is analogous to the minimum-distance classifier introduced earlier.

12.4 OPTIMUM (BAYES) STATISTICAL CLASSIFIERS

In this section, we develop a probabilistic approach to pattern classification. As is true in most fields that deal with measuring and interpreting physical events, probability considerations become important in pattern recognition because of the randomness under which pattern classes normally are generated. As shown in the following discussion, it is possible to derive a classification approach that is optimal in the sense that, on average, it yields the lowest probability of committing classification errors (see Problem 12.12).

DERIVATION OF THE BAYES CLASSIFIER

The probability that a pattern vector $\mathbf{x}$ comes from class c_i is denoted by $p(c_i/\mathbf{x})$. If the pattern classifier decides that $\mathbf{x}$ came from class c_j when it actually came from c_i it incurs a *loss* (to be defined shortly), denoted by L_{ij} . Because pattern $\mathbf{x}$ may belong to any one of N_c possible classes, the average loss incurred in assigning $\mathbf{x}$ to class c_j is

$$r_j(\mathbf{x}) = \sum_{k=1}^{N_c} L_{kj} p(c_k/\mathbf{x}) \quad (12-16)$$

Quantity $r_j(\mathbf{x})$ is called the *conditional average risk* or *loss* in decision-theory terminology.

We know from Bayes' rule that $p(a/b) = [p(a)p(b/a)]/p(b)$, so we can write Eq. (12-16) as

$$r_j(\mathbf{x}) = \frac{1}{p(\mathbf{x})} \sum_{k=1}^{N_c} L_{kj} p(\mathbf{x}/c_k) P(c_k) \quad (12-17)$$

where $p(\mathbf{x}/c_k)$ is the probability density function (PDF) of the patterns from class c_k , and $P(c_k)$ is the probability of occurrence of class c_k (sometimes $P(c_k)$ is referred to as the *a priori*, or simply the *prior, probability*). Because $1/p(\mathbf{x})$ is positive and common to all the $r_j(\mathbf{x})$, $j = 1, 2, \dots, N_c$, it can be dropped from Eq. (12-17) without affecting the relative order of these functions from the smallest to the largest value. The expression for the average loss then reduces to

$$r_j(\mathbf{x}) = \sum_{k=1}^{N_c} L_{kj} p(\mathbf{x}/c_k) P(c_k) \quad (12-18)$$

Given an unknown pattern, the classifier has N_c possible classes from which to choose. If the classifier computes $r_1(\mathbf{x}), r_2(\mathbf{x}), \dots, r_{N_c}(\mathbf{x})$ for each pattern $\mathbf{x}$ and assigns the pattern to the class with the smallest loss, the total average loss with respect to all decisions will be minimum. The classifier that minimizes the total average loss is called the *Bayes classifier*. This classifier assigns an unknown pattern $\mathbf{x}$ to class c_i if $r_i(\mathbf{x}) < r_j(\mathbf{x})$ for $j = 1, 2, \dots, N_c$; $j \neq i$. In other words, $\mathbf{x}$ is assigned to class c_i if

$$\sum_{k=1}^{N_c} L_{ki} p(\mathbf{x}/c_k) P(c_k) < \sum_{q=1}^{N_c} L_{qj} p(\mathbf{x}/c_q) P(c_q) \quad (12-19)$$

for all j ; $j \neq i$. The loss for a correct decision generally is assigned a value of 0, and the loss for any incorrect decision usually is assigned a value of 1. Then, the loss function becomes

$$L_{ij} = 1 - \delta_{ij} \quad (12-20)$$

where $\delta_{ij} = 1$ if $i = j$, and $\delta_{ij} = 0$ if $i \neq j$. Equation (12-20) indicates a loss of unity for incorrect decisions and a loss of zero for correct decisions. Substituting Eq. (12-20) into Eq. (12-18) yields

$$\begin{aligned} r_j(\mathbf{x}) &= \sum_{k=1}^{N_c} (1 - \delta_{kj}) p(\mathbf{x}/c_k) P(c_k) \\ &= p(\mathbf{x}) - p(\mathbf{x}/c_j) P(c_j) \end{aligned} \quad (12-21)$$

The Bayes classifier then assigns a pattern $\mathbf{x}$ to class c_i if, for all $j \neq i$,

$$p(\mathbf{x}) - p(\mathbf{x}/c_i) P(c_i) < p(\mathbf{x}) - p(\mathbf{x}/c_j) P(c_j) \quad (12-22)$$

or, equivalently, if

$$p(\mathbf{x}/c_i) P(c_i) > p(\mathbf{x}/c_j) P(c_j) \quad j = 1, 2, \dots, N_c; \quad j \neq i \quad (12-23)$$

Thus, the Bayes classifier for a 0-1 loss function computes *decision functions* of the form

$$d_j(\mathbf{x}) = p(\mathbf{x}/c_j) P(c_j) \quad j = 1, 2, \dots, N_c \quad (12-24)$$

and assigns a pattern to class c_i if $d_i(\mathbf{x}) > d_j(\mathbf{x})$ for all $j \neq i$. This is exactly the same process described in Eq. (12-5), but we are now dealing with decision functions that have been shown to be optimal in the sense that they minimize the average loss in misclassification.

For the optimality of Bayes decision functions to hold, the probability density functions of the patterns in each class, as well as the probability of occurrence of each class, must be known. The latter requirement usually is not a problem. For instance, if all classes are equally likely to occur, then $P(c_j) = 1/N_c$. Even if this condition is not true, these probabilities generally can be inferred from knowledge of the problem. Estimating the probability density functions $p(\mathbf{x}/c_j)$ is more difficult. If the pattern vectors are n -dimensional, then $p(\mathbf{x}/c_j)$ is a function of n variables. If the form of $p(\mathbf{x}/c_j)$ is not known, estimating it requires using multivariate estimation methods. These methods are difficult to apply in practice, especially if the number of representative patterns from each class is not large, or if the probability density functions are not well behaved. For these reasons, uses of the Bayes classifier often are based on assuming an analytic expression for the density functions. This in turn reduces the problem to one of estimating the necessary parameters from sample patterns from each class using training patterns. By far, the most prevalent form assumed for $p(\mathbf{x}/c_j)$ is the Gaussian probability density function. The closer this assumption is to reality, the closer the Bayes classifier approaches the minimum average loss in classification.

BAYES CLASSIFIER FOR GAUSSIAN PATTERN CLASSES

You may find it helpful to review the tutorial on probability available in the book website.

To begin, let us consider a 1-D problem ($n = 1$) involving two pattern classes ($N_c = 2$) governed by Gaussian densities, with means m_1 and m_2 , and standard deviations σ_1 and σ_2 , respectively. From Eq. (12-24) the Bayes decision functions have the form

$$\begin{aligned} d_j(x) &= p(x/c_j)P(c_j) \\ &= \frac{1}{\sqrt{2\pi}\sigma_j} e^{-\frac{(x-m_j)^2}{2\sigma_j^2}} P(c_j) \quad j = 1, 2 \end{aligned} \quad (12-25)$$

where the patterns are now scalars, denoted by x . Figure 12.19 shows a plot of the probability density functions for the two classes. The boundary between the two classes is a single point, x_0 , such that $d_1(x_0) = d_2(x_0)$. If the two classes are equally likely to occur, then $P(c_1) = P(c_2) = 1/2$, and the decision boundary is the value of x_0 for which $p(x_0/c_1) = p(x_0/c_2)$. This point is the intersection of the two probability density functions, as shown in Fig. 12.19. Any pattern (point) to the right of x_0 is classified as belonging to class c_1 . Similarly, any pattern to the left of x_0 is classified as belonging to class c_2 . When the classes are not equally likely to occur, x_0 moves to the left if class c_1 is more likely to occur or, conversely, it moves to the right if class c_2 is more likely to occur. This result is to be expected, because the classifier is trying to minimize the loss of misclassification. For instance, in the extreme case, if class c_2 never occurs, the classifier would never make a mistake by always assigning all patterns to class c_1 (that is, x_0 would move to negative infinity).

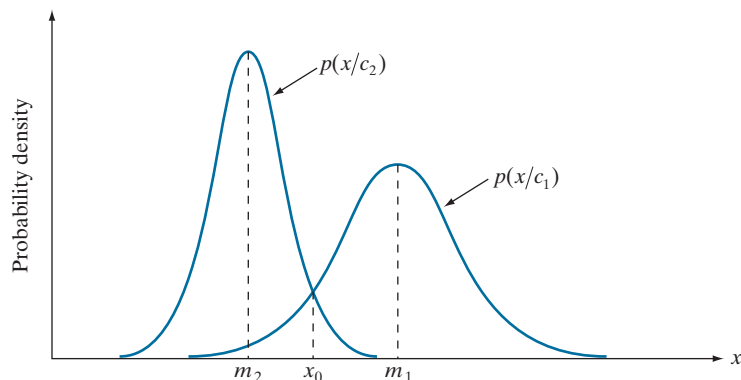
In the n -dimensional case, the Gaussian density of the vectors in the j th pattern class has the form

$$p(\mathbf{x}/c_j) = \frac{1}{(2\pi)^{n/2} |\mathbf{C}_j|^{1/2}} e^{-\frac{1}{2}(\mathbf{x} - \mathbf{m}_j)^T \mathbf{C}_j^{-1} (\mathbf{x} - \mathbf{m}_j)} \quad (12-26)$$

where each density is specified completely by its mean vector $\mathbf{m}_j$ and covariance matrix $\mathbf{C}_j$, which are defined as

FIGURE 12.19

Probability density functions for two 1-D pattern classes. Point x_0 (at the intersection of the two curves) is the Bayes decision boundary if the two classes are equally likely to occur.



$$\mathbf{m}_j = E_j \{ \mathbf{x} \} \quad (12-27)$$

and

$$\mathbf{C}_j = E_j \left\{ (\mathbf{x} - \mathbf{m}_j)(\mathbf{x} - \mathbf{m}_j)^T \right\} \quad (12-28)$$

where $E_j \{ \cdot \}$ is the expected value of the argument over the patterns of class c_j . In Eq. (12-26), n is the dimensionality of the pattern vectors, and $|\mathbf{C}_j|$ is the determinant of matrix $\mathbf{C}_j$. Approximating the expected value E_j by the sample average yields an estimate of the mean vector and covariance matrix:

$$\mathbf{m}_j = \frac{1}{n_j} \sum_{\mathbf{x} \in c_j} \mathbf{x} \quad (12-29)$$

and

$$\mathbf{C}_j = \frac{1}{n_j} \sum_{\mathbf{x} \in c_j} \mathbf{x} \mathbf{x}^T - \mathbf{m}_j \mathbf{m}_j^T \quad (12-30)$$

where n_j is the number of sample pattern vectors from class c_j and the summation is taken over these vectors. We will give an example later in this section of how to use these two expressions.

The covariance matrix is symmetric and positive semidefinite. Its k th diagonal element is the variance of the k th element of the pattern vectors. The kj th off-diagonal matrix element is the covariance of elements x_k and x_j in these vectors. The multivariate Gaussian density function reduces to the product of the univariate Gaussian density of each element of $\mathbf{x}$ when the off-diagonal elements of the covariance matrix are zero, which happens when the vector elements x_k and x_j are uncorrelated.

From Eq. (12-24), the Bayes decision function for class c_j is $d_j(\mathbf{x}) = p(\mathbf{x}/c_j)P(c_j)$. However, the exponential form of the Gaussian density allows us to work with the natural logarithm of this decision function, which is more convenient. In other words, we can use the form

$$\begin{aligned} d_j(\mathbf{x}) &= \ln \left[p(\mathbf{x}/c_j)P(c_j) \right] \\ &= \ln p(\mathbf{x}/c_j) + \ln P(c_j) \end{aligned} \quad (12-31)$$

This expression is equivalent to Eq. (12-24) in terms of classification performance because the logarithm is a monotonically increasing function. That is, the numerical order of the decision functions in Eqs. (12-24) and (12-31) is the same. Substituting Eq. (12-26) into Eq. (12-31) yields

$$d_j(\mathbf{x}) = \ln P(c_j) - \frac{n}{2} \ln 2\pi - \frac{1}{2} \ln |\mathbf{C}_j| - \frac{1}{2} \left[(\mathbf{x} - \mathbf{m}_j)^T \mathbf{C}_j^{-1} (\mathbf{x} - \mathbf{m}_j) \right] \quad (12-32)$$

The term $(n/2) \ln 2\pi$ is the same for all classes, so it can be eliminated from Eq. (12-32), which then becomes

$$d_j(\mathbf{x}) = \ln P(c_j) - \frac{1}{2} \ln |\mathbf{C}_j| - \frac{1}{2} \left[(\mathbf{x} - \mathbf{m}_j)^T \mathbf{C}_j^{-1} (\mathbf{x} - \mathbf{m}_j) \right] \quad (12-33)$$

As noted in Section 6.7 [see Eq. (6-49)], the square root of the rightmost term in this equation is called the *Mahalanobis distance*.

for $j = 1, 2, \dots, N_c$. This equation gives the Bayes decision functions for Gaussian pattern classes under the condition of a 0-1 loss function.

The decision functions in Eq. (12-33) are hyperquadrics (quadratic functions in n -dimensional space), because no terms higher than the second degree in the components of $\mathbf{x}$ appear in the equation. Clearly, then, the best that a Bayes classifier for Gaussian patterns can do is to place a second-order decision boundary between each pair of pattern classes. If the pattern populations are truly Gaussian, no other boundary would yield a lesser average loss in classification.

If all covariance matrices are equal, then $\mathbf{C}_j = \mathbf{C}$ for $j = 1, 2, \dots, N_c$. By expanding Eq. (12-33), and dropping all terms that do not depend on j , we obtain

$$d_j(\mathbf{x}) = \ln P(c_j) + \mathbf{x}^T \mathbf{C}^{-1} \mathbf{m}_j - \frac{1}{2} \mathbf{m}_j^T \mathbf{C}^{-1} \mathbf{m}_j \quad (12-34)$$

which are linear decision functions (hyperplanes) for $j = 1, 2, \dots, N_c$.

If, in addition, $\mathbf{C} = \mathbf{I}$, where $\mathbf{I}$ is the identity matrix, and also if the classes are equally likely (i.e., $P(c_j) = 1/N_c$ for all j), then we can drop the term $\ln P(c_j)$ because it would be the same for all values of j . Equation (12-34) then becomes

$$d_j(\mathbf{x}) = \mathbf{m}_j^T \mathbf{x} - \frac{1}{2} \mathbf{m}_j^T \mathbf{m}_j \quad j = 1, 2, \dots, N_c \quad (12-35)$$

which we recognize as the decision functions for a minimum-distance classifier [see Eq. (12-4)]. Thus, as mentioned earlier, the minimum-distance classifier is optimum in the Bayes sense if (1) the pattern classes follow a Gaussian distribution, (2) all covariance matrices are equal to the identity matrix, and (3) all classes are equally likely to occur. Gaussian pattern classes satisfying these conditions are spherical clouds of identical shape in n dimensions (called *hyperspheres*). The minimum-distance classifier establishes a hyperplane between every pair of classes, with the property that the hyperplane is the perpendicular bisector of the line segment joining the center of the pair of hyperspheres. In 2-D, the patterns are distributed in circular regions, and the boundaries become lines that bisect the line segment joining the center of every pair of such circles.

EXAMPLE 12.5: A Bayes classifier for 3-D patterns.

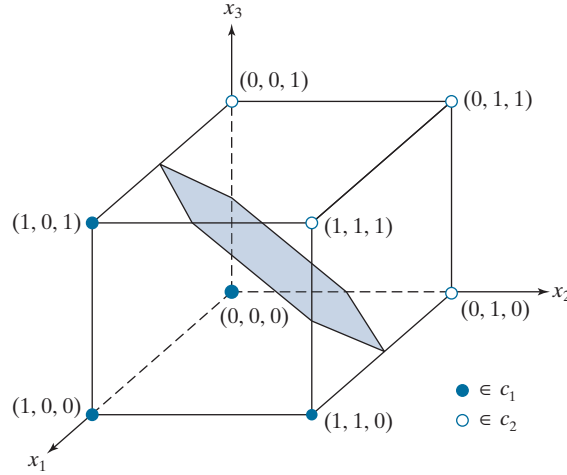
We illustrate the mechanics of the preceding development using the simple patterns in Fig. 12.20. We assume that the patterns are samples from two Gaussian populations, and that the classes are equally likely to occur. Applying Eq. (12-29) to the patterns in the figure results in

$$\mathbf{m}_1 = \frac{1}{3} \begin{bmatrix} 3 \\ 1 \\ 1 \end{bmatrix} \quad \text{and} \quad \mathbf{m}_2 = \frac{1}{3} \begin{bmatrix} 1 \\ 3 \\ 3 \end{bmatrix}$$

And, from Eq. (12-30),

FIGURE 12.20

Two simple pattern classes and the portion of their Bayes decision boundary (shaded) that intersects the cube.



$$\mathbf{C}_1 = \mathbf{C}_2 = \frac{1}{16} \begin{bmatrix} 3 & 1 & 1 \\ 1 & 3 & -1 \\ 1 & -1 & 3 \end{bmatrix}$$

The inverse of this matrix is

$$\mathbf{C}_1^{-1} = \mathbf{C}_2^{-1} = \begin{bmatrix} 8 & -4 & -4 \\ -4 & 8 & 4 \\ -4 & 4 & 8 \end{bmatrix}$$

Next, we obtain the decision functions. Equation (12-34) applies because the covariance matrices are equal, and we are assuming that the classes are equally likely:

$$d_j(\mathbf{x}) = \mathbf{x}^T \mathbf{C}^{-1} \mathbf{m}_j - \frac{1}{2} \mathbf{m}_j^T \mathbf{C}^{-1} \mathbf{m}_j$$

Carrying out the vector-matrix expansion, we obtain the two decision functions:

$$d_1(\mathbf{x}) = 4x_1 - 1.5 \quad \text{and} \quad d_2(\mathbf{x}) = -4x_1 + 8x_2 + 8x_3 - 5.5$$

The decision boundary separating the two classes is then

$$d_1(\mathbf{x}) - d_2(\mathbf{x}) = 8x_1 - 8x_2 - 8x_3 + 4 = 0$$

Figure 12.20 shows a section of this planar surface. Note that the classes were separated effectively.

EXAMPLE 12.6: Classification of multispectral data using a Bayes classifier.

As discussed in Sections 1.3 and 11.5, a multispectral scanner responds to selected bands of the electromagnetic energy spectrum, such as the bands: 0.45–0.52, 0.53–0.61, 0.63–0.69, and 0.78–0.90 microns. These ranges are in the visible blue, visible green, visible red, and near infrared bands, respectively. A region on the ground scanned using these multispectral bands produces four digital images of the region,

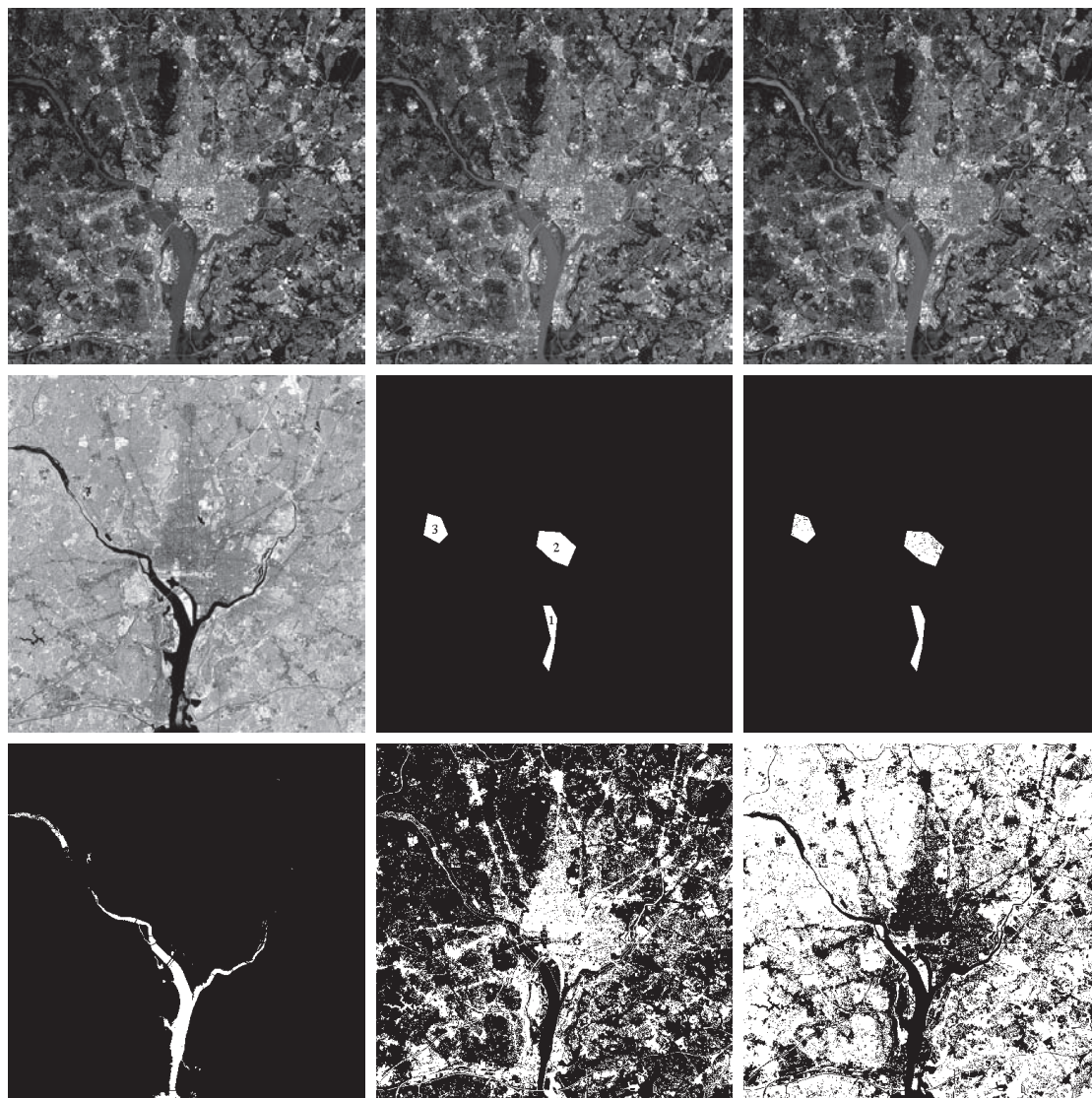
one for each band. If the images are registered spatially, they can be visualized as being stacked one behind the other, as illustrated in Fig. 12.7. As we explained in that figure, every point on the ground in this example can be represented by a 4-D pattern vector of the form $\mathbf{x} = (x_1, x_2, x_3, x_4)^T$, where x_1 is a shade of blue, x_2 a shade of green, and so on. If the images are of size 512×512 pixels, each stack of four multispectral images can be represented by 266,144 four-dimensional pattern vectors. As noted previously, the Bayes classifier for Gaussian patterns requires estimates of the mean vector and covariance matrix for each class. In remote sensing applications, these estimates are obtained using training multispectral data whose classes are known from each region of interest (this knowledge sometimes is referred to as *ground truth*). The resulting vectors are then used to estimate the required mean vectors and covariance matrices, as in Example 12.5.

Figures 12.21(a) through (d) show four 512×512 multispectral images of the Washington, D.C. area, taken in the bands mentioned in the previous paragraph. We are interested in classifying the pixels in these images into one of three pattern classes: *water*, *urban development*, or *vegetation*. The masks in Fig. 12.21(e) were superimposed on the images to extract samples representative of these three classes. Half of the samples were used for training (i.e., for estimating the mean vectors and covariance matrices), and the other half were used for independent testing to assess classifier performance. We assume that the a priori probabilities are equal, $P(c_j) = 1/3$; $j = 1, 2, 3$.

Table 12.1 summarizes the classification results we obtained with the training and test data sets. The percentage of training and test pattern vectors recognized correctly was about the same with both data sets, indicating that the learned parameters did not over-fit the parameters to the training data. The largest error in both cases was with patterns from the urban area. This is not unexpected, as vegetation is present there also (note that no patterns in the vegetation or urban areas were misclassified as water). Figure 12.21(f) shows as black dots the training and test patterns that were misclassified, and as white dots the patterns that were classified correctly. No black dots are visible in region 1, because the seven misclassified points are very close to the boundary of the white region. You can compute from the numbers in the table that the correct recognition rate was 96.4% for the training patterns, and 96.1% for the test patterns.

Figures 12.21(g) through (i) are more interesting. Here, we let the system classify *all* image pixels into one of the three categories. Figure 12.21(g) shows in white all pixels that were classified as water. Pixels not classified as water are shown in black. We see that the Bayes classifier did an excellent job of determining which parts of the image were water. Figure 12.21(h) shows in white all pixels classified as urban development; observe how well the system performed in recognizing urban features, such as the bridges and highways. Figure 12.21(i) shows the pixels classified as vegetation. The center area in Fig. 12.21(h) shows a high concentration of white pixels in the downtown area, with the density decreasing as a function of distance from the center of the image. Figure 12.21(i) shows the opposite effect, indicating the least vegetation toward the center of the image, where urban development is the densest.

We mentioned in Section 10.3 when discussing Otsu's method that thresholding may be viewed as a Bayes classification problem, which optimally assigns patterns to two or more classes. In fact, as the previous example shows, pixel-by-pixel classification may be viewed as a segmentation that partitions an image into two or more possible types of regions. If only one single variable (e.g., intensity) is used, then Eq. (12-24) becomes an optimum function that similarly partitions an image based on the intensity of its pixels, as we did in Section 10.3. Keep in mind that optimality requires that the PDF and a priori probability of each class be known. As we



a	b	c
d	e	f
g	h	i

FIGURE 12.21 Bayes classification of multispectral data. (a)–(d) Images in the visible blue, visible green, visible red, and near infrared wavelength bands. (e) Masks for regions of water (labeled 1), urban development (labeled 2), and vegetation (labeled 3). (f) Results of classification; the black dots denote points classified incorrectly. The other (white) points were classified correctly. (g) All image pixels classified as water (in white). (h) All image pixels classified as urban development (in white). (i) All image pixels classified as vegetation (in white).

TABLE 12.1

Bayes classification of multispectral image data. Classes 1, 2, and 3 are water, urban, and vegetation, respectively.

Training Patterns						Test Patterns					
Class	No. of Samples	Classified into Class			% Correct	Class	No. of Samples	Classified into Class			% Correct
		1	2	3				1	2	3	
1	484	482	2	0	99.6	1	483	478	3	2	98.9
2	933	0	885	48	94.9	2	932	0	880	52	94.4
3	483	0	19	464	96.1	3	482	0	16	466	96.7

have mentioned previously, estimating these densities is not a trivial task. If assumptions have to be made (e.g., as in assuming Gaussian densities), then the degree of optimality achieved in classification depends on how close the assumptions are to reality.

12.5 NEURAL NETWORKS AND DEEP LEARNING

The principal objectives of the material in this section and in Section 12.6 are to present an introduction to deep neural networks, and to derive the equations that are the foundation of deep learning. We will discuss two types of networks. In this section, we focus attention on multilayer, fully connected neural networks, whose inputs are pattern vectors of the form introduced in Section 12.2. In Section 12.6, we will discuss convolutional neural networks, which are capable of accepting images as inputs. We follow the same basic approach in presenting the material in these two sections. That is, we begin by developing the equations that describe how an input is mapped through the networks to generate the outputs that are used to classify that input. Then, we derive the equations of backpropagation, which are the tools used to train both types of networks. We give examples in both sections that illustrate the power of deep neural networks and deep learning for solving complex pattern classification problems.

BACKGROUND

The essence of the material that follows is the use of a multitude of elemental non-linear computing elements (called *artificial neurons*), organized as networks whose interconnections are similar in some respects to the way in which neurons are interconnected in the visual cortex of mammals. The resulting models are referred to by various names, including *neural networks*, *neurocomputers*, *parallel distributed processing models*, *neuromorphic systems*, *layered self-adaptive networks*, and *connectionist models*. Here, we use the name *neural networks*, or *neural nets* for short. We use these networks as vehicles for adaptively learning the parameters of decision functions via successive presentations of training patterns.

Interest in neural networks dates back to the early 1940s, as exemplified by the work of McCulloch and Pitts [1943], who proposed neuron models in the form of

binary thresholding devices, and stochastic algorithms involving sudden 0–1 and 1–0 changes of states, as the basis for modeling neural systems. Subsequent work by Hebb [1949] was based on mathematical models that attempted to capture the concept of learning by reinforcement or association.

During the mid-1950s and early 1960s, a class of so-called *learning machines* originated by Rosenblatt [1959, 1962] caused a great deal of excitement among researchers and practitioners of pattern recognition. The reason for the interest in these machines, called *perceptrons*, was the development of mathematical proofs showing that perceptrons, when trained with linearly separable training sets (i.e., training sets separable by a hyperplane), would converge to a solution in a finite number of iterative steps. The solution took the form of parameters (coefficients) of hyperplanes that were capable of correctly separating the classes represented by patterns of the training set.

Unfortunately, the expectations following discovery of what appeared to be a well-founded theoretical model of learning soon met with disappointment. The basic perceptron, and some of its generalizations, were inadequate for most pattern recognition tasks of practical significance. Subsequent attempts to extend the power of perceptron-like machines by considering multiple layers of these devices lacked effective training algorithms, such as those that had created interest in the perceptron itself. The state of the field of learning machines in the mid-1960s was summarized by Nilsson [1965]. A few years later, Minsky and Papert [1969] presented a discouraging analysis of the limitation of perceptron-like machines. This view was held as late as the mid-1980s, as evidenced by comments made by Simon [1986]. In this work, originally published in French in 1984, Simon dismisses the perceptron under the heading “Birth and Death of a Myth.”

More recent results by Rumelhart, Hinton, and Williams [1986] dealing with the development of new training algorithms for multilayers of perceptron-like units have changed matters considerably. Their basic method, called *backpropagation* (*backprop* for short), provides an effective training method for multilayer networks. Although this training algorithm cannot be shown to converge to a solution in the sense of the proof for the single-layer perceptron, backpropagation is capable of generating results that have revolutionized the field of pattern recognition.

The approaches to pattern recognition we have studied up to this point rely on human-engineered techniques to transform raw data into formats suitable for computer processing. The methods of feature extraction we studied in Chapter 11 are examples of this. Unlike these approaches, neural networks can use backpropagation to automatically learn representations suitable for recognition, starting with raw data. Each layer in the network “refines” the representation into more abstract levels. This type of multilayered learning is commonly referred to as *deep learning*, and this capability is one of the underlying reasons why applications of neural networks have been so successful. As we noted at the beginning of this section, practical implementations of deep learning generally are associated with large data sets.

Of course, these are not “magical” systems that assemble themselves. Human intervention is still required for specifying parameters such as the number of layers, the number of artificial neurons per layer, and various coefficients that are problem

dependent. Teaching proper recognition to a complex multilayer neural network is not a science; rather, it is an art that requires considerable knowledge and experimentation on the part of the designer. Countless applications of pattern recognition, especially in constrained environments, are best handled by more “traditional” methods. A good example of this is stylized font recognition. It would be senseless to develop a neural network to recognize the E-13B font we studied in Fig. 12.11. A minimum-distance classifier implemented on a hard-wired architecture is the ideal solution to this problem, provided that interest is limited to reading only the E-13B font printed on bank checks. On the other hand, neural networks have proved to be the ideal solution if the scope of application is expanded to require that all relevant text written on checks, including cursive script, be read with high accuracy.

Deep learning has shined in applications that defy other methods of solution. In the two decades following the introduction of backpropagation, neural networks have been used successfully in a broad range of applications. Some of them, such as speech recognition, have become an integral part of everyday life. When you speak into a smart phone, the nearly flawless recognition is performed by a neural network. This type of performance was unachievable just a few years ago. Other applications from which you benefit, perhaps without realizing it, are smart filters that learn user preferences for rerouting spam and other junk mail from email accounts, and the systems that read zip codes on postal mail. Often, you see television clips of vehicles navigating autonomously, and robots that are capable of interacting with their environment. Most are solutions based on neural networks. Less familiar applications include the automated discovery of new medicines, the prediction of gene mutations in DNA research, and advances in natural language understanding.

Although the list of practical uses of neural nets is long, applications of this technology in image pattern classification has been slower in gaining popularity. As you will learn shortly, using neural nets in image processing is based principally on neural network architectures called *convolutional neural nets* (denoted by *CNNs* or *ConvNets*). One of the earliest well-known applications of CNNs is the work of LeCun et al. [1989] for reading handwritten U.S. postal zip codes. A number of other applications followed shortly thereafter, but it was not until the results of the 2012 ImageNet Challenge were published (e.g., see Krizhevsky, Sutskever, and Hinton [2012]) that CNNs became widely used in image pattern recognition. Today, this is the approach of choice for addressing complex image recognition tasks.

The neural network literature is vast and rapidly evolving, so as usual, our approach is to focus on fundamentals. In this and the following sections, we will establish the foundation of how neural nets are trained, and how they operate after training. We will begin by briefly discussing perceptrons. Although these computing elements are not used per se in current neural network architectures, the operations they perform are almost identical to artificial neurons, which are the basic computing units of neural nets. In fact, an introduction to neural networks would be incomplete without a discussion of perceptrons. We will follow this discussion by developing in detail the theoretical foundation of backpropagation. After developing the basic backpropagation equations, we will recast them in matrix form, which

reduces the training and operation of neural nets to a simple, straightforward cascade of matrix multiplications.

After studying several examples of fully connected neural nets, we will follow a similar approach in developing the foundation of CNNs, including how they differ from fully connected neural nets, and how their training is different. This is followed by several examples of how CNNs are used for image pattern classification.

THE PERCEPTRON

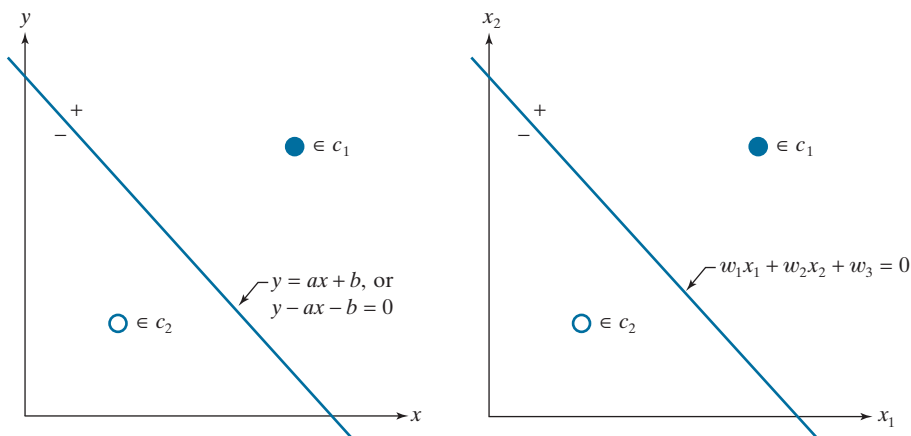
A single perceptron unit learns a linear boundary between two linearly separable pattern classes. Figure 12.22(a) shows the simplest possible example in two dimensions: two pattern classes, consisting of a single pattern each. A linear boundary in 2-D is a straight line with equation $y = ax + b$, where coefficient a is the *slope* and b is the *y-intercept*. Note that if $b = 0$, the line goes through the origin. Therefore, the function of parameter b is to displace the line from the origin without affecting its slope. For this reason, this “floating” coefficient that is not multiplied by a coordinate is often referred to as the *bias*, the *bias coefficient*, or the *bias weight*.

We are interested in a line that separates the two classes in Fig. 12.22. This is a line positioned in such a way that pattern (x_1, y_1) from class c_1 lies on one side of the line, and pattern (x_2, y_2) from class c_2 lies on the other. The locus of points (x, y) that are on the line, satisfy the equation $y - ax - b = 0$. It then follows that any point on one side of the line would yield a positive value when its coordinates are plugged into this equation, and conversely for a point on the other side.

Generally, we work with patterns in much higher dimensions than two, so we need more general notation. Points in n dimensions are vectors. The components of a vector, $x_1, x_2, \dots, x_n$, are the coordinates of the point. For the coefficients of the boundary separating the two classes, we use the notation $w_1, w_2, \dots, w_n, w_{n+1}$, where w_{n+1} is the bias. The general equation of our line using this notation is $w_1x_1 + w_2x_2 + w_3 = 0$ (we can express this equation in slope-intercept form as $x_2 + (w_1/w_2)x_1 + w_3/w_2 = 0$). Figure 12.22(b) is the same as (a), but using this notation. Comparing the two figures, we see that $y = x_2$, $x = x_1$, $a = w_1/w_2$, and $b = w_3/w_2$. Equipped with our more

FIGURE 12.22

(a) The simplest two-class example in 2-D, showing one possible decision boundary out of an infinite number of such boundaries.
(b) Same as (a), but with the decision boundary expressed using more general notation.



general notation, we say that an arbitrary point (x_1, x_2) is on the positive side of a line if $w_1x_1 + w_2x_2 + w_3 > 0$, and conversely for any point on the negative side. For points in 3-D, we work with the equation of a plane, $w_1x_1 + w_2x_2 + w_3x_3 + w_4 = 0$, but would perform exactly the same test to see if a point lies on the positive or negative side of the plane. For a point in n dimensions, the test would be against a *hyperplane*, whose equation is

$$w_1x_1 + w_2x_2 + \cdots + w_nx_n + w_{n+1} = 0 \quad (12-36)$$

This equation is expressed in summation form as

$$\sum_{i=1}^n w_i x_i + w_{n+1} = 0 \quad (12-37)$$

or in vector form as

$$\mathbf{w}^T \mathbf{x} + w_{n+1} = 0 \quad (12-38)$$

where $\mathbf{w}$ and $\mathbf{x}$ are n -dimensional *column* vectors and $\mathbf{w}^T \mathbf{x}$ is the dot (inner) product of the two vectors. Because the inner product is commutative, we can express Eq. (12-38) in the equivalent form $\mathbf{x}^T \mathbf{w} + w_{n+1} = 0$. We refer to $\mathbf{w}$ as a *weight vector* and, as above, to w_{n+1} as a *bias*. Because the bias is a weight that is always multiplied by 1, sometimes we avoid repetition by using the term *weights, coefficients, or parameters* when referring to the bias and the elements of a weight vector collectively.

Stating the class separation problem in general form we say that, given any pattern vector $\mathbf{x}$ from a vector population, we want to find a set of weights with the property

$$\mathbf{w}^T \mathbf{x} + w_{n+1} = \begin{cases} > 0 & \text{if } \mathbf{x} \in c_1 \\ < 0 & \text{if } \mathbf{x} \in c_2 \end{cases} \quad (12-39)$$

Finding a line that separates two *linearly separable* pattern classes in 2-D can be done by inspection. Finding a separating plane by visual inspection of 3-D data is more difficult, but it is doable. For $n > 3$, finding a separating hyperplane by inspection becomes impossible in general. We have to resort instead to an algorithm to find a solution. The perceptron is an implementation of such an algorithm. It attempts to find a solution by iteratively stepping through the patterns of each of two classes. It starts with an arbitrary weight vector and bias, and is guaranteed to converge in a finite number of iterations if the classes are linearly separable.

The perceptron algorithm is simple. Let $\alpha > 0$ denote a *correction increment* (also called the *learning increment* or the *learning rate*), let $\mathbf{w}(1)$ be a vector with arbitrary values, and let $w_{n+1}(1)$ be an arbitrary constant. Then, do the following for $k = 2, 3, \dots$: For a pattern vector, $\mathbf{x}(k)$, at step k ,

1) If $\mathbf{x}(k) \in c_1$ and $\mathbf{w}^T(k)\mathbf{x}(k) + w_{n+1}(k) \leq 0$, let

$$\begin{aligned} \mathbf{w}(k+1) &= \mathbf{w}(k) + \alpha \mathbf{x}(k) \\ w_{n+1}(k+1) &= w_{n+1}(k) + \alpha \end{aligned} \quad (12-40)$$

It is customary to associate $>$ with class c_1 and $<$ with class c_2 , but the sense of the inequality is arbitrary, provided that you are consistent. Note that this equation implements a *linear decision function*.

Linearly separable classes satisfy Eq. (12-39). That is, they are separable by single hyperplanes.

2) If $\mathbf{x}(k) \in c_2$ and $\mathbf{w}^T(k)\mathbf{x}(k) + w_{n+1}(k) \geq 0$, let

$$\begin{aligned}\mathbf{w}(k+1) &= \mathbf{w}(k) - \alpha \mathbf{x}(k) \\ \omega_{n+1}(k+1) &= \omega_{n+1}(k) - \alpha\end{aligned}\tag{12-41}$$

3) Otherwise, let

$$\begin{aligned}\mathbf{w}(k+1) &= \mathbf{w}(k) \\ \omega_{n+1}(k+1) &= \omega_{n+1}(k)\end{aligned}\tag{12-42}$$

The correction in Eq. (12-40) is applied when the pattern is from class c_1 and Eq. (12-39) does not give a positive response. Similarly, the correction in Eq. (12-41) is applied when the pattern is from class c_2 and Eq. (12-39) does not give a negative response. As Eq. (12-42) shows, no change is made when Eq. (12-39) gives the correct response.

The notation in Eqs. (12-40) through (12-42) can be simplified if we add a 1 at the end of every pattern vector and include the bias in the weight vector. That is, we define $\mathbf{x} \triangleq [x_1, x_2, \dots, x_n, 1]^T$ and $\mathbf{w} \triangleq [w_1, w_2, \dots, w_n, w_{n+1}]^T$. Then, Eq. (12-39) becomes

$$\mathbf{w}^T \mathbf{x} = \begin{cases} > 0 & \text{if } \mathbf{x} \in c_1 \\ < 0 & \text{if } \mathbf{x} \in c_2 \end{cases}\tag{12-43}$$

where both vectors are now $(n+1)$ -dimensional. In this formulation, $\mathbf{x}$ and $\mathbf{w}$ are referred to as *augmented* pattern and weight vectors, respectively. The algorithm in Eqs. (12-40) through (12-42) then becomes: For any pattern vector, $\mathbf{x}(k)$, at step k

1') If $\mathbf{x}(k) \in c_1$ and $\mathbf{w}^T(k)\mathbf{x}(k) \leq 0$, let

$$\mathbf{w}(k+1) = \mathbf{w}(k) + \alpha \mathbf{x}(k)\tag{12-44}$$

2') If $\mathbf{x}(k) \in c_2$ and $\mathbf{w}^T(k)\mathbf{x}(k) \geq 0$, let

$$\mathbf{w}(k+1) = \mathbf{w}(k) - \alpha \mathbf{x}(k)\tag{12-45}$$

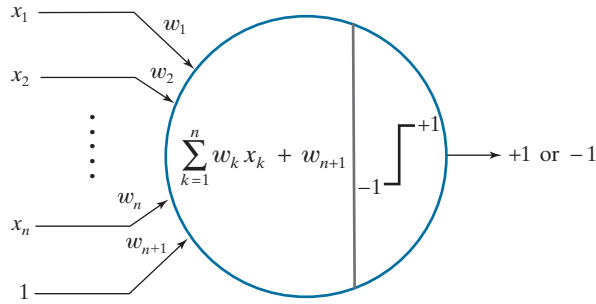
3') Otherwise, let

$$\mathbf{w}(k+1) = \mathbf{w}(k)\tag{12-46}$$

where the starting weight vector, $\mathbf{w}(1)$, is arbitrary and, as above, α is a positive constant. The procedure implemented by Eqs. (12-40)–(12-42) or (12-44)–(12-46) is called the *perceptron training algorithm*. The *perceptron convergence theorem* states that the algorithm is guaranteed to converge to a solution (i.e., a separating hyperplane) in a finite number of steps if the two pattern classes are linearly separable (see Problem 12.15). Normally, Eqs. (12-44)–(12-46) are the basis for implementing the perceptron training algorithm, and we will use it in the following paragraphs of this section. However, the notation in Eqs. (12-40)–(12-42), in which the bias is

FIGURE 12.23

Schematic of a perceptron, showing the operations it performs.



shown separately, is more prevalent in neural networks, so you need to be familiar with it as well.

Figure 12.23 shows a schematic diagram of the perceptron. As you can see, all this simple “machine” does is form a *sum of products* of an input pattern using the weights and bias found during training. The output of this operation is a scalar value that is then passed through an *activation function* to produce the unit’s output. For the perceptron, the activation function is a thresholding function (we will consider other forms of activation when we discuss neural networks). If the thresholded output is a +1, we say that the pattern belongs to class c_1 . Otherwise, a -1 indicates that the pattern belongs to class c_2 . Values 1 and 0 sometimes are used to denote the two possible states of the output.

Note that the perceptron model implements Eq. (12-39), which is in the form of a decision function.

EXAMPLE 12.7: Using the perceptron algorithm to learn a decision boundary.

We illustrate the steps taken by a perceptron in learning the coefficients of a linear boundary by solving the mini problem in Fig. 12.22. To simplify manual computations, let the pattern vector furthest from the origin be $\mathbf{x} = [3 \ 3 \ 1]^T$, and the other be $\mathbf{x} = [1 \ 1 \ 1]^T$, where we augmented the vectors by appending a 1 at the end, as discussed earlier. To match the figure, let these two patterns belong to classes c_1 and c_2 , respectively. Also, assume the patterns are “cycled” through the perceptron in that order during training (one complete iteration through all patterns of the training is called an *epoch*). To start, we let $\alpha = 1$ and $\mathbf{w}(1) = \mathbf{0} = [0 \ 0 \ 0]^T$; then,

For $k = 1$, $\mathbf{x}(1) = [3 \ 3 \ 1]^T \in c_1$, and $\mathbf{w}(1) = [0 \ 0 \ 0]^T$. Their inner product is zero,

$$\mathbf{w}^T(1)\mathbf{x}(1) = [0 \ 0 \ 0] \begin{bmatrix} 3 \\ 3 \\ 1 \end{bmatrix} = 0$$

so Step 1’ of the second version of the training algorithm applies:

$$\mathbf{w}(2) = \mathbf{w}(1) + \alpha \mathbf{x}(1) = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} + (1) \begin{bmatrix} 3 \\ 3 \\ 1 \end{bmatrix} = \begin{bmatrix} 3 \\ 3 \\ 1 \end{bmatrix}$$

For $k = 2$, $\mathbf{x}(2) = [1 \ 1 \ 1]^T \in c_2$ and $\mathbf{w}(2) = [3 \ 3 \ 1]^T$. Their inner product is

$$\mathbf{w}^T(2)\mathbf{x}(2) = \begin{bmatrix} 3 & 3 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} = 7$$

The result is positive when it should have been negative, so Step 2' applies:

$$\mathbf{w}(3) = \mathbf{w}(2) - \alpha \mathbf{x}(2) = \begin{bmatrix} 3 \\ 3 \\ 1 \end{bmatrix} - (1) \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 2 \\ 2 \\ 0 \end{bmatrix}$$

We have gone through a complete training epoch with at least one correction, so we cycle through the training set again.

For $k = 3$, $\mathbf{x}(3) = [3 \ 3 \ 1]^T \in c_1$, and $\mathbf{w}(3) = [2 \ 2 \ 0]^T$. Their inner product is positive (i.e., 6) as it should be because $\mathbf{x}(3) \in c_1$. Therefore, Step 3' applies and the weight vector is not changed:

$$\mathbf{w}(4) = \mathbf{w}(3) = \begin{bmatrix} 2 \\ 2 \\ 0 \end{bmatrix}$$

For $k = 4$, $\mathbf{x}(4) = [1 \ 1 \ 1]^T \in c_2$, and $\mathbf{w}(4) = [2 \ 2 \ 0]^T$. Their inner product is positive (i.e., 4) and it should have been negative, so Step 2' applies:

$$\mathbf{w}(5) = \mathbf{w}(4) - \alpha \mathbf{x}(4) = \begin{bmatrix} 2 \\ 2 \\ 0 \end{bmatrix} - (1) \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \\ -1 \end{bmatrix}$$

At least one correction was made, so we cycle through the training patterns again. For $k = 5$, we have $\mathbf{x}(5) = [3 \ 3 \ 1]^T \in c_1$, and, using $\mathbf{w}(5)$, we compute their inner product to be 5. This is positive as it should be, so Step 3' applies and we let $\mathbf{w}(6) = \mathbf{w}(5) = [1 \ 1 \ -1]^T$. Following this procedure just discussed, you can show (see Problem 12.13) that the algorithm converges to the solution weight vector

$$\mathbf{w} = \mathbf{w}(12) = \begin{bmatrix} 1 \\ 1 \\ -3 \end{bmatrix}$$

which gives the decision boundary

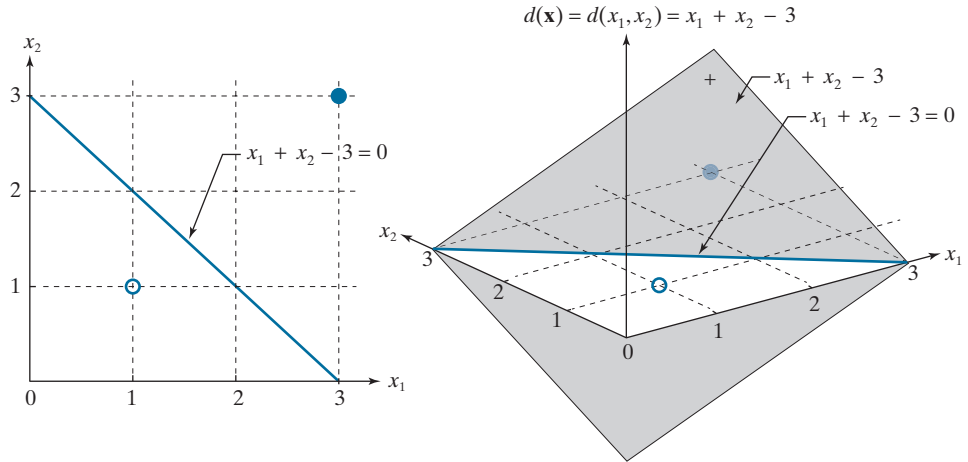
$$x_1 + x_2 - 3 = 0$$

Figure 12.24(a) shows the boundary defined by this equation. As you can see, it clearly separates the patterns of the two classes. In terms of the terminology we used in the previous section, the *decision surface* learned by the perceptron is $d(\mathbf{x}) = d(x_1, x_2) = x_1 + x_2 - 3$, which is a plane. As before, the *decision boundary* is the locus of points such that $d(\mathbf{x}) = d(x_1, x_2) = 0$, which is a line. Another way to visualize this boundary is that it is the intersection of the decision surface (a plane) with the x_1x_2 -plane, as Fig. 12.24(b) shows. All points (x_1, x_2) such that $d(x_1, x_2) > 0$ are on the positive side of the boundary, and vice versa for $d(x_1, x_2) < 0$.

a b

FIGURE 12.24

(a) Segment of the decision boundary learned by the perceptron algorithm.
 (b) Section of the decision surface.
 The decision boundary is the intersection of the decision surface with the x_1x_2 -plane.



EXAMPLE 12.8: Using the perceptron to classify two sets of iris data measurements.

In Fig. 12.10 we showed a reduced set of the iris database in two dimensions, and mentioned that the only class that was separable from the others is the class of Iris setosa. As another illustration of the perceptron, we now find the full decision boundary between the Iris setosa and the Iris versicolor classes. As we mentioned when discussing Fig. 12.10, these are 4-D data sets. Letting $\alpha = 0.5$, and starting with all parameters equal to zero, the perceptron converged in only four epochs to the solution weight vector $\mathbf{w} = [0.65, 2.05, -2.60, -1.10, 0.50]^T$, where the last element is w_{n+1} .

In practice, linearly separable pattern classes are rare, and a significant amount of research effort during the 1960s and 1970s went into developing techniques for dealing with nonseparable pattern classes. With recent advances in neural networks, many of those methods have become items of mere historical interest, and we will not dwell on them here. However, we mention briefly one approach because it is relevant to the discussion of neural networks in the next section. The method is based on minimizing the error between the actual and desired response at any training step.

Let r denote the response we want the perceptron to have for any pattern during training. The output of our perceptron is either +1 or -1, so these are the two possible values that r can have. We want to find the augmented weight vector, $\mathbf{w}$, that minimizes the mean squared error (MSE) between the desired and actual responses of the perceptron. The function should be differentiable and have a unique minimum. The function of choice for this purpose is a quadratic of the form

$$E(\mathbf{w}) = \frac{1}{2} (r - \mathbf{w}^T \mathbf{x})^2 \quad (12-47)$$

where E is our error measure, $\mathbf{w}$ is the weight vector we are seeking, $\mathbf{x}$ is any pattern from the training set, and r is the response we desire for that pattern. Both $\mathbf{w}$ and $\mathbf{x}$ are augmented vectors.

The 1/2 is used to cancel out the 2 that will result from taking the derivative of this expression. Also, remember that $\mathbf{w}^T \mathbf{x}$ is a scalar.

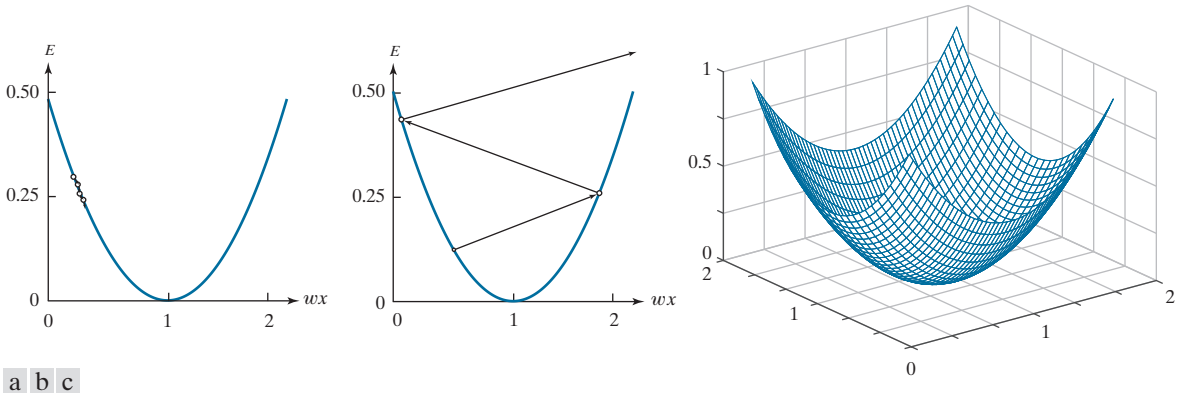


FIGURE 12.25 Plots of E as a function of $w x$ for $r = 1$. (a) A value of α that is too small can slow down convergence. (b) If α is too large, large oscillations or divergence may occur. (c) Shape of the error function in 2-D.

We find the minimum of $E(\mathbf{w})$ using an iterative gradient descent algorithm, whose form is

Note that the right side of this equation is the gradient of $E(\mathbf{w})$.

$$\mathbf{w}(k+1) = \mathbf{w}(k) - \alpha \left[\frac{\partial E(\mathbf{w})}{\partial \mathbf{w}} \right]_{\mathbf{w}=\mathbf{w}(k)} \quad (12-48)$$

where the starting weight vector is arbitrary, and $\alpha > 0$.

Figure 12.25(a) shows a plot of E for scalar values, w and x , of $\mathbf{w}$ and $\mathbf{x}$. We want to move w incrementally so $E(w)$ approaches a minimum, which implies that E should stop changing or, equivalently, that $\partial E(w)/\partial w = 0$. Equation (12-48) does precisely this. If $\partial E(w)/\partial w > 0$, a portion of this quantity (determined by the value of the learning increment α) is subtracted from $w(k)$ to create a new, updated value $w(k+1)$, of the weight. The opposite happens if $\partial E(w)/\partial w < 0$. If $\partial E(w)/\partial w = 0$, the weight is unchanged, meaning that we have arrived at a minimum, which is the solution we are seeking. The value of α determines the relative magnitude of the correction in weight value. If α is too small, the step changes will be correspondingly small and the weight would move slowly toward convergence, as Fig. 12.25(a) illustrates. On the other hand, choosing α too large could cause large oscillations on either side of the minimum, or even become unstable, as Fig. 12.25(b) illustrates. There is no general rule for choosing α . However, a logical approach is to start small and experiment by increasing α to determine its influence on a particular set of training patterns. Figure 12.25(c) shows the shape of the error function for two variables.

Because the error function is given analytically and it is differentiable, we can express Eq. (12-48) in a form that does not require computing the gradient explicitly at every step. The partial of $E(\mathbf{w})$ with respect to $\mathbf{w}$ is

$$\frac{\partial E(\mathbf{w})}{\partial \mathbf{w}} = -(\mathbf{r} - \mathbf{w}^T \mathbf{x}) \mathbf{x} \quad (12-49)$$

Substituting this result into Eq. (12-48) yields

$$\mathbf{w}(k+1) = \mathbf{w}(k) + \alpha [r(k) - \mathbf{w}^T(k)\mathbf{x}(k)]\mathbf{x}(k) \quad (12-50)$$

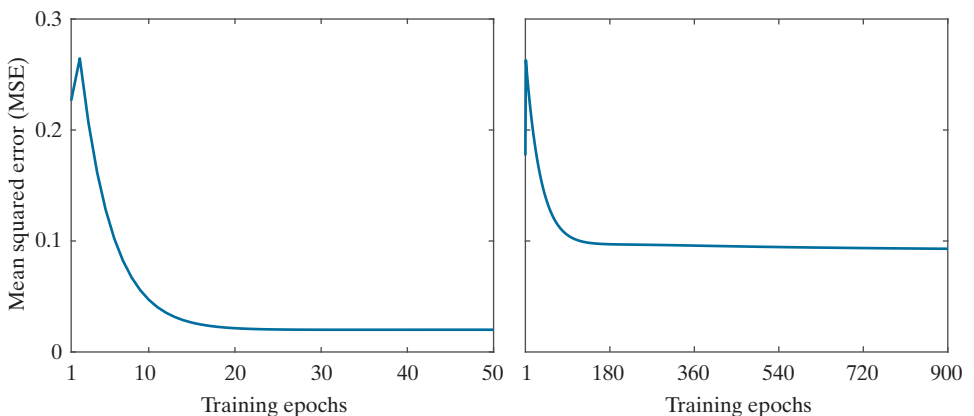
which is in terms of known or easily computable terms. As before, $\mathbf{w}(1)$ is arbitrary.

Widrow and Stearns [1985] have shown that it is necessary (but not sufficient) for α to be in the range $0 < \alpha < 2$ for the algorithm in Eq. (12-50) to converge. A typical range for α is $0.1 < \alpha < 1.0$. Although the proof is not shown here, the algorithm converges to a solution that minimizes the mean squared error over the patterns of the training set. For this reason, the algorithm is often referred to as the *least-mean-squared-error* (LMSE) algorithm. In practice, we say that the algorithm has converged when the error decreases below a specified threshold. The solution at convergence may not be a hyperplane that fully partitions two linearly separable classes. That is, a mean-square-error solution does not imply a solution in the sense of the perceptron training theorem. This uncertainty is the price of using an algorithm whose convergence is independent of the linear separability of the pattern classes.

EXAMPLE 12.9: Using the LMSE algorithm.

It will be interesting to compare the performance of the LMSE algorithm using the same set of separable iris data as in Example 12.8. Figure 12.26(a) is a plot of the error [Eq. (12-47)] as a function of epoch for 50 epochs, using Eq. (12-50) (with $\alpha = 0.001$) to obtain the weights (we started with $\mathbf{w}(1) = \mathbf{0}$). Each epoch of training consisted of sequentially updating the weights, one pattern at a time, and computing Eq. (12-47) for each weight and the corresponding pattern. At the end of the epoch, the errors were added and divided by 100 (the total number of patterns) to obtain the mean squared error (MSE). This yielded one point of the curve of Fig. 12.26(a). After increasing and then decreasing rapidly, no appreciable difference in error occurred after about 20 epochs. For example, the error at the end of the 50th epoch was 0.02 and, at the end of 1,000 epochs, it was 0.0192. Getting smaller error values is possible by further decreasing α , but at the expense of slower decay in the error, as noted in Fig. 12.25. Keep in mind also that MSE is not directly proportional to correct recognition rate.

a b
FIGURE 12.26
MSE as a function of epoch for: (a) the linearly separable Iris classes (setosa and versicolor); and (b) the linearly nonseparable Iris classes (versicolor and virginica).



The weight vector at the end of 50 epochs of training was $\mathbf{w} = [0.098 \ 0.357 \ -0.548 \ -0.255 \ 0.075]^T$. All patterns were classified correctly into their two respective classes using this vector. That is, although the MSE did not become zero, the resulting weight vector was able to classify all the patterns correctly. But keep in mind that the LMSE algorithm does not always achieve 100% correct recognition of linearly separable classes.

As noted earlier, only the Iris setosa samples are linearly separable from the others. But the Iris versicolor and virginica samples are not. The perceptron algorithm would not converge when presented with these data, whereas the LMSE algorithm does. Figure 12.26(b) is the MSE as a function of training epoch for these two data sets, obtained using the same values for $\mathbf{w}(1)$ and α as in (a). This time, it took 900 epochs for the MSE to stabilize at 0.09, which is much higher than before. The resulting weight vector was $\mathbf{w} = [0.534 \ 0.584 \ -0.878 \ -1.028 \ 0.651]^T$. Using this vector resulted in seven misclassification errors out of 100 patterns, giving a recognition rate of 93%.

A classic example used to show the limitations of single linear decision boundaries (and hence single perceptron units) is the XOR classification problem. The table in Fig. 12.27(a) shows the definition of the XOR operator for two variables. As you can see, the XOR operation produces a logical true (1) value when either of the variables (but not both) is true; otherwise, the result is false (0). The XOR two-class pattern classification problem is set up by letting each pair of values A and B be a point in 2-D space, and letting the true (1) XOR values define one class, and the false (0) values define the other. In this case, we assigned the class c_1 label to patterns $\{(0, 0), (1, 1)\}$, and the c_2 label to patterns $\{(1, 0), (0, 1)\}$. A classifier capable of solving the XOR problem must respond with a value, say, 1, when a pattern from class c_1 is presented, and a different value, say, 0 or -1 , when the input pattern is from class c_2 . You can tell by inspection of Fig. 12.27(b) that a single linear decision boundary (a straight line) cannot separate the two classes correctly. This means that we cannot solve the problem with a single perceptron. The simplest linear boundary consists of two straight lines, as Fig. 12.27(b) shows. A more complex, nonlinear, boundary capable of solving the problem is a quadratic function, as in Fig. 12.27(c).

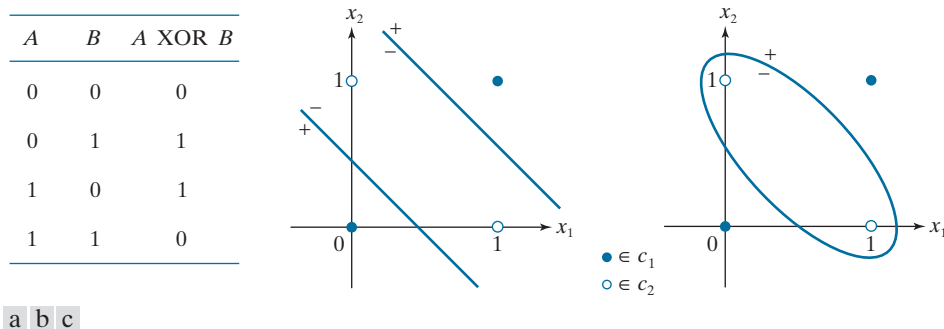
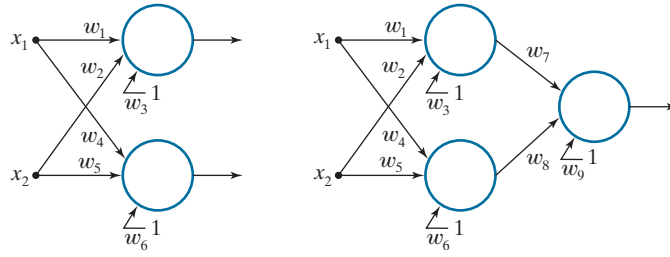


FIGURE 12.27 The XOR classification problem in 2-D. (a) Truth table definition of the XOR operator. (b) 2-D pattern classes formed by assigning the XOR truth values (1) to one pattern class, and false values (0) to another. The simplest decision boundary between the two classes consists of two straight lines. (c) Nonlinear (quadratic) boundary separating the two classes.

a b

FIGURE 12.28

(a) Minimum perceptron solution to the XOR problem in 2-D. (b) A solution that implements the XOR truth table in Fig. 12.27(a).



Natural questions at this point are: Can more than one perceptron solve the XOR problem? If so, what is the minimum number of units required? We know that a single perceptron can implement one straight line, and we need to implement two lines, so the obvious answers are: yes to the first question, and two units to the second. Figure 12.28(a) shows the solution for two variables, which requires a total of six coefficients because we need two lines. The solution coefficients are such that, for either of the two patterns from class c_1 , one output is true (1) and the other is false (0). The opposite condition must hold for either pattern from class c_2 . This solution requires that we analyze two outputs. If we want to implement the truth table, meaning that a single output should give the same response as the XOR function [the third column in Fig. 12.27(a)], then we need one additional perceptron. Figure 12.28(b) shows the architecture for this solution. Here, one perceptron in the first layer maps any input from one class into a 1, and the other perceptron maps a pattern from the other class into a 0. This reduces the four possible inputs into two outputs, which is a two-point problem. As you know from Fig. 12.24, a single perceptron can solve this problem. Therefore, we need three perceptrons to implement the XOR table, as in Fig. 12.28(b).

With a little work, we could determine by inspection the coefficients needed to implement either solution in Fig. 12.28. However, rather than dwell on that, we focus attention in the following section on a more general, layered architecture, of which the XOR solution is a trivial, special case.

MULTILAYER FEEDFORWARD NEURAL NETWORKS

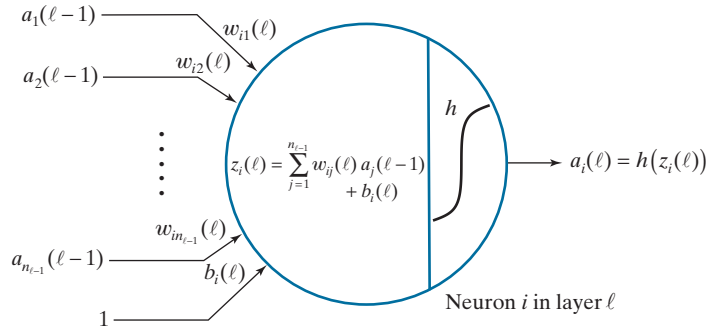
In this section, we discuss the architecture and operation of multilayer neural networks, and derive the equations of backpropagation used to train them. We then give several examples illustrating the capabilities of neural nets

Model of an Artificial Neuron

Neural networks are interconnected perceptron-like computing elements called *artificial neurons*. These neurons perform the same computations as the perceptron, but they differ from the latter in how they process the result of the computations. As illustrated in Fig. 12.23, the perceptron uses a “hard” thresholding function that outputs two values, such as +1 and −1, to perform classification. Suppose that in a network of perceptrons, the output before thresholding of one of the perceptrons is infinitesimally greater than zero. When thresholded, this very small signal will be turned into a +1. But a similarly small signal with the opposite sign would cause

FIGURE 12.29

Model of an artificial neuron, showing all the operations it performs. The “ ℓ ” is used to denote a particular layer in a layered network.



a large swing in value from $+1$ to -1 . Neural networks are formed from layers of computing units, in which the output of one unit affects the behavior of all units following it. The perceptron’s sensitivity to the sign of small signals can cause serious stability problems in an interconnected system of such units, making perceptrons unsuitable for layered architectures.

The solution is to change the characteristic of the activation function from a hard limiter to a smooth function. Figure 12.29 shows an example based on using the activation function

$$h(z) = \frac{1}{1 + e^{-z}} \quad (12-51)$$

where z is the result of the computation performed by the neuron, as shown in Fig. 12.29. Except for more complicated notation, and the use of a smooth function rather than a hard threshold, this model performs the same *sum-of-products* operations as in Eq. (12-36) for the perceptron. Note that the *bias* term is denoted by b instead

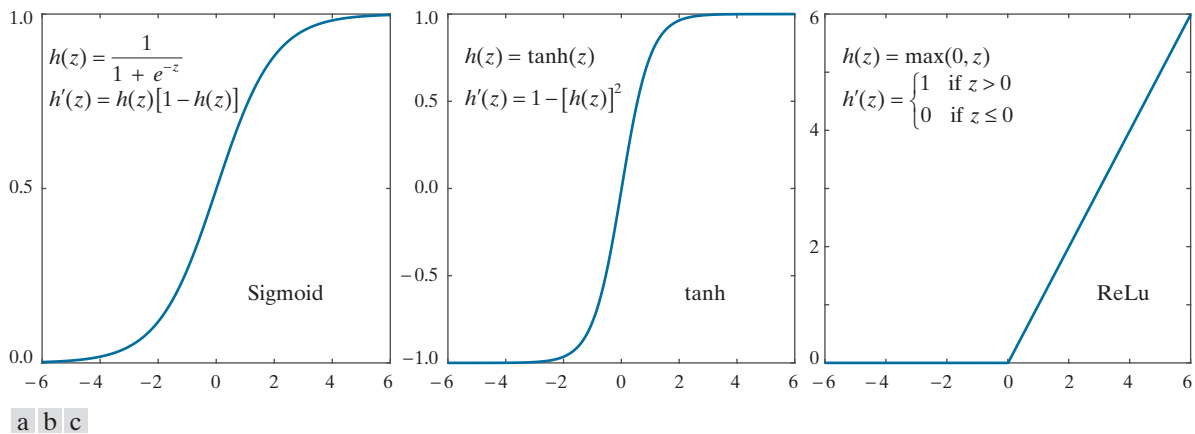


FIGURE 12.30 Various activation functions. (a) Sigmoid. (b) Hyperbolic tangent (also has a sigmoid shape, but it is centered about 0 in both dimensions). (c) Rectifier linear unit (ReLU).

of w_{n+1} , as we do the perceptron. It is customary to use different notation, typically b , in neural networks to denote the bias term, so we are following convention. The more complicated notation used in Fig. 12.29, which we will explain shortly, is needed because we will be dealing with multilayer arrangements with several neurons per layer. We use the symbol “ ℓ ” to denote layers.

As you can see by comparing Figs. 12.29 and 12.23, we use variable z to denote the sum-of-products computed by the neuron. The output of the unit, denoted by a , is obtained by passing z through h . We call h the *activation function*, and refer to its output, $a = h(z)$, as the *activation value* of the unit. Note in Fig. 12.29 that the inputs to a neuron are activation values from neurons in the previous layer. Figure 12.30(a) shows a plot of $h(z)$ from Eq. (12-51). Because this function has the shape of a sigmoid function, the unit in Fig. 12.29 is sometimes called an *artificial sigmoid neuron*, or simply a *sigmoid neuron*. Its derivative has a very nice form, expressible in terms of $h(z)$ [see Problem 12.16(a)]:

$$h'(z) = \frac{\partial h(z)}{\partial z} = h(z)[1 - h(z)] \quad (12-52)$$

Figures 12.30(b) and (c) show two other forms of $h(z)$ used frequently. The hyperbolic tangent also has the shape of a sigmoid function, but it is symmetric about both axes. This property can help improve the convergence of the backpropagation algorithm to be discussed later. The function in Fig. 12.30(c) is called the *rectifier function*, and a unit using it is referred to a *rectifier linear unit* (ReLU). Often, you see the function itself referred to as the ReLU *activation function*. Experimental results suggest that this function tends to outperform the other two in deep neural networks.

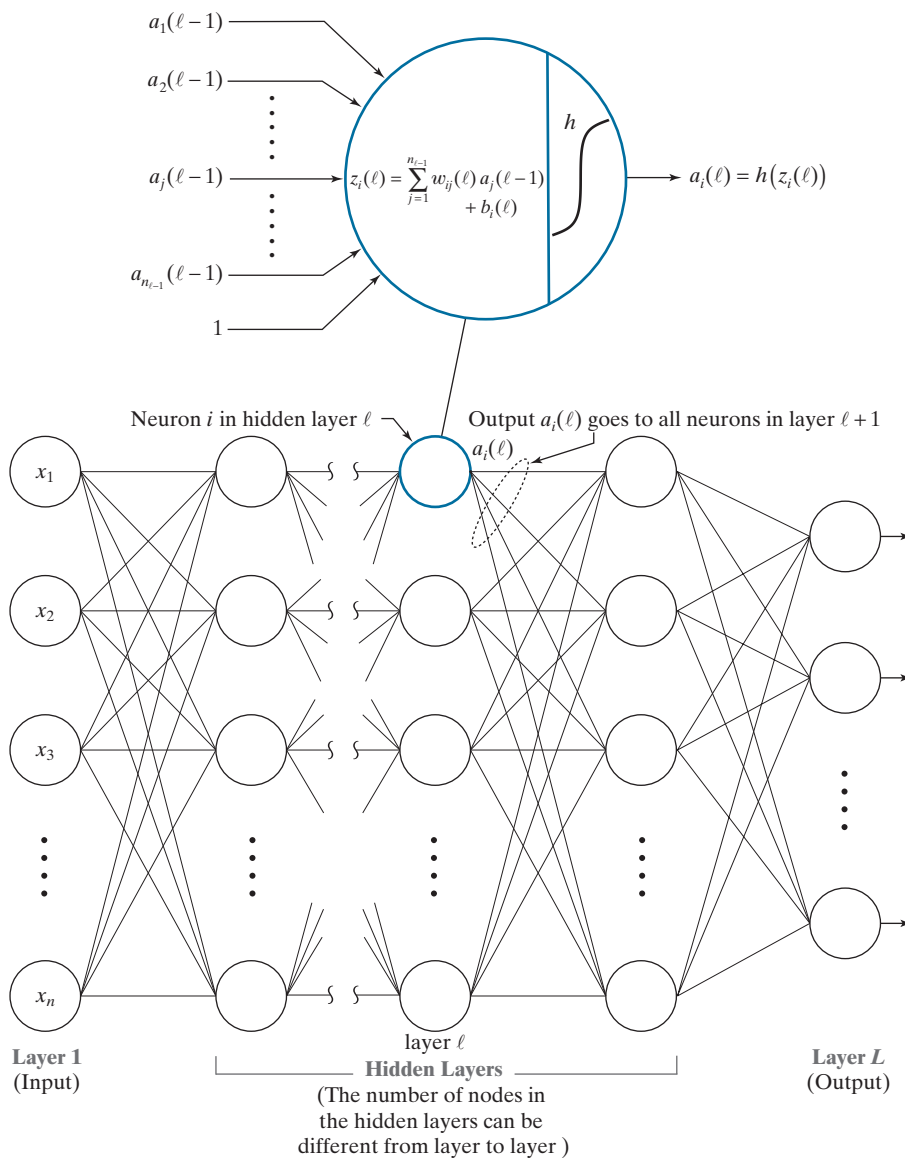
Interconnecting Neurons to Form a Fully Connected Neural Network

Figure 12.31 shows a generic diagram of a multilayer neural network. A *layer* in the network is the set of nodes (neurons) in a column of the network. As indicated by the zoomed node in Fig. 12.31, all the nodes in the network are artificial neurons of the form shown in Fig. 12.29, except for the input layer, whose nodes are the components of an input pattern vector $\mathbf{x}$. Therefore, the outputs (activation values) of the first layer are the values of the elements of $\mathbf{x}$. The outputs of all other nodes are the activation values of neurons in a particular layer. Each layer in the network can have a different number of nodes, but each node has a *single* output. The multiple lines shown at the outputs of the neurons in Fig. 12.31 indicate that the output of every node is connected to the input of all nodes in the next layer, to form a *fully connected* network. We also require that there be no loops in the network. Such networks are called *feedforward networks*. Fully connected, feedforward neural nets are the only types of networks considered in this section.

We obviously know the values of the nodes in the first layer, and we can observe the values of the output neurons. All others are *hidden neurons*, and the layers that contain them are called *hidden layers*. Generally, we call a neural net with a single hidden layer a *shallow neural network*, and refer to network with two or more hidden layers as a *deep neural network*. However, this terminology is not universal, and

FIGURE 12.31

General model of a feedforward, fully connected neural net. The neuron is the same as in Fig. 12.29. Note how the output of each neuron goes to the input of all neurons in the following layer, hence the name *fully connected* for this type of architecture.



sometimes you will see the words “shallow” and “deep” used subjectively to denote networks with a “few” and with “many” layers, respectively.

We used the notation in Eq. (12-37) to label all the inputs and weights of a perceptron. In a neural network, the notation is more complicated because we have to account for neuron weights, inputs, and outputs within a layer, and also from layer to layer. Ignoring layer notation for a moment, we denote by w_{ij} the weight that associates the link connecting the *output* of neuron j to the input of neuron i . That is,

the first subscript denotes the neuron that *receives* the signal, and the second refers to the neuron that *sends* the signal. Because i precedes j alphabetically, it would seem to make more sense for i to send and for j to receive. The reason we use the notation as stated is to avoid a matrix transposition in the equation that describes propagation of signals through the network. This notation is convention, but there is no doubt that it is confusing, so special care is necessary to keep the notation straight.

Remember, a bias is a weight that is always multiplied by 1.

Because the biases depend only on the neuron containing it, a single subscript that associates a bias with a neuron is sufficient. For example, we use b_i to denote the bias value associated with the i th neuron in a given layer of the network. Our use of b instead of w_{n+1} (as we did for perceptrons) follows notational convention used in neural networks. The weights, biases, and activation function(s) completely define a neural network. Although the activation function of any neuron in a neural network could be different from the others, there is no convincing evidence to suggest that there is anything to be gained by doing so. We assume in all subsequent discussions that the same form of activation function is used in all neurons.

Let ℓ denote a layer in the network, for $\ell = 1, 2, \dots, L$. With reference to Fig. 12.31, $\ell = 1$ denotes the input layer, $\ell = L$ is the output layer, and all other values of ℓ denote hidden layers. The number of neurons in layer ℓ is denoted n_ℓ . We have two options to include layer indexing in the parameters of a neural network. We can do it as a superscript, for example, w_{ij}^ℓ and b_i^ℓ ; or we can use the notation $w_{ij}(\ell)$ and $b_i(\ell)$. The first option is more prevalent in the literature on neural network. We use the second option because it is more consistent with the way we describe iterative expressions in the book, and also because you may find it easier to follow. Using this notation, the output (activation value) of neuron k in layer ℓ is denoted $a_k(\ell)$.

Keep in mind that our objective in using neural networks is the same as for perceptrons: to determine the class membership of unknown input patterns. The most common way to perform pattern classification using a neural network is to assign a class label to each output neuron. Thus, a neural network with n_L outputs can classify an unknown pattern into one of n_L classes. The network assigns an unknown pattern vector $\mathbf{x}$ to class c_k if output neuron k has the largest activation value; that is, if $a_k(L) > a_j(L)$, $j = 1, 2, \dots, n_L$; $j \neq k$.[†]

In this and the following section, the number of outputs of our neural networks will always equal the number of classes. But this is not a requirement. For instance, a network for classifying two pattern classes could be structured with a single output (Problem 12.17 illustrates such a case) because all we need for this task is two states, and a single neuron is capable of that. For three and four classes, we need three and four states, respectively, which can be achieved with two output neurons. Of course, the problem with this approach is that we would need additional logic to decipher the output combinations. It is simply more practical to have one neuron per output, and let the neuron with the highest output value determine the class of the input.

[†] Instead of a sigmoid or similar function in the final output layer, you will sometimes see a *softmax function* used instead. The concept is the same as we explained earlier, but the activation values in a softmax implementation are given by $a_i(L) = \exp[z_i(L)] / \sum_k \exp[z_k(L)]$, where the summation is over all outputs. In this formulation, the sum of all activations is 1, thus giving the outputs a probabilistic interpretation.

FORWARD PASS THROUGH A FEEDFORWARD NEURAL NETWORK

A *forward pass* through a neural network maps the input layer (i.e., values of $\mathbf{x}$) to the output layer. The values in the output layer are used for determining the class of an input vector. The equations developed in this section explain how a feedforward neural network carries out the computations that result in its output. Implicit in the discussion in this section is that the network parameters (weights and biases) are known. The important results in this section will be summarized in Table 12.2 at the end of our discussion, but understanding the material that gets us there is important when we discuss training of neural nets in the next section.

The Equations of a Forward Pass

The outputs of the layer 1 are the components of input vector $\mathbf{x}$:

$$a_j(1) = x_j \quad j = 1, 2, \dots, n_1 \quad (12-53)$$

where $n_1 = n$ is the dimensionality of $\mathbf{x}$. As illustrated in Figs. 12.29 and 12.31, the computation performed by neuron i in layer ℓ is given by

$$z_i(\ell) = \sum_{j=1}^{n_{\ell-1}} w_{ij}(\ell) a_j(\ell-1) + b_i(\ell) \quad (12-54)$$

for $i = 1, 2, \dots, n_\ell$ and $\ell = 2, \dots, L$. Quantity $z_i(\ell)$ is called the *net* (or *total*) *input* to neuron i in layer ℓ , and is sometimes denoted by net_i . The reason for this terminology is that $z_i(\ell)$ is formed using *all* outputs from layer $\ell - 1$. The output (activation value) of neuron i in layer ℓ is given by

$$a_i(\ell) = h(z_i(\ell)) \quad i = 1, 2, \dots, n_\ell \quad (12-55)$$

where h is an activation function. The value of network *output* node i is

$$a_i(L) = h(z_i(L)) \quad i = 1, 2, \dots, n_L \quad (12-56)$$

Equations (12-53) through (12-56) describe all the operations required to map the input of a fully connected feedforward network to its output.

EXAMPLE 12.10: Illustration of a forward pass through a fully connected neural network.

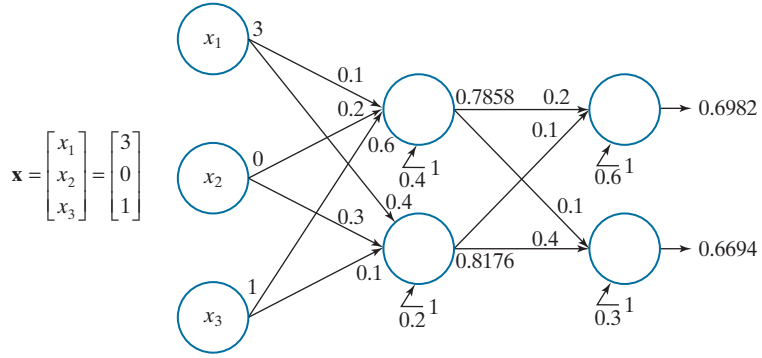
It will be helpful to consider a simple numerical example. Figure 12.32 shows a three-layer neural network consisting of the input layer, one hidden layer, and the output layer. The network accepts three inputs, and has two outputs. Thus, this network is capable of classifying 3-D patterns into one of two classes.

The numbers shown above the arrow heads on each input to a node are the weights of that node associated with the outputs from the nodes in the preceding layer. Similarly, the number shown in the output of each node is the activation value, a , of that node. As noted earlier, there is only one output value for each node, but it is routed to the input of every node in the next layer. The inputs associated with the 1's are bias values.

Let us look at the computations performed at each node, starting with the first (top) node in layer 2. We use Eq. (12-54) to compute the net input, $z_1(2)$, for that node:

FIGURE 12.32

A small, fully connected, feedforward net with labeled weights, biases, and outputs. The activation function is a sigmoid.



$$z_1(2) = \sum_{j=1}^3 w_{1j}(2)a_j(1) + b_1(2) = (0.1)(3) + (0.2)(0) + (0.6)(1) + 0.4 = 1.3$$

We obtain the output of this node using Eqs. (12-51) and (12-55):

$$a_1(2) = h(z_1(2)) = \frac{1}{1 + e^{-1.3}} = 0.7858$$

A similar computation gives the value for the output of the second node in the second layer,

$$z_2(2) = \sum_{j=1}^3 w_{2j}(2)a_j(1) + b_2(2) = (0.4)(3) + (0.3)(0) + (0.1)(1) + 0.2 = 1.5$$

and

$$a_2(2) = h(z_2(2)) = \frac{1}{1 + e^{-1.5}} = 0.8176$$

We use the outputs of the nodes in layer 2 to obtain the net values of the neurons in layer 3:

$$z_1(3) = \sum_{j=1}^2 w_{1j}(3)a_j(2) + b_1(3) = (0.2)(0.7858) + (0.1)(0.8176) + 0.6 = 0.8389$$

The output of this neuron is

$$a_1(3) = h(z_1(3)) = \frac{1}{1 + e^{-0.8389}} = 0.6982$$

Similarly,

$$z_2(3) = \sum_{j=1}^2 w_{2j}(3)a_j(2) + b_2(3) = (0.1)(0.7858) + (0.4)(0.8176) + 0.3 = 0.7056$$

and

$$a_2(3) = h(z_2(3)) = \frac{1}{1 + e^{-0.7056}} = 0.6694$$

If we were using this network to classify the input, we would say that pattern $\mathbf{x}$ belongs to class c_1 because $a_1(L) > a_2(L)$, where $L = 3$ and $n_L = 2$ in this case.

Matrix Formulation

The details of the preceding example reveal that there are numerous individual computations involved in a pass through a neural network. If you wrote a computer program to automate the steps we just discussed, you would find the code to be very inefficient because of all the required loop computations, the numerous node and layer indexing you would need, and so forth. We can develop a more elegant (and computationally faster) implementation by using matrix operations. This means writing Eqs. (12-53) through (12-55) as follows.

First, note that the number of outputs in layer 1 is always of the same dimension as an input pattern, $\mathbf{x}$, so its matrix (vector) form is simple:

$$\mathbf{a}(1) = \mathbf{x} \quad (12-57)$$

Next, we look at Eq. (12-54). We know that the summation term is just the inner product of two vectors [see Eqs. (12-37) and (12-38)]. However, this equation has to be evaluated for all nodes in every layer past the first. This implies that a loop is required if we do the computations node by node. The solution is to form a matrix, $\mathbf{W}(\ell)$, that contains *all* the weights in layer ℓ . The structure of this matrix is simple—each of its *rows* contains the weights for one of the nodes in layer ℓ :

$$\mathbf{W}(\ell) = \begin{bmatrix} w_{11}(\ell) & w_{12}(\ell) & \cdots & w_{1n_{\ell-1}}(\ell) \\ w_{21}(\ell) & w_{22}(\ell) & \cdots & w_{2n_{\ell-1}}(\ell) \\ \vdots & \vdots & \cdots & \vdots \\ w_{n_{\ell}1}(\ell) & w_{n_{\ell}2}(\ell) & \cdots & w_{n_{\ell}n_{\ell-1}}(\ell) \end{bmatrix} \quad (12-58)$$

With reference to our earlier discussion on the order of the subscripts i and j , if we had let i be the sending node and j the receiver, this matrix would have to be transposed.

Then, we can obtain all the sum-of-products computations, $z_i(\ell)$, for layer ℓ simultaneously:

$$\mathbf{z}(\ell) = \mathbf{W}(\ell)\mathbf{a}(\ell-1) + \mathbf{b}(\ell) \quad \ell = 2, 3, \dots, L \quad (12-59)$$

where $\mathbf{a}(\ell-1)$ is a column vector of dimension $n_{\ell-1} \times 1$ containing the outputs of layer $\ell-1$, $\mathbf{b}(\ell)$ is a column vector of dimension $n_{\ell} \times 1$ containing the bias values of all the neurons in layer ℓ , and $\mathbf{z}(\ell)$ is an $n_{\ell} \times 1$ column vector containing the net input values, $z_i(\ell)$, $i = 1, 2, \dots, n_{\ell}$, to all the nodes in layer ℓ . You can easily verify that Eq. (12-59) is dimensionally correct.

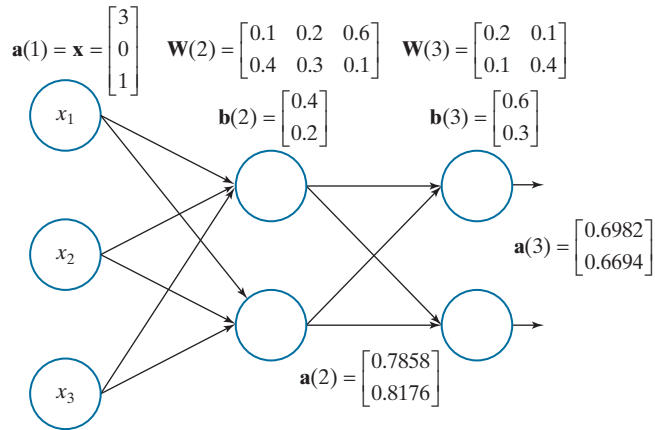
Because the activation function is applied to each net input independently of the others, the outputs of the network at any layer can be expressed in vector form as:

$$\mathbf{a}(\ell) = h[\mathbf{z}(\ell)] = \begin{bmatrix} h(z_1(\ell)) \\ h(z_2(\ell)) \\ \vdots \\ h(z_{n_{\ell}}(\ell)) \end{bmatrix} \quad (12-60)$$

Implementing Eqs. (12-57) through (12-60) requires just a series of matrix operations, with no loops.

FIGURE 12.33

Same as Fig. 12.32, but using matrix labeling.



EXAMPLE 12.11: Redoing Example 12.10 using matrix operations.

Figure 12.33 shows the same neural network as in Fig. 12.32, but with all its parameters shown in matrix form. As you can see, the representation in Fig. 12.33 is more compact. Starting with

$$\mathbf{a}(1) = \begin{bmatrix} 3 \\ 0 \\ 1 \end{bmatrix}$$

it follows that

$$\mathbf{z}(2) = \mathbf{W}(2)\mathbf{a}(1) + \mathbf{b}(2) = \begin{bmatrix} 0.1 & 0.2 & 0.6 \\ 0.4 & 0.3 & 0.1 \end{bmatrix} \begin{bmatrix} 3 \\ 0 \\ 1 \end{bmatrix} + \begin{bmatrix} 0.4 \\ 0.2 \end{bmatrix} = \begin{bmatrix} 1.3 \\ 1.5 \end{bmatrix}$$

Then,

$$\mathbf{a}(2) = h[\mathbf{z}(2)] = \begin{bmatrix} h(z_1(2)) \\ h(z_2(2)) \end{bmatrix} = \begin{bmatrix} h(1.3) \\ h(1.5) \end{bmatrix} = \begin{bmatrix} 0.7858 \\ 0.8176 \end{bmatrix}$$

With $\mathbf{a}(2)$ as input to the next layer, we obtain

$$\mathbf{z}(3) = \mathbf{W}(3)\mathbf{a}(2) + \mathbf{b}(3) = \begin{bmatrix} 0.2 & 0.1 \\ 0.1 & 0.4 \end{bmatrix} \begin{bmatrix} 0.7858 \\ 0.8176 \end{bmatrix} + \begin{bmatrix} 0.6 \\ 0.3 \end{bmatrix} = \begin{bmatrix} 0.8389 \\ 0.7056 \end{bmatrix}$$

and, as before,

$$\mathbf{a}(3) = h[\mathbf{z}(3)] = \begin{bmatrix} h(z_1(3)) \\ h(z_2(3)) \end{bmatrix} = \begin{bmatrix} h(0.8389) \\ h(0.7056) \end{bmatrix} = \begin{bmatrix} 0.6982 \\ 0.6694 \end{bmatrix}$$

The clarity of the matrix formulation over the indexed notation used in Example 12.10 is evident.

Equations (12-57) through (12-60) are a significant improvement over node-by-node computations, but they apply only to one pattern. To classify multiple pattern vectors, we would have to loop through each pattern using the same set of matrix equations per loop iteration. What we are after is one set of matrix equations

capable of processing *all* patterns in a single forward pass. Extending Eqs. (12-57) through (12-60) to this more general formulation is straightforward. We begin by arranging all our input pattern vectors as *columns* of a single matrix, $\mathbf{X}$, of dimension $n \times n_p$ where, as before, n is the dimensionality of the vectors and n_p is the number of pattern vectors. It follows from Eq. (12-57) that

$$\mathbf{A}(1) = \mathbf{X} \quad (12-61)$$

where each column of matrix $\mathbf{A}(1)$ contains the initial activation values (i.e., the vector values) for one pattern. This is a straightforward extension of Eq. (12-57), except that we are now dealing with an $n \times n_p$ matrix instead of an $n \times 1$ vector.

The parameters of a network do not change because we are processing more pattern vectors, so the weight matrix is as given in Eq. (12-58). This matrix is of size $n_\ell \times n_{\ell-1}$. When $\ell = 2$, we have that $\mathbf{W}(2)$ is of size $n_2 \times n$, because n_1 is always equal to n . Then, extending the product term of Eq. (12-59) to use $\mathbf{A}(2)$ instead of $\mathbf{a}(2)$, results in the matrix product $\mathbf{W}(2)\mathbf{A}(2)$, which is of size $(n_2 \times n)(n \times n_p) = n_2 \times n_p$. To this, we have to add the bias vector for the second layer, which is of size $n_2 \times 1$. Obviously, we cannot add a matrix of size $n_2 \times n_p$ and a vector of size $n_2 \times 1$. However, as is true of the weight matrices, the bias vectors do not change because we are processing more pattern vectors. We just have to account for one identical bias vector, $\mathbf{b}(2)$, per input vector. We do this by creating a matrix $\mathbf{B}(2)$ of size $n_2 \times n_p$, formed by concatenating column vector $\mathbf{b}(2)$ n_p times, horizontally. Then, Eq. (12-59) written in matrix becomes $\mathbf{Z}(2) = \mathbf{W}(2)\mathbf{A}(1) + \mathbf{B}(2)$. Matrix $\mathbf{Z}(2)$ is of size $n_2 \times n_p$; it contains the computation performed by Eq. (12-59), but for *all* input patterns. That is, each column of $\mathbf{Z}(2)$ is exactly the computation performed by Eq. (12-59) for one input pattern.

The concept just discussed applies to the transition from any layer to the next in the neural network, provided that we use the weights and bias appropriate for a particular location in the network. Therefore, the full matrix version of Eq. (12-59) is

$$\mathbf{Z}(\ell) = \mathbf{W}(\ell)\mathbf{A}(\ell - 1) + \mathbf{B}(\ell) \quad (12-62)$$

where $\mathbf{W}(\ell)$ is given by Eq. (12-58) and $\mathbf{B}(\ell)$ is an $n_\ell \times n_p$ matrix whose columns are duplicates of $\mathbf{b}(\ell)$, the bias vector containing the biases of the neurons in layer ℓ .

All that remains is the matrix formulation of the output of layer ℓ . As Eq. (12-60) shows, the activation function is applied independently to each element of the vector $\mathbf{z}(\ell)$. Because each column of $\mathbf{Z}(\ell)$ is simply the application of Eq. (12-60) corresponding to a particular input vector, it follows that

$$\mathbf{A}(\ell) = h[\mathbf{Z}(\ell)] \quad (12-63)$$

where activation function h is applied to each element of matrix $\mathbf{Z}(\ell)$.

Summarizing the dimensions in our matrix formulation, we have: $\mathbf{X}$ and $\mathbf{A}(1)$ are of size $n \times n_p$, $\mathbf{Z}(\ell)$ is of size $n_\ell \times n_p$, $\mathbf{W}(\ell)$ is of size $n_\ell \times n_{\ell-1}$, $\mathbf{A}(\ell - 1)$ is of

TABLE 12.2

Steps in the matrix computation of a forward pass through a fully connected, feedforward multilayer neural net.

Step	Description	Equations
Step 1	Input patterns	$\mathbf{A}(1) = \mathbf{X}$
Step 2	Feedforward	For $\ell = 2, \dots, L$, compute $\mathbf{Z}(\ell) = \mathbf{W}(\ell)\mathbf{A}(\ell - 1) + \mathbf{B}(\ell)$ and $\mathbf{A}(\ell) = h(\mathbf{Z}(\ell))$
Step 3	Output	$\mathbf{A}(L) = h(\mathbf{Z}(L))$

size $n_{\ell-1} \times n_p$, $\mathbf{B}(\ell)$ is of size $n_\ell \times n_p$, and $\mathbf{A}(\ell)$ is of size $n_\ell \times n_p$. Table 12.2 summarizes the matrix formulation for the forward pass through a fully connected, feedforward neural network for all pattern vectors. Implementing these operations in a matrix-oriented language like MATLAB is a trivial undertaking. Performance can be improved significantly by using dedicated hardware, such as one or more graphics processing units (GPUs).

The equations in Table 12.2 are used to classify each of a set of patterns into one of n_L pattern classes. Each column of output matrix $\mathbf{A}(L)$ contains the activation values of the n_L output neurons for a specific pattern vector. The class membership of that pattern is given by the location of the output neuron with the highest activation value. Of course, this assumes we know the weights and biases of the network. These are obtained during training using backpropagation, as we explain next.

USING BACKPROPAGATION TO TRAIN DEEP NEURAL NETWORKS

A neural network is defined completely by its weights, biases, and activation function. Training a neural network refers to using one or more sets of training patterns to estimate these parameters. During training, we know the desired response of every output neuron of a multilayer neural net. However, we have no way of knowing what the values of the outputs of hidden neurons should be. In this section, we develop the equations of *backpropagation*, the tool of choice for finding the value of the weights and biases in a multilayer network. This *training by backpropagation* involves four basic steps: (1) inputting the pattern vectors; (2) a forward pass through the network to classify all the patterns of the training set and determine the classification error; (3) a backward (backpropagation) pass that feeds the output error back through the network to compute the changes required to update the parameters; and (4) updating the weights and biases in the network. These steps are repeated until the error reaches an acceptable level. We will provide a summary of all principal results derived in this section at the end of the discussion (see Table 12.3). As you will see shortly, the principal mathematical tool needed to derive the equations of backpropagation is the chain rule from basic calculus.

The Equations of Backpropagation

Given a set of training patterns and a multilayer feedforward neural network architecture, the approach in the following discussion is to find the network parameters

that minimize an *error* (also called *cost* or *objective*) *function*. Our interest is in classification performance, so we define the error function for a neural network as the average of the differences between desired and actual responses. Let $\mathbf{r}$ denote the desired response for a given pattern vector, $\mathbf{x}$, and let $\mathbf{a}(L)$ denote the actual response of the network to that input. For example, in a ten-class recognition application, $\mathbf{r}$ and $\mathbf{a}(L)$ would be 10-D column vectors. The ten components of $\mathbf{a}(L)$ would be the ten outputs of the neural network, and the components of $\mathbf{r}$ would be zero, except for the element corresponding to the class of $\mathbf{x}$, which would be 1. For example, if the input training pattern belongs to class 6, the 6th element of $\mathbf{r}$ would be 1 and the rest would be 0's.

The activation values of neuron j in the output layer is $a_j(L)$. We define the error of that neuron as

$$E_j = \frac{1}{2} (r_j - a_j(L))^2 \quad (12-64)$$

for $j = 1, 2, \dots, n_L$, where r_j is the desired response of output neuron $a_j(L)$ for a given pattern $\mathbf{x}$. The output error with respect to a single $\mathbf{x}$ is the sum of the errors of all output neurons with respect to that vector:

$$\begin{aligned} E &= \sum_{j=1}^{n_L} E_j = \frac{1}{2} \sum_{j=1}^{n_L} (r_j - a_j(L))^2 \\ &= \frac{1}{2} \|\mathbf{r} - \mathbf{a}(L)\|^2 \end{aligned} \quad (12-65)$$

See Eqs. (2-50) and (2-51) regarding the Euclidean vector norm.

When the meaning is clear, we sometimes include the bias term in the word “weights.”

where the second line follows from the definition of the Euclidean vector norm. The *total network output error* over all training patterns is defined as the sum of the errors of the individual patterns. We want to find the weights that minimize this total error. As we did for the LMSE perceptron, we find the solution using gradient descent. However, unlike the perceptron, we have no way for computing the gradients of the weights in the hidden nodes. The beauty of backpropagation is that we can achieve an equivalent result by propagating the output error back into the network.

The key objective is to find a scheme to adjust all weights in a network using training patterns. In order to do this, we need to know how E changes with respect to the weights in the network. The weights are contained in the expression for the net input to each node [see Eq. (12-54)], so the quantity we are after is $\partial E / \partial z_j(\ell)$ where, as defined in Eq. (12-54), $z_j(\ell)$ is the net input to node j in layer ℓ . In order to simplify the notation later, we use the symbol $\delta_j(\ell)$ to denote $\partial E / \partial z_j(\ell)$. Because backpropagation starts with the output and works backward from there, we look first at

$$\delta_j(L) = \frac{\partial E}{\partial z_j(L)} \quad (12-66)$$

We can express this equation in terms of the output $a_j(L)$ using the chain rule:

We use “ j ” generically to mean any node in the network. We are not concerned at the moment with inputs to, or outputs from, a node.

$$\begin{aligned}
\delta_j(L) &= \frac{\partial E}{\partial z_j(L)} = \frac{\partial E}{\partial a_j(L)} \frac{\partial a_j(L)}{\partial z_j(L)} = \frac{\partial E}{\partial a_j(L)} \frac{\partial h(z_j(L))}{\partial z_j(L)} \\
&= \frac{\partial E}{\partial a_j(L)} h'(z_j(L))
\end{aligned} \tag{12-67}$$

where we used Eq. (12-56) to obtain the last expression in the first line. This equation gives us the value of $\delta_j(L)$ in terms of quantities that can be observed or computed. For example, if we use Eq. (12-64) as our error measure, and Eq. (12-52) for $h'(z_j(x))$, then

$$\delta_j(L) = h(z_j(L)) [1 - h(z_j(L))] [a_j(L) - r_j] \tag{12-68}$$

where we interchanged the order of the terms. The $h(z_j(L))$ are computed in the forward pass, $a_j(L)$ can be observed in the output of the network, and r_j is given along with $\mathbf{x}$ during training. Therefore, we can compute $\delta_j(L)$.

Because the relationship between the net input and the output of any neuron in any layer (except the first) is the same, the form of Eq. (12-66) is valid for any node j in any hidden layer:

$$\delta_j(\ell) = \frac{\partial E}{\partial z_j(\ell)} \tag{12-69}$$

This equation tells us how E changes with respect to a change in the net input to any neuron in the network. What we want to do next is express $\delta_j(\ell)$ in terms of $\delta_j(\ell + 1)$. Because we will be proceeding backward in the network, this means that if we have this relationship, then we can start with $\delta_j(L)$ and find $\delta_j(L - 1)$. We then use this result to find $\delta_j(L - 2)$, and so on until we arrive at layer 2. We obtain the desired expression using the chain rule (see Problem 12.25):

$$\begin{aligned}
\delta_j(\ell) &= \frac{\partial E}{\partial z_j(\ell)} = \sum_i \frac{\partial E}{\partial z_i(\ell + 1)} \frac{\partial z_i(\ell + 1)}{\partial a_j(\ell)} \frac{\partial a_j(\ell)}{\partial z_j(\ell)} \\
&= \sum_i \delta_i(\ell + 1) \frac{\partial z_i(\ell + 1)}{\partial a_j(\ell)} h'(z_j(\ell)) \\
&= h'(z_j(\ell)) \sum_i w_{ij}(\ell + 1) \delta_i(\ell + 1)
\end{aligned} \tag{12-70}$$

for $\ell = L - 1, L - 2, \dots, 2$, where we used Eqs. (12-55) and (12-69) to obtain the middle line, and Eq. (12-54), plus some rearranging to obtain the last line.

The preceding development tells us how we can start with the error in the output (which we can compute) and obtain how that error changes as function of the net inputs to every node in the network. This is an intermediate step toward our final objective, which is to obtain expressions for $\partial E / \partial w_{ij}(\ell)$ and $\partial E / \partial b_i(\ell)$ in terms of $\delta_j(\ell) = \partial E / \partial z_j(\ell)$. For this, we use the chain rule again:

$$\begin{aligned}
\frac{\partial E}{\partial w_{ij}(\ell)} &= \frac{\partial E}{\partial z_i(\ell)} \frac{\partial z_i(\ell)}{\partial w_{ij}(\ell)} \\
&= \delta_i(\ell) \frac{\partial z_i(\ell)}{\partial w_{ij}(\ell)} \\
&= a_j(\ell - 1) \delta_i(\ell)
\end{aligned} \tag{12-71}$$

where we used Eq. (12-54), Eq. (12-69), and interchanged the order of the results to clarify matrix formulations later in our discussion. Similarly (see Problem 12.26),

$$\frac{\partial E}{\partial b_i(\ell)} = \delta_i(\ell) \tag{12-72}$$

Now we have the rate of change of E with respect to the network weights and biases in terms of quantities we can compute. The last step is to use these results to update the network parameters using gradient descent:

$$\begin{aligned}
w_{ij}(\ell) &= w_{ij}(\ell) - \alpha \frac{\partial E(\ell)}{\partial w_{ij}(\ell)} \\
&= w_{ij}(\ell) - \alpha \delta_i(\ell) a_j(\ell - 1)
\end{aligned} \tag{12-73}$$

and

$$\begin{aligned}
b_i(\ell) &= b_i(\ell) - \alpha \frac{\partial E}{\partial b_i(\ell)} \\
&= b_i(\ell) - \alpha \delta_i(\ell)
\end{aligned} \tag{12-74}$$

for $\ell = L - 1, L - 2, \dots, 2$, where the a 's are computed in the forward pass, and the δ 's are computed during backpropagation. As with the perceptron, α is the learning rate constant used in gradient descent. There are numerous approaches that attempt to find optimal learning rates, but ultimately this is a problem-dependent parameter that involves experimenting. A reasonable approach is to start with a small value of α (e.g., 0.01), then experiment with vectors from the training set to determine a suitable value in a given application. Remember, α is used only during training, so it has no effect on post-training operating performance.

Matrix Formulation

As with the equations that describe the forward pass through a neural network, the equations of backpropagation developed in the previous discussion are excellent for describing how the method works at a fundamental level, but they are clumsy when it comes to implementation. In this section, we follow a procedure similar to the one we used for the forward pass to develop the matrix equations for backpropagation.

As before, we arrange all the pattern vectors as columns of matrix $\mathbf{X}$, and package the weights of layer ℓ as matrix $\mathbf{W}(\ell)$. We use $\mathbf{D}(\ell)$ to denote the matrix equivalent of $\delta(\ell)$, the vector containing the errors in layer ℓ . Our first step is to find an expression for $\mathbf{D}(L)$. We begin at the output and proceed backward, as before. From Eq. (12-67),

$$\delta(L) = \begin{bmatrix} \delta_1(L) \\ \delta_2(L) \\ \vdots \\ \delta_{n_L}(L) \end{bmatrix} = \begin{bmatrix} \frac{\partial E}{\partial a_1(L)} h'(z_1(L)) \\ \frac{\partial E}{\partial a_2(L)} h'(z_2(L)) \\ \vdots \\ \frac{\partial E}{\partial a_{n_L}(L)} h'(z_{n_L}(L)) \end{bmatrix} = \begin{bmatrix} \frac{\partial E}{\partial a_1(L)} \\ \frac{\partial E}{\partial a_2(L)} \\ \vdots \\ \frac{\partial E}{\partial a_{n_L}(L)} \end{bmatrix} \odot \begin{bmatrix} h'(z_1(L)) \\ h'(z_2(L)) \\ \vdots \\ h'(z_{n_L}(L)) \end{bmatrix} \quad (12-75)$$

where, as defined in Section 2.6, “ $\odot$ ” denotes elementwise multiplication (of two vectors in this case). We can write the vector on the left of this symbol as $\partial E / \partial \mathbf{a}(L)$, and the vector on the right as $h'(\mathbf{z}(L))$. Then, we can write Eq. (12-75) as

$$\delta(L) = \frac{\partial E}{\partial \mathbf{a}(L)} \odot h'(\mathbf{z}(L)) \quad (12-76)$$

This $n_L \times 1$ column vector contains the activation values of all the output neurons for *one* pattern vector. The only error function we use in this chapter is a quadratic function, which is given in vector form in Eq. (12-65). The partial of that quadratic function with respect to $\mathbf{a}(L)$ is $(\mathbf{a}(L) - \mathbf{r})$ which, when substituted into Eq. (12-76), gives us

$$\delta(L) = (\mathbf{a}(L) - \mathbf{r}) \odot h'(\mathbf{z}(L)) \quad (12-77)$$

Column vector $\delta(L)$ accounts for one pattern vector. To account for *all* n_p patterns simultaneously we form a matrix $\mathbf{D}(\ell)$, whose columns are the $\delta(L)$ from Eq. (12-77), evaluated for a specific pattern vector. This is equivalent to writing Eq. (12-77) directly in matrix form as

$$\mathbf{D}(L) = (\mathbf{A}(L) - \mathbf{R}) \odot h'(\mathbf{Z}(L)) \quad (12-78)$$

Each column of $\mathbf{A}(L)$ is the network output for one pattern. Similarly, each column of $\mathbf{R}$ is a binary vector with a 1 in the location corresponding to the class of a particular pattern vector, and 0's elsewhere, as explained earlier. Each column of the difference $(\mathbf{A}(L) - \mathbf{R})$ contains the components of $\|\mathbf{a} - \mathbf{r}\|$. Therefore, squaring the elements of a column, adding them, and dividing by 2 is the same as computing the error measure defined in Eq. (12-65), for one pattern. Adding all the column computations gives an average measure of error for all the patterns. Similarly, the columns of matrix $h'(\mathbf{Z}(L))$ are values of the net inputs to all output neurons, with

each column corresponding to one pattern vector. All matrices in Eq. (12-78) are of size $n_L \times n_p$.

Following a similar line of reasoning, we can express Eq. (12-70) in matrix form as

$$\mathbf{D}(\ell) = (\mathbf{W}^T(\ell+1)\mathbf{D}(\ell+1)) \odot h'(\mathbf{Z}(\ell)) \quad (12-79)$$

It is easily confirmed by dimensional analysis that the matrix $\mathbf{D}(\ell)$ is of size $n_\ell \times n_p$ (see Problem 12.27). Note that Eq. (12-79) uses the weight matrix transposed. This reflects the fact that the inputs to layer ℓ are coming from layer $\ell+1$, because in backpropagation we move in the direction opposite of a forward pass.

We complete the matrix formulation by expressing the weight and bias update equations in matrix form. Considering the weight matrix first, we can tell from Eqs. (12-70) and (12-73) that we are going to need matrices $\mathbf{W}(\ell)$, $\mathbf{D}(\ell)$, and $\mathbf{A}(\ell-1)$. We already know that $\mathbf{W}(\ell)$ is of size $n_\ell \times n_{\ell-1}$ and that $\mathbf{D}(\ell)$ is of size $n_\ell \times n_p$. Each column of matrix $\mathbf{A}(\ell-1)$ is the set of outputs of the neurons in layer $\ell-1$ for one pattern vector. There are n_p patterns, so $\mathbf{A}(\ell-1)$ is of size $n_{\ell-1} \times n_p$. From Eq. (12-73) we infer that $\mathbf{A}$ post-multiplies $\mathbf{D}$, so we are also going to need $\mathbf{A}^T(\ell-1)$, which is of size $n_p \times n_{\ell-1}$. Finally, recall that in a matrix formulation, we construct a matrix $\mathbf{B}(\ell)$ of size $n_\ell \times n_p$ whose columns are copies of vector $\mathbf{b}(\ell)$, which contains all the biases in layer ℓ .

Next, we look at updating the biases. We know from Eq. (12-74) that each element $b_i(\ell)$ of $\mathbf{b}(\ell)$ is updated as $b_i(\ell) = b_i(\ell) - \alpha \delta_i(\ell)$, for $i = 1, 2, \dots, n_\ell$. Therefore, $\mathbf{b}(\ell) = \mathbf{b}(\ell) - \alpha \boldsymbol{\delta}(\ell)$. But this is for *one* pattern, and the columns of $\mathbf{D}(\ell)$ are the $\boldsymbol{\delta}(\ell)$'s for *all* patterns in the training set. This is handled in a matrix formulation by using the *average* of the columns of $\mathbf{D}(\ell)$ (this is the average error over all patterns) to update $\mathbf{b}(\ell)$.

Putting it all together results in the following two equations for updating the network parameters:

$$\mathbf{W}(\ell) = \mathbf{W}(\ell) - \alpha \mathbf{D}(\ell) \mathbf{A}^T(\ell-1) \quad (12-80)$$

and

$$\mathbf{b}(\ell) = \mathbf{b}(\ell) - \alpha \sum_{k=1}^{n_p} \boldsymbol{\delta}_k(\ell) \quad (12-81)$$

where $\boldsymbol{\delta}_k(\ell)$ is the k th column of matrix $\mathbf{D}(\ell)$. As before, we form matrix $\mathbf{B}(\ell)$ of size $n_\ell \times n_p$ by concatenating $\mathbf{b}(\ell)$ n_p times in the horizontal direction:

$$\mathbf{B}(\ell) = \underset{n_p \text{ times}}{\text{concatenate}} \{ \mathbf{b}(\ell) \} \quad (12-82)$$

As we mentioned earlier, backpropagation consists of four principal steps: (1) inputting the patterns, (2) a forward pass, (3) a backpropagation pass, and (4) a parameter update step. The process begins by specifying the initial weights and biases as (small) random numbers. Table 12.3 summarizes the matrix formulations of these four steps. During training, these steps are repeated for a number of specified epochs, or until a predefined measure of error is deemed to be small enough.

TABLE 12.3

Matrix formulation for training a feedforward, fully connected multilayer neural network using backpropagation. Steps 1–4 are for one epoch of training. $\mathbf{X}$, $\mathbf{R}$, and the learning rate parameter α , are provided to the network for training. The network is initialized by specifying weights, $\mathbf{W}(1)$, and biases, $\mathbf{B}(1)$, as small random numbers.

Step	Description	Equations
Step 1	Input patterns	$\mathbf{A}(1) = \mathbf{X}$
Step 2	Forward pass	For $\ell = 2, \dots, L$, compute: $\mathbf{Z}(\ell) = \mathbf{W}(\ell)\mathbf{A}(\ell-1) + \mathbf{B}(\ell)$; $\mathbf{A}(\ell) = h(\mathbf{Z}(\ell))$; $h'(\mathbf{Z}(\ell))$; and $\mathbf{D}(L) = (\mathbf{A}(L) - \mathbf{R}) \odot h'(\mathbf{Z}(L))$
Step 3	Backpropagation	For $\ell = L-1, L-2, \dots, 2$, compute $\mathbf{D}(\ell) = (\mathbf{W}^T(\ell+1)\mathbf{D}(\ell+1)) \odot h'(\mathbf{Z}(\ell))$
Step 4	Update weights and biases	For $\ell = 2, \dots, L$, let $\mathbf{W}(\ell) = \mathbf{W}(\ell) - \alpha \mathbf{D}(\ell) \mathbf{A}^T(\ell-1)$, $\mathbf{b}(\ell) = \mathbf{b}(\ell) - \alpha \sum_{k=1}^{n_p} \delta_k(\ell)$, and $\mathbf{B}(\ell) = \text{concatenate}_{n_p \text{ times}} \{\mathbf{b}(\ell)\}$, where the $\delta_k(\ell)$ are the columns of $\mathbf{D}(\ell)$

There are two major types of errors in which we are interested. One is the *classification error*, which we compute by counting the number of patterns that were misclassified and dividing by the total number of patterns in the training set. Multiplying the result by 100 gives the percentage of patterns misclassified. Subtracting the result from 1 and multiplying by 100 gives the percent correct recognition. The other is the *mean squared error* (MSE), which is based on actual values of E . For the error defined in Eq. (12-65), this value is obtained (for one pattern) by squaring the elements of a column of the matrix $(\mathbf{A}(L) - \mathbf{R})$, adding them, and dividing by the result by 2 (see Problem 12.28). Repeating this operation for all columns and dividing the result by the number of patterns in $\mathbf{X}$ gives the MSE over the entire training set.

EXAMPLE 12.12: Using a fully connected neural net to solve the XOR problem.

Figure 12.34(a) shows the XOR classification problem discussed previously (the coordinates were chosen to center the patterns for convenience in indexing, but the spatial relationships are as before). Pattern matrix $\mathbf{X}$ and class membership matrix $\mathbf{R}$ are:

$$\mathbf{X} = \begin{bmatrix} 1 & -1 & -1 & 1 \\ 1 & -1 & 1 & -1 \end{bmatrix}; \quad \mathbf{R} = \begin{bmatrix} 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \end{bmatrix}$$

We specified a neural network having three layers, with two nodes each (see Fig. 12.35). This is the smallest network consistent with our architecture in Fig. 12.31. Comparing it to the minimum perceptron arrangements in Fig. 12.28(a), we see that our neural network performs the same basic function, in the sense that it has two inputs and two outputs.

We used $\alpha = 1.0$, an initial set of Gaussian random weights of zero mean and standard deviation of 0.02, and the activation function in Eq. (12-51). We then trained the network for 10,000 epochs (we used a large number of epochs to get close to the values in the $\mathbf{R}$; we discuss below solutions with fewer epochs). The resulting weights and biases were:

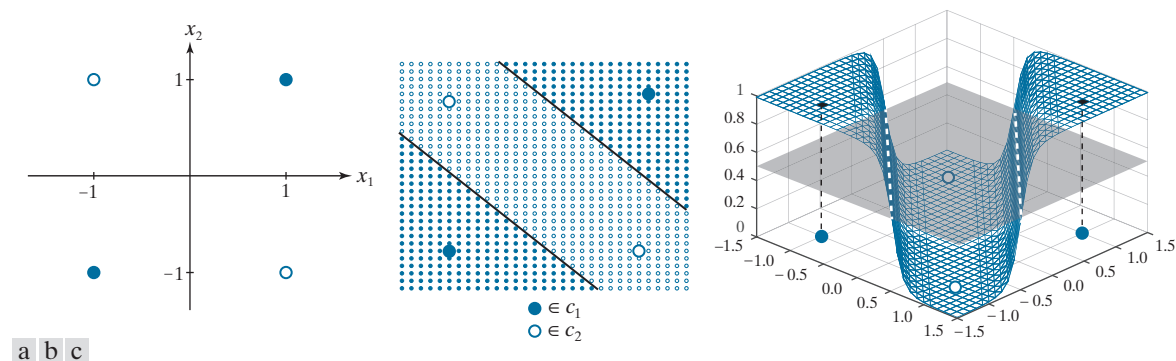


FIGURE 12.34 Neural net solution to the XOR problem. (a) Four patterns in an XOR arrangement. (b) Results of classifying additional points in the range -1.5 to 1.5 in increments of 0.1 . All solid points were classified as belonging to class c_1 and all open circles were classified as belonging to class c_2 . Together, the two lines separating the regions constitute the decision boundary [compare with Fig. 12.27(b)]. (c) Decision surface, shown as a mesh. The decision boundary is the pair of dashed, white lines in the intersection of the surface and a plane perpendicular to the vertical axis, intersecting that axis at 0.5 . (Figure (c) is shown in a different perspective than (b) in order to make all four patterns visible.)

$$\mathbf{W}(2) = \begin{bmatrix} 4.792 & 4.792 \\ 4.486 & 4.486 \end{bmatrix}; \quad \mathbf{b}(2) = \begin{bmatrix} 4.590 \\ -4.486 \end{bmatrix}; \quad \mathbf{W}(3) = \begin{bmatrix} -9.180 & 9.429 \\ 9.178 & -9.427 \end{bmatrix}; \quad \mathbf{b}(3) = \begin{bmatrix} 4.420 \\ -4.419 \end{bmatrix}$$

Figure 12.35 shows the neural net based on these values.

When presented with the four training patterns after training was completed, the results at the two outputs should have been equal to the values in $\mathbf{R}$. Instead, the values were close:

$$\mathbf{A}(3) = \begin{bmatrix} 0.987 & 0.990 & 0.010 & 0.010 \\ 0.013 & 0.010 & 0.990 & 0.990 \end{bmatrix}$$

These weights and biases, along with the sigmoid activation function, completely specify our trained neural network. To test its performance with values other than the training patterns, which we know it classifies correctly, we created a set of 2-D test patterns by subdividing the pattern space into increments of 0.1 , from -1.5 to 1.5 in both directions, and classified the resulting points using a forward pass through

FIGURE 12.35

Neural net used to solve the XOR problem, showing the weights and biases learned via training using the equations in Table 12.3.

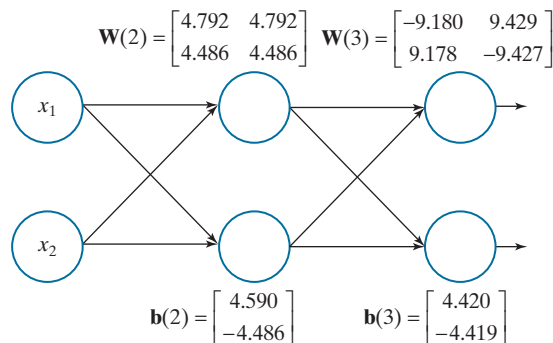
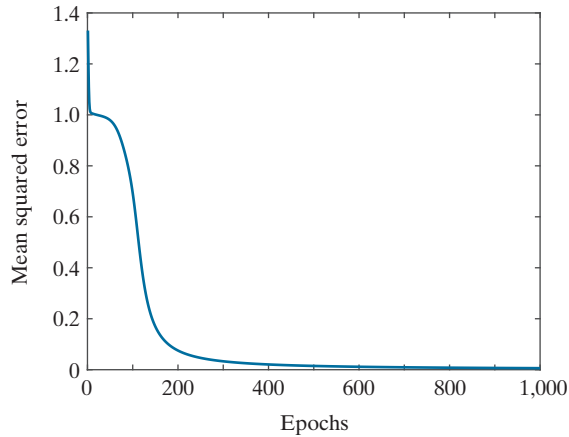


FIGURE 12.36

MSE as a function of training epochs for the XOR pattern arrangement.



the network. If the activation value of output node 1 was greater than the activation value of output node 2, the pattern was assigned to class c_1 ; otherwise, it was assigned to class c_2 . Fig. 12.34(b) is a plot of the results. Solid dots are points classified into to class c_1 , and white dots were classified as belonging to class c_2 . The boundaries between these two regions (shown as solid black lines) are precisely the boundaries in Fig. 12.27(b). Thus, our small neural network found the simplest boundary between the two classes, and thus performed the same function as the perceptron arrangement in Fig. 12.28(a).

Figure 12.34(c) shows the decision surface. This figure is analogous to Fig. 12.24(b), but it intersects the plane twice because the patterns are not linearly separable. Our decision boundary is the intersection of the decision surface with a plane perpendicular to the vertical axis, and intersecting that axis at 0.5. This is because the range of values in the output nodes is in the $[0, 1]$ range, and we assign a pattern to the class for which one the two outputs had the largest value. The plane is shown shaded in the figure, and the decision boundary is shown as dashed white lines. We adjusted the viewing perspective of Fig. 12.34(c) so you can see all the XOR points.

Because classification in this case is based on selecting the largest output, we do not need the outputs to be so close to 1 and 0 as we showed above, provided they are greater for the patterns of class c_1 and conversely for the patterns of class c_2 . This means that we can train the network using fewer epochs and still achieve correct recognition. For example, correct classification of the XOR patterns can be achieved using the parameters learned with as few as 150 epochs. Figure 12.36 shows the reason why this is possible. By the end of the 1000th epoch, the mean squared error has decreased almost to zero, so we would expect it to decrease very little from there for 10,000 epochs. We know from the preceding results that the neural net performed flawlessly using the weights learned with 10,000 epochs. Because the error for 1,000 and 10,000 epochs is close, we can expect the weights to be close as well. At 150 epochs, the error has decreased by close to 90% from its maximum, so the probability that the weights would perform well should be reasonably high, which was true in this case.

EXAMPLE 12.13: Using neural nets to classify multispectral image data.

In this example, we compare the recognition performance of the Bayes classifier we discussed in Section 12.4 and the multilayer neural nets discussed in this section. The objective here is the same as in Example 12.6: to classify the pixels of multispectral image data into three pattern classes: *water*, *urban*,

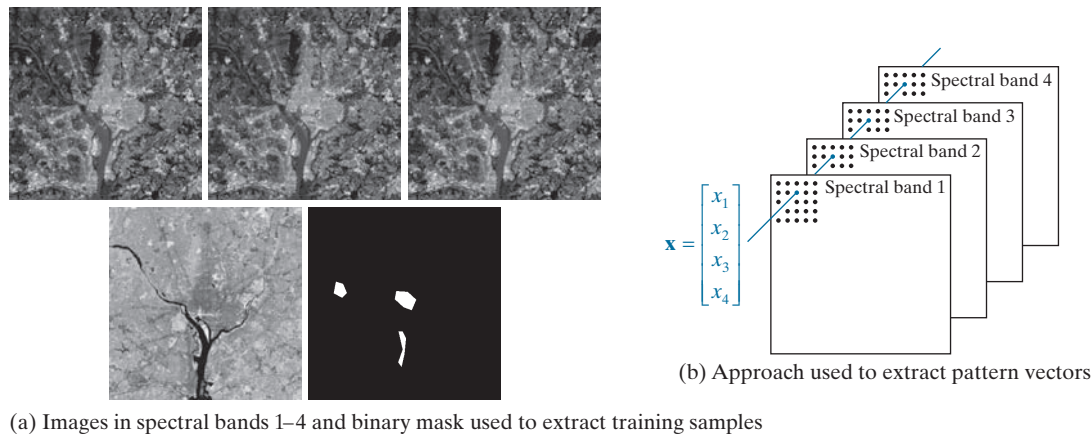


FIGURE 12.37 (a) Starting with the leftmost image: blue, green, red, near infrared, and binary mask images. In the mask, the lower region is for water, the center region is for the urban area, and the left mask corresponds to vegetation. All images are of size 512×512 pixels. (b) Approach used for generating 4-D pattern vectors from a stack of the four multispectral images. (Multispectral images courtesy of NASA.)

and *vegetation*. Figure 12.37 shows the four multispectral images used in the experiment, the masks used to extract the training and test samples, and the approach used to generate the 4-D pattern vectors.

As in Example 12.6, we extracted a total of 1900 training pattern vectors and 1887 test pattern vectors (see Table 12.1 for a listing of vectors by class). After preliminary runs with the training data to establish that the mean squared error was decreasing as a function of epoch, we determined that a neural net with one hidden layer of two nodes achieved stable learning with $\alpha = 0.001$ and 1,000 training epochs. Keeping those two parameters fixed, we varied the number of nodes in the internal layer, as listed in Table 12.4. The objective of these preliminary runs was to determine the smallest neural net that would give the best recognition rate. As you can see from the results in the table, [4 3 3] is clearly the architecture of choice in this case. Figure 12.38 shows this neural net, along with the parameters learned during training.

After the basic architecture was defined, we kept the learning rate constant at $\alpha = 0.001$ and varied the number of epochs to determine the best recognition rate with the architecture in Fig. 12.38. Table 12.5 shows the results. As you can see, the recognition rate improved slowly as a function of epoch, reaching a plateau at around 50,000 epochs. In fact, as Fig. 12.39 shows, the MSE decreased quickly up to about 800 training epochs and decreased slowly after that, explaining why the correct recognition rate changed so little after about 2,000 epochs. Similar results were obtained with $\alpha = 0.01$, but decreasing

TABLE 12.4 Recognition rate as a function of neural net architecture for $\alpha = 0.001$ and 1,000 training epochs. The network architecture is defined by the numbers in brackets. The first and last number inside each bracket refer to the number of input and output nodes, respectively. The inner entries give the number of nodes in each hidden layer.

Network Architecture	[4 2 3]	[4 3 3]	[4 4 3]	[4 5 3]	[4 2 2 3]	[4 4 3 3]	[4 4 4 3]	[4 10 3 3]	[4 10 10 3]
Recognition Rate	95.8%	96.2%	95.9%	96.1%	74.6%	90.8%	87.1%	84.9%	89.7%

FIGURE 12.38
Neural net architecture used to classify the multispectral image data in Fig. 12.37 into three classes: water, urban, and vegetation. The parameters shown were obtained in 50,000 epochs of training using $\alpha = 0.001$.

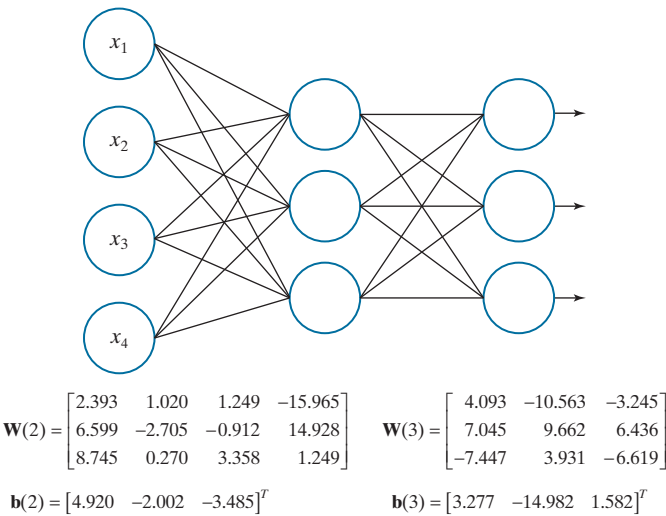
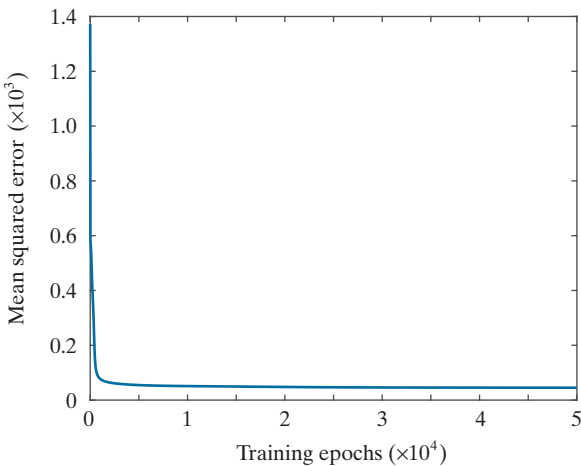


TABLE 12.5
Recognition performance on the training set as a function of training epochs. The learning rate constant was $\alpha = 0.001$ in all cases.

Training Epochs	1,000	10,000	20,000	30,000	40,000	50,000	60,000	70,000	80,000
Recognition Rate	95.3%	96.6%	96.7%	96.8%	96.9%	97.0%	97.0%	97.0%	97.0%

FIGURE 12.39
MSE for the network architecture in Fig. 12.38 as a function of the number of training epochs. The learning rate parameter was $\alpha = 0.001$ in all cases.



this parameter to $\alpha = 0.1$ resulted in a drop of the best correct recognition rate to 49.1%. Based on the preceding results, we used $\alpha = 0.001$ and 50,000 epochs to train the network.

The parameters in Fig. 12.38 were the result of training. The recognition rate for the training data using these parameters was 97%. We achieved a recognition rate of 95.6% on the test set using the same parameters. The difference between these two figures, and the 96.4% and 96.2%, respectively, obtained for the same data with the Bayes classifier (see Example 12.6), are statistically insignificant.

The fact that our neural networks achieved results comparable to those obtained with the Bayes classifier is not surprising. It can be shown (Duda, Hart, and Stork [2001]) that a three-layer neural net, trained by backpropagation using a sum of errors squared criterion, approximates the Bayes decision functions in the limit, as the number of training samples approaches infinity. Although our training sets were small, the data were well behaved enough to yield results that are close to what theory predicts.

12.6 DEEP CONVOLUTIONAL NEURAL NETWORKS

Up to this point, we have organized pattern features as vectors. Generally, this assumes that the form of those features has been specified (i.e., “engineered” by a human designer) and extracted from images prior to being input to a neural network (Example 12.13 is an illustration of this approach). But one of the strengths of neural networks is that they are capable of learning pattern features directly from training data. What we would like to do is input a set of training images directly into a neural network, and have the network learn the necessary features *on its own*. One way to do this would be to convert images to vectors directly by organizing the pixels based on a linear index (see Fig. 12.1), and then letting each element (pixel) of the linear index be an element of the vector. However, this approach does not utilize any spatial relationships that may exist between pixels in an image, such as pixel arrangements into corners, the presence of edge segments, and other features that may help to differentiate one image from another. In this section, we present a class of neural networks called *deep convolutional neural networks* (CNNs or *ConvNets* for short) that accept images as inputs and are ideally suited for automatic learning and image classification. In order to differentiate between CNNs and the neural nets we studied in Section 12.5, we will refer to the latter as “fully connected” neural networks.

A BASIC CNN ARCHITECTURE

In the following discussion, we use a *LeNet* architecture (see references at the end of this chapter) to introduce convolutional nets. We do this for two main reasons: First, the LeNet architecture is reasonably simple to understand. This makes it ideal for introducing basic CNN concepts. Second, our real interest is in deriving the equations of backpropagation for convolutional networks, a task that is simplified by the intuitiveness of LeNets.

The CNN in Fig. 12.40 contains all the basic elements of a LeNet architecture, and we use it without loss of generality. A key difference between this architecture and the neural net architectures we studied in the previous section is that inputs to CNNs are 2-D arrays (images), while inputs to our fully connected neural networks are vectors. However, as you will see shortly, the computations performed by both networks are very similar: (1) a sum of products is formed, (2) a bias value is added,

To simplify the explanation of the CNN in Fig. 12.40, we focus attention initially on a single image input. Multiple input images are a trivial extension we will consider later in our discussion.

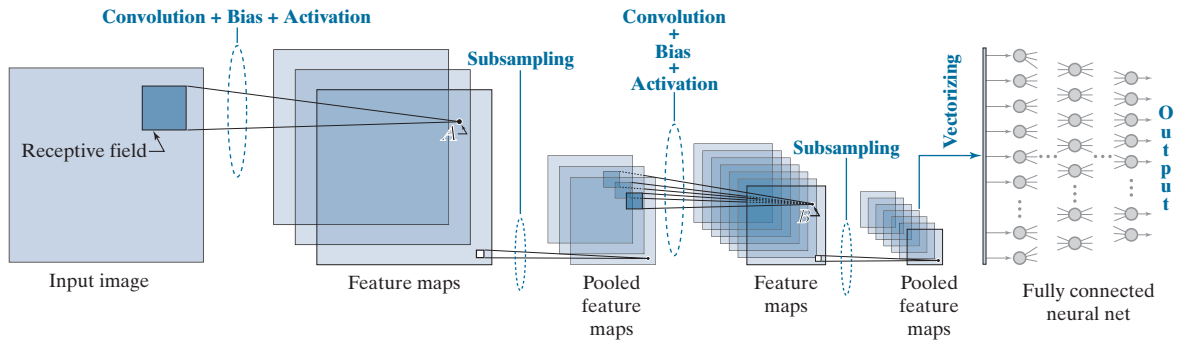


FIGURE 12.40 A CNN containing all the basic elements of a LeNet architecture. Points *A* and *B* are specific values to be addressed later in this section. The last pooled feature maps are vectorized and serve as the input to a fully connected neural network. The class to which the input image belongs is determined by the output neuron with the highest value.

(3) the result is passed through an activation function, and (4) the activation value becomes a single input to a following layer.

Despite the fact that the computations performed by CNNs and fully connected neural nets are similar, there are some basic differences between the two, beyond their input formats being 2-D versus vectors. An important difference is that CNNs are capable of learning 2-D features directly from raw image data, as mentioned earlier. Because the tools for systematically engineering comprehensive feature sets for complex image recognition tasks do not exist, having a system that can learn its own image features from raw image data is a crucial advantage of CNNs. Another major difference is in the way in which layers are connected. In a fully connected neural net, we feed the output of *every* neuron in a layer directly into the input of *every* neuron in the next layer. By contrast, in a CNN we feed into every input of a layer, a *single* value, determined by the convolution (hence the name *convolutional* neural net) over a spatial neighborhood in the output of the previous layer. Therefore, CNNs are not fully connected in the sense defined in the last section. Another difference is that the 2-D arrays from one layer to the next are subsampled to reduce sensitivity to translational variations in the input. These differences and their meaning will become clear as we look at various CNN configurations in the following discussion.

Basics of How a CNN Operates

As noted above, the type of neighborhood processing in CNNs is spatial convolution. We explained the mechanics of spatial convolution in Fig. 3.29, and expressed it mathematically in Eq. (3-35). As that equation shows, convolution computes a sum of products between pixels and a set of kernel weights. This operation is carried out at every spatial location in the input image. The result at each location (x, y) in the input is a scalar value. Think of this value as the output of a neuron in a layer of a fully connected neural net. If we add a bias and pass the result through an activation function (see Fig. 12.29), we have a complete analogy between the

We will discuss in the next subsection the exact form of neural computations in a CNN, and show they are equivalent in form to the computations performed by neurons in a fully connected neural net.

basic computations performed by a CNN and those performed by the neural nets discussed in the previous section.

These remarks are summarized in Fig. 12.40, the leftmost part of which shows a neighborhood at one location in the input image. In CNN terminology, these neighborhoods are called *receptive fields*. All a receptive field does is select a region of pixels in the input image. As the figure shows, the first operation performed by a CNN is convolution, whose values are generated by moving the receptive field over the image and, at each location, forming a sum of products of a set of weights and the pixels contained in the receptive field. The set of weights, arranged in the shape of the receptive field, is a *kernel*, as in Chapter 3. The number of spatial increments by which a receptive field is moved is called the *stride*. Our spatial convolutions in previous chapters had a stride of one, but that is not a requirement of the equations themselves. In CNNs, an important motivation for using strides greater than one is data reduction. For example, changing the stride from one to two reduces the image resolution by one-half in each spatial dimension, resulting in a three-fourths reduction in the amount of data per image. Another important motivation is as a substitute for subsampling which, as we discuss below, is used to reduce system sensitivity to spatial translation.

To each convolution value (sum of products) we add a bias, then pass the result through an activation function to generate a single value. Then, this value is fed to the corresponding (x, y) location in the input of the next layer. When repeated for all locations in the input image, the process just explained results in a 2-D set of values that we store in next layer as a 2-D array, called a *feature map*. This terminology is motivated by the fact that the role performed by convolution is to extract features such as edges, points, and blobs from the input (remember, convolution is the basis of spatial filtering, which we used in Chapter 3 for tasks such as smoothing, sharpening, and computing edges in an image). The same weights and a single bias are used to generate the convolution (feature map) values corresponding to all locations of the receptive field in the input image. This is done to cause the same feature to be detected at all points in the image. Using the same weights and bias for this purpose is called *weight* (or *parameter*) *sharing*.

Figure 12.40 shows three feature maps in the first layer of the network. The other two feature maps are generated in the manner just explained, but using a *different* set of weights and bias for each feature map. Because each set of weights and bias is different, each feature map generally will contain a different set of features, all extracted from the same input image. The feature maps are referred to collectively as a *convolutional layer*. Thus, the CNN in Fig. 12.40 has two convolutional layers.

The process after convolution and activation is *subsampling* (also called *pooling*), which is motivated by a model of the mammal visual cortex proposed by Hubel and Wiesel [1959]. Their findings suggest that parts of the visual cortex consist of *simple* and *complex* cells. The simple cells perform feature extraction, while the complex cells combine (aggregate) those features into a more meaningful whole. In this model, a reduction in spatial resolution appears to be responsible for achieving translational invariance. Pooling is a way of modeling this reduction in dimensionality. When training a CNN with large image databases, pooling has the additional

In the terminology of Chapter 3, a feature map is a spatially filtered image.

advantage of reducing the volume of data being processed. You can think of the results of subsampling as producing *pooled feature maps*. In other words, a pooled feature map is a feature map of reduced spatial resolution. Pooling is done by subdividing a feature map into a set of small (typically 2×2) regions, called *pooling neighborhoods*, and replacing *all* elements in such a neighborhood by a *single* value. We assume that pooling neighborhoods are *adjacent* (i.e., they do not overlap). There are several ways to compute the pooled values; collectively, the different approaches are called *pooling methods*. Three common pooling methods are: (1) *average pooling*, in which the values in each neighborhood are replaced by the average of the values in the neighborhood; (2) *max-pooling*, which replaces the values in a neighborhood by the maximum value of its elements; and (3) L_2 pooling, in which the resulting pooled value is the square root of the sum of the neighborhood values squared. There is one pooled feature map for each feature map. The pooled feature maps are referred to collectively as a *pooling layer*. In Fig. 12.40 we used 2×2 pooling so each resulting pooled map is one-fourth the size of the preceding feature map. The use of receptive fields, convolution, parameter sharing, and pooling are characteristics unique to CNNs.

Because feature maps are the result of spatial convolution, we know from Chapter 3 that they are simply filtered images. It then follows that pooled feature maps are filtered images of lower resolution. As Fig. 12.40 illustrates, the pooled feature maps in the first layer become the inputs to the next layer in the network. But, whereas we showed a single image as an input to the first layer, we now have multiple pooled feature maps (filtered images) that are inputs into the second layer.

To see how these multiple inputs to the second layer are handled, focus for a moment on one pooled feature map. To generate the values for the first feature map in the second convolutional layer, we perform convolution, add a bias, and use activation, as before. Then, we change the kernel and bias, and repeat the procedure for the second feature map, still using the same input. We do this for every remaining feature map, changing the kernel weights and bias for each. Then, we consider the next pooled feature map input and perform the same procedure (convolution, plus bias, plus activation) for every feature map in the second layer, using yet another set of different kernels and biases. When we are finished, we will have generated three values for the same location in every feature map, with one value coming from the corresponding location in each of the three inputs. The question now is: How do we combine these three individual values into one? The answer lies in the fact that convolution is a linear process, from which it follows that the three individual values are combined into one by superposition (that is, by adding them).

In the first layer, we had one input image and three feature maps, so we needed three kernels to complete all required convolutions. In the second layer, we have three inputs and seven feature maps, so the total number of kernels (and biases) needed is $3 \times 7 = 21$. Each feature map is pooled to generate a corresponding pooled feature map, resulting in seven pooled feature maps. In Fig. 12.40, there are only two layers, so these seven pooled feature maps are the outputs of the last layer.

As usual, the ultimate objective is to use features for classification, so we need a classifier. As Fig. 12.40 shows, in a CNN we perform classification by feeding the

Adjacency is not a requirement of pooling per se. We assume it here for simplicity and because this is an approach that is used frequently.

You could interpret the convolution with several input images as 3-D convolution, but with movement only in the spatial (x and y) directions. The result would be identical to summing individual convolutions with each image separately, as we do here.

The parameters of the fully connected neural net are learned during training of the CNN, to be discussed shortly.

value of the last pooled layer into a fully connected neural net, the details of which you learned in Section 12.5. But the outputs of a CNN are 2-D arrays (i.e., filtered images of reduced resolution), whereas the inputs to a fully connected net are vectors. Therefore, we have to *vectorize* the 2-D pooled feature maps in the last layer. We do this using linear indexing (see Fig. 12.1). Each 2-D array in the last layer of the CNN is converted into a vector, then all resulting vectors are concatenated (vertically for a column) to form a single vector. This vector propagates through the neural net, as explained in Section 12.5. In any given application, the number of outputs in the fully connected net is equal to the number of pattern classes being classified. As before, the output with the highest value determines the class of the input.

EXAMPLE 12.14: Receptive fields, pooling neighborhoods, and their corresponding feature maps.

The top row of Fig. 12.41 shows a numerical example of the relative sizes of feature maps and pooled feature maps as a function of the sizes of receptive fields and pooling neighborhoods. The input image is of size 28×28 pixels, and the receptive field is of size 5×5 . If we require that the receptive field be contained in the image during convolution, you know from Section 3.4 that the resulting convolution array (feature map) will be of size 24×24 . If we use a pooling neighborhood of size 2×2 , the resulting pooled feature maps will be of size 12×12 , as the figure shows. As noted earlier, we assume that pooling neighborhoods do not overlap.

As an analogy with fully connected neural nets, think of each element of a 2-D array in the top row of Fig. 12.41 as a neuron. The outputs of the neurons in the input are pixel values. The neurons in the feature map of the first layer have output values generated by convolving with the input image a kernel whose size and shape are the same as the receptive field, and whose coefficients are learned during training. To each convolution value we add a bias and pass the result through an activation function to generate the output value of the corresponding neuron in the feature map. The output values of the neurons in the pooled feature maps are generated by pooling the output values of the neurons in the feature maps.

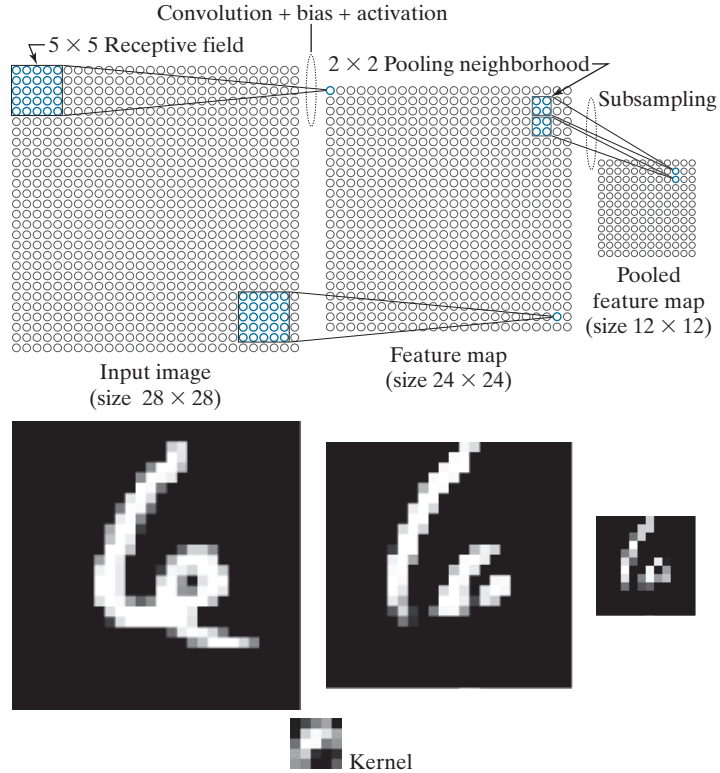
The second row in Fig. 12.41 illustrates visually how feature maps and pooled feature maps look based on the input image shown in the figure. The kernel shown is as described in the previous paragraph, and its weights (shown as intensity values) were learned from sample images using the training of the CNN described later in Example 12.17. Therefore, the nature of the learned features is determined by the learned kernel coefficients. Note that the contents of the feature maps are specific features detected by convolution. For example, some of the features emphasize edges in the the character. As mentioned earlier, the pooled features are lower-resolution versions of this effect.

EXAMPLE 12.15: Graphical illustration of the functions performed by the components of a CNN.

Figure 12.42 shows the 28×28 image from Fig. 12.41, input into an expanded version of the CNN architecture from Fig. 12.40. The expanded CNN, which we will discuss in more detail in Example 12.17, has six feature maps in the first layer, and twelve in the second. It uses receptive fields of size 5×5 , and pooling neighborhoods of size 2×2 . Because the receptive fields are of size 5×5 , the feature maps in the first layer are of size 24×24 , as we explained in Example 12.14. Each feature map has its own set of weights and bias, so we will need a total of $(5 \times 5) \times 6 + 6 = 156$ parameters (six kernels with twenty-five weights each, and six biases) to generate the feature maps in the first layer. The top row of Fig. 12.43(a) shows the kernels with the weights learned during training of the CNN displayed as images, with intensity being proportional to kernel values.

FIGURE 12.41

Top row: How the sizes of receptive fields and pooling neighborhoods affect the sizes of feature maps and pooled feature maps.
Bottom row: An image example. This figure is explained in more detail in Example 12.17. (Image courtesy of NIST.)



Because we used pooling neighborhoods of size 2×2 , the pooled feature maps in the first layer of Fig. 12.42 are of size 12×12 . As we discussed earlier, the number of feature maps and pooled feature maps is the same, so we will have six arrays of size 12×12 acting as inputs to the twelve feature maps in the second layer (the number of feature maps generally is different from layer to layer). Each feature map will have its own set of weights and bias, so will need a total of $6 \times (5 \times 5) \times 12 + 12 = 1812$

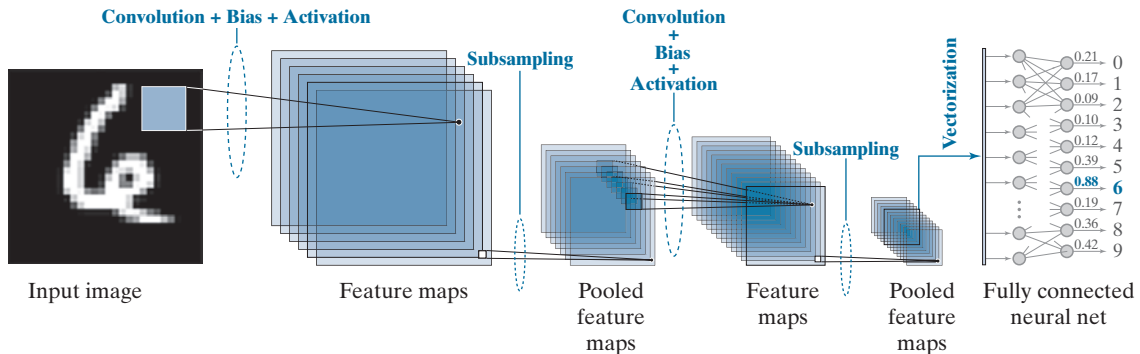


FIGURE 12.42 Numerical example illustrating the various functions of a CNN, including recognition of an input image. A sigmoid activation function was used throughout.

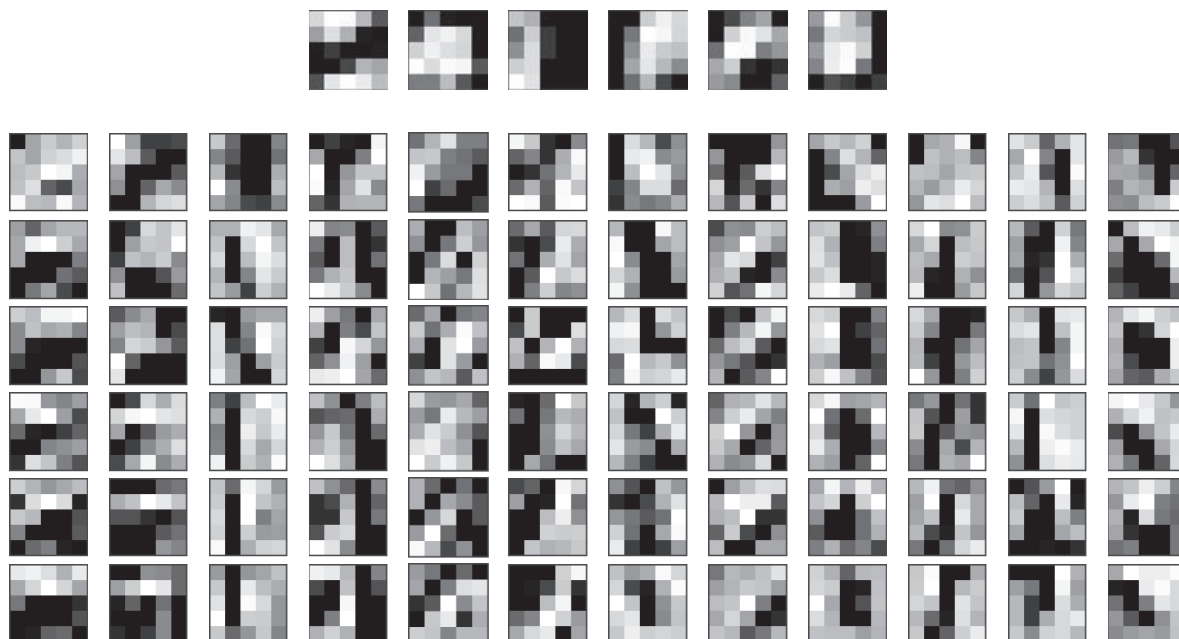


FIGURE 12.43 Top: The weights (shown as images of size 5×5) corresponding to the six feature maps in the first layer of the CNN in Fig. 12.42. Bottom: The weights corresponding to the twelve feature maps in the second layer.

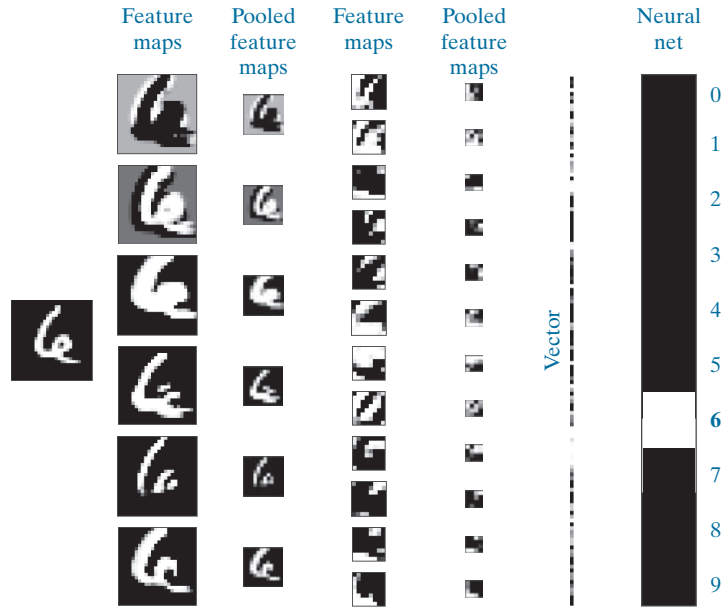
parameters to generate the feature maps in the second layer (i.e., twelve sets of six kernels with twenty-five weights each, plus twelve biases). The bottom part of Fig. 12.43 shows the kernels as images. Because we are using receptive fields of size 5×5 , the feature maps in the second layer are of size 8×8 . Using 2×2 pooling neighborhoods resulted in pooled feature maps of size 4×4 in the second layer.

As we discussed earlier, the pooled feature maps in the last layer have to be vectorized to be able to input them into the fully connected neural net. Each pooled feature map resulted in a column vector of size 16×1 . There are 12 of these vectors which, when concatenated vertically, resulted in a single vector of size 192×1 . Therefore, our fully connected neural net has 192 input neurons. There are ten numeral classes, so there are 10 output neurons. As you will see later, we obtained excellent performance by using a neural net with no hidden layers, so our complete neural net had a total of 192 input neurons and 10 output neurons. For the input character shown in Fig. 12.42, the highest value in the output of the fully connected neural net was in the seventh neuron, which corresponds to the class of 6's. Therefore, the input was recognized properly. This is shown in bold text in the figure.

Figure 12.44 shows graphically what the feature maps look like as the input image propagates through the CNN. Consider the feature maps in the first layer. If you look at each map carefully, you will notice that it highlights a different characteristic of the input. For example, the map on the top of the first column highlights the two principal edges on the top of the character. The second map highlights the edges of the entire inner region, and the third highlights a “blob-like” nature of the digit, almost as if it had been blurred by a lowpass kernel. The other three images show other features. Although the pooled feature maps are lower-resolution versions of the original feature maps, they still retained the key characteristics of the features in the latter. If you look at the first two feature maps in the second layer, and compare them with the first two in the first layer, you can see that they could be interpreted as higher-

FIGURE 12.44

Visual summary of an input image propagating through the CNN in Fig. 12.42. Shown as images are all the results of convolution (feature maps) and pooling (pooled feature maps) for both layers of the network. (Example 12.17 contains more details about this figure.)



level abstractions of the top part of the character, in the sense that they show an area flanked on both sides by areas of opposite intensity. These abstractions are not always easy to analyze visually, but as you will see in later examples, they can be very effective. The vectorized version of the last pooled layer is self-explanatory. The output of the fully connected neural net shows dark for low values and white for the highest value, indicating that the input was properly recognized as a number 6. Later in this section, we will show that the simple CNN architecture in Fig. 12.42 is capable of recognizing the correct class of over 70,000 numerical samples with nearly perfect accuracy.

Neural Computations in a CNN

Recall from Fig. 12.29 that the basic computation performed by an artificial neuron is a sum of products between weights and values from a previous layer. To this we add a bias and call the result the *net (total) input* to the neuron, which we denoted by z_i . As we showed in Eq. (12-54), the sum involved in generating z_i is a single sum. The computations performed in a CNN to generate a single value in a feature map is 2-D convolution. As you learned in Chapter 3, this is a double sum of products between the coefficients of a kernel and the corresponding elements of the image array overlapped by the kernel. With reference to Fig. 12.40, let w denote a kernel formed by arranging the weights in the shape of the receptive field we discussed in connection with that figure. For notational consistency with Section 12.5, let $a_{x,y}$ denote image or pooled feature values, depending on the layer. The convolution value at any point (x, y) in the input is given by

$$w \star a_{x,y} = \sum_l \sum_k w_{l,k} a_{x-l,y-k} \quad (12-83)$$

where l and k span the dimensions of the kernel. Suppose that w is of size 3×3 . Then, we can then expand this equation into the following sum of products:

$$\begin{aligned} w \star a_{x,y} &= w \star a_{x,y} = \sum_l \sum_k w_{l,k} a_{x-l,y-k} \\ &= w_{1,1} a_{x-1,y-1} + w_{1,2} a_{x-1,y-2} + \cdots + w_{3,3} a_{x-3,y-3} \end{aligned} \quad (12-84)$$

We could relabel the subscripts on w and a , and write instead

$$\begin{aligned} w \star a_{x,y} &= w_1 a_1 + w_2 a_2 + \cdots + w_9 a_9 \\ &= \sum_{i=1}^9 w_i a_i \end{aligned} \quad (12-85)$$

The results of Eqs. (12-84) and (12-85) are identical. If we add a bias to the latter equation and call the result z we have

$$\begin{aligned} z &= \sum_{j=1}^9 w_j a_j + b \\ &= w \star a_{x,y} + b \end{aligned} \quad (12-86)$$

The form of the first line of this equation is identical to Eq. (12-54). Therefore, we conclude that if we add a bias to the spatial convolution computation performed by a CNN at any fixed position (x, y) in the input, the result can be expressed in a form identical to the computation performed by an artificial neuron in a fully connected neural net. We need the x, y only to account for the fact that we are working in 2-D. If we think of z as the net input to a neuron, the analogy with the neurons discussed in Section 12.5 is completed by passing z through an activation function, h , to get the output of the neuron:

$$a = h(z) \quad (12-87)$$

This is exactly how the value of any point in a feature map (such as the point labeled A in Fig. 12.40) is computed.

Now consider point B in that figure. As mentioned earlier, its value is given by adding three convolution equations:

$$\begin{aligned} w_{l,k}^{(1)} \star a_{x,y}^{(1)} + w_{l,k}^{(2)} \star a_{x,y}^{(2)} + w_{l,k}^{(3)} \star a_{x,y}^{(3)} &= \sum_l \sum_k w_{l,k}^{(1)} a_{x-l,y-k}^{(1)} + \\ &\quad \sum_l \sum_k w_{l,k}^{(2)} a_{x-l,y-k}^{(2)} + \sum_l \sum_k w_{l,k}^{(3)} a_{x-l,y-k}^{(3)} \end{aligned} \quad (12-88)$$

where the superscripts refer to the three pooled feature maps in Fig. 12.40. The values of l, k, x , and y are the same in all three equations because all three kernels are of the same size and they move in unison. We could expand this equation and obtain a sum of products that is lengthier than for point A in Fig. 12.40, but we could still relabel all terms and obtain a sum of products that involves only one summation, *exactly* as before.

The preceding result tells us that the equations used to obtain the value of an element of any feature map in a CNN can be expressed in the form of the computation performed by an artificial neuron. This holds for any feature map, regardless of how many convolutions are involved in the computation of the elements of that feature map, in which case we would simply be dealing with the sum of more convolution equations. The implication is that we can use the basic form of Eqs. (12-86) and (12-87) to describe how the value of an element in any feature map of a CNN is obtained. This means we do not have to account explicitly for the number of different pooled feature maps (and hence the number of different kernels) used in a pooling layer. The result is a significant simplification of the equations that describe forward and backpropagation in a CNN.

Multiple Input Images

The values of $a_{x,y}$ just discussed are pixel values in the first layer but, in layers past the first, $a_{x,y}$ denotes values of pooled features. However, our equations do not differentiate based on what these variables actually represent. For example, suppose we replace the input to Fig. 12.40 with three images, such as the three components of an RGB image. The equations for the value of point *A* in the figure would now have the same form as those we stated for point *B*—only the weights and biases would be different. Thus, the results in the previous discussion for one input image are applicable directly to multiple input images. We will give an example of a CNN with three input images later in our discussion.

THE EQUATIONS OF A FORWARD PASS THROUGH A CNN

We concluded in the preceding discussion that we can express the result of convolving a kernel, w , and an input array with values $a_{x,y}$, as

$$\begin{aligned} z_{x,y} &= \sum_l \sum_k w_{l,k} a_{x-l,y-k} + b \\ &= w \star a_{x,y} + b \end{aligned} \quad (12-89)$$

where l and k span the dimensions of the kernel, x and y span the dimensions of the input, and b is a bias. The corresponding value of $a_{x,y}$ is

$$a_{x,y} = h(z_{x,y}) \quad (12-90)$$

But this $a_{x,y}$ is different from the one we used to compute Eq. (12-89), in which $a_{x,y}$ represents values from the previous layer. Thus, we are going to need additional notation to differentiate between layers. As in fully connected neural nets, we use ℓ for this purpose, and write Eqs. (12-89) and (12-90) as

$$\begin{aligned} z_{x,y}(\ell) &= \sum_l \sum_k w_{l,k}(\ell) a_{x-l,y-k}(\ell-1) + b(\ell) \\ &= w(\ell) \star a_{x,y}(\ell-1) + b(\ell) \end{aligned} \quad (12-91)$$

As noted earlier, a kernel is formed by organizing the weights in the shape of a corresponding receptive field. Also keep in mind that w and $a_{x,y}$ represent all the weights and corresponding values in a set of input images or pooled features.

and

$$a_{x,y}(\ell) = h(z_{x,y}(\ell)) \quad (12-92)$$

for $\ell = 1, 2, \dots, L_c$, where L_c is the number of convolutional layers, and $a_{x,y}(\ell)$ denotes the values of pooled features in convolutional layer ℓ . When $\ell = 1$,

$$a_{x,y}(0) = \{\text{values of pixels in the input image(s)}\} \quad (12-93)$$

When $\ell = L_c$,

$$a_{x,y}(L_c) = \{\text{values of pooled features in last layer of the CNN}\} \quad (12-94)$$

Note that ℓ starts at 1 instead of 2, as we did in Section 12.5. The reason is that we are naming layers, as in “convolutional layer ℓ .” It would be confusing to start at convolutional layer 2. Finally, we note that the pooling does not require any convolutions. The only function of pooling is to reduce the spatial dimensions of the feature map preceding it, so we do not include explicit pooling equations here.

Equations (12-91) through (12-94) are all we need to compute all values in a forward pass through the convolutional section of a CNN. As described in Fig. 12.40, the values of the pooled features of the last layer are vectorized and fed into a fully connected feedforward neural network, whose forward propagation is explained in Eqs. (12-54) and (12-55) or, in matrix form, in Table 12.2.

THE EQUATIONS OF BACKPROPAGATION USED TO TRAIN CNNs

As you saw in the previous section, the feedforward equations of a CNN are similar to those of a fully connected neural net, but with multiplication replaced by convolution, and notation that reflects the fact that CNNs are not fully connected in the sense defined in Section 12.5. As you will see in this section, the equations of backpropagation also are similar in many respects to those in fully connected neural nets.

As in the derivation of backpropagation in Section 12.5, we start with the definition of how the output error of our CNN changes with respect to each neuron in the network. The form of the error is the same as for fully connected neural nets, but now it is a function of x and y instead of j :

$$\delta_{x,y}(\ell) = \frac{\partial E}{\partial z_{x,y}(\ell)} \quad (12-95)$$

As in Section 12.5, we want to relate this quantity to $\delta_{xy}(\ell + 1)$, which we again do using the chain rule:

$$\delta_{x,y}(\ell) = \frac{\partial E}{\partial z_{x,y}(\ell)} = \sum_u \sum_v \frac{\partial E}{\partial z_{u,v}(\ell + 1)} \frac{\partial z_{u,v}(\ell + 1)}{\partial z_{x,y}(\ell)} \quad (12-96)$$

where u and v are any two variables of summation over the range of possible values of z . As noted in Section 12.5, these summations result from applying the chain rule.

By definition, the first term of the double summation of Eq. (12-96) is $\delta_{x,y}(\ell + 1)$. So, we can write this equation as

$$\delta_{x,y}(\ell) = \frac{\partial E}{\partial z_{x,y}(\ell)} = \sum_u \sum_v \delta_{u,v}(\ell + 1) \frac{\partial z_{u,v}(\ell + 1)}{\partial z_{x,y}(\ell)} \quad (12-97)$$

Substituting Eq. (12-92) into Eq. (12-91), and using the resulting $z_{u,v}$ in Eq. (12-97), we obtain

$$\delta_{x,y}(\ell) = \sum_u \sum_v \delta_{u,v}(\ell + 1) \frac{\partial}{\partial z_{x,y}(\ell)} \left[\sum_l \sum_k w_{l,k}(\ell + 1) h(z_{u-l,v-k}(\ell)) + b(\ell + 1) \right] \quad (12-98)$$

The derivative of the expression inside the brackets is zero unless $u - l = x$ and $v - k = y$, and because the derivative of $b(\ell + 1)$ with respect to $z_{x,y}(\ell)$ is zero. But, if $u - l = x$ and $v - k = y$, then $l = u - x$ and $k = v - y$. Therefore, taking the indicated derivative of the expression in brackets, we can write Eq. (12-98) as

$$\delta_{x,y}(\ell) = \sum_u \sum_v \delta_{u,v}(\ell + 1) \left[\sum_{u-x} \sum_{v-y} w_{u-x,v-y}(\ell + 1) h'(z_{x,y}(\ell)) \right] \quad (12-99)$$

Values of x , y , u , and v are specified outside of the terms inside the brackets. Once the values of these variables are fixed, $u - x$ and $v - y$ inside the brackets are simply two *constants*. Therefore, the double summation evaluates to $w_{u-x,v-y}(\ell + 1) h'(z_{x,y}(\ell))$, and we can write Eq. (12-99) as

$$\begin{aligned} \delta_{x,y}(\ell) &= \sum_u \sum_v \delta_{u,v}(\ell + 1) w_{u-x,v-y}(\ell + 1) h'(z_{x,y}(\ell)) \\ &= h'(z_{x,y}(\ell)) \sum_u \sum_v \delta_{u,v}(\ell + 1) w_{u-x,v-y}(\ell + 1) \end{aligned} \quad (12-100)$$

The double sum expression in the second line of this equation is in the form of a convolution, but the displacements are the negatives of those in Eq. (12-91). Therefore, we can write Eq. (12-100) as

$$\delta_{x,y}(\ell) = h'(z_{x,y}(\ell)) \left[\delta_{x,y}(\ell + 1) \star w_{-x,-y}(\ell + 1) \right] \quad (12-101)$$

The negatives in the subscripts indicate that w is *reflected* about both spatial axes. This is the same as rotating w by 180° , as we explained in connection with Eq. (3-35). Using this fact, we finally arrive at an expression for the error at a layer ℓ by writing Eq. (12-101) equivalently as

$$\delta_{x,y}(\ell) = h'(z_{x,y}(\ell)) \left[\delta_{x,y}(\ell + 1) \star \text{rot}180(w_{x,y}(\ell + 1)) \right] \quad (12-102)$$

The 180° rotation is for each 2-D kernel in a layer.

But the kernels do not depend on x and y , so we can write this equation as

$$\delta_{x,y}(\ell) = h'(z_{x,y}(\ell)) \left[\delta_{x,y}(\ell+1) \star \text{rot180}(w(\ell+1)) \right] \quad (12-103)$$

As in Section 12.5, our final objective is to compute the change in E with respect to the weights and biases. Following a similar procedure as above, we obtain

$$\begin{aligned} \frac{\partial E}{\partial w_{l,k}} &= \sum_x \sum_y \frac{\partial E}{\partial z_{x,y}(\ell)} \frac{\partial z_{x,y}(\ell)}{\partial w_{l,k}} \\ &= \sum_x \sum_y \delta_{x,y}(\ell) \frac{\partial z_{x,y}(\ell)}{\partial w_{l,k}} \\ &= \sum_x \sum_y \delta_{x,y}(\ell) \frac{\partial}{\partial w_{l,k}} \left[\sum_l \sum_k w_{l,k}(\ell) h(z_{x-l,y-k}(\ell-1)) + b(\ell) \right] \quad (12-104) \\ &= \sum_x \sum_y \delta_{x,y}(\ell) h(z_{x-l,y-k}(\ell-1)) \\ &= \sum_x \sum_y \delta_{x,y}(\ell) a_{x-l,y-k}(\ell-1) \end{aligned}$$

where the last line follows from Eq. (12-92). This line is in the form of a convolution but, comparing it to Eq. (12-91), we see there is a sign reversal between the summation variables and their corresponding subscripts. To put it in the form of a convolution, we write the last line of Eq. (12-104) as

$$\begin{aligned} \frac{\partial E}{\partial w_{l,k}} &= \sum_x \sum_y \delta_{x,y}(\ell) a_{-(l-x),-(k-y)}(\ell-1) \\ &= \delta_{l,k}(\ell) \star a_{-l,-k}(\ell-1) \quad (12-105) \\ &= \delta_{l,k}(\ell) \star \text{rot180}(a(\ell-1)) \end{aligned}$$

Similarly (see Problem 12.32),

$$\frac{\partial E}{\partial b(\ell)} = \sum_x \sum_y \delta_{x,y}(\ell) \quad (12-106)$$

Using the preceding two expressions in the gradient descent equations (see Section 12.5), it follows that

$$\begin{aligned} w_{l,k}(\ell) &= w_{l,k}(\ell) - \alpha \frac{\partial E}{\partial w_{l,k}} \\ &= w_{l,k}(\ell) - \alpha \delta_{l,k}(\ell) \star \text{rot180}(a(\ell-1)) \quad (12-107) \end{aligned}$$

and

$$\begin{aligned}
 b(\ell) &= b(\ell) - \alpha \frac{\partial E}{\partial b(\ell)} \\
 &= b(\ell) - \alpha \sum_x \sum_y \delta_{x,y}(\ell)
 \end{aligned} \tag{12-108}$$

Equations (12-107) and (12-108) update the weights and bias of each convolution layer in a CNN. As we have mentioned before, it is understood that the $w_{l,k}$ represents *all* the weights of a layer. The variables l and k span the spatial dimensions of the 2-D kernels, all of which are of the same size.

In a forward pass, we went from a convolution layer to a pooled layer. In backpropagation, we are going in the opposite direction. But the pooled feature maps are smaller than their corresponding feature maps (see Fig. 12.40). Therefore, when going in the reverse direction, we *upsample* (e.g., by pixel replication) each pooled feature map to match the size of the feature map that generated it. Each pooled feature map corresponds to a unique feature map, so the path of backpropagation is clearly defined.

With reference to Fig. 12.40, backpropagation starts at the output of the fully connected neural net. We know from Section 12.5 how to update the weights of this network. When we get to the “interface” between the neural net and the CNN, we have to reverse the vectorization method used to generate input vectors. That is, before we can proceed with backpropagation using Eqs. (12-107) and (12-108), we have to *regenerate* the individual pooled feature maps from the single vector propagated back by the fully connected neural net.

We summarized in Table 12.3 the backpropagation steps for a fully connected neural net. Table 12.6 summarizes the steps for performing backpropagation in the CNN architecture in Fig. 12.40. The procedure is repeated for a specified number of

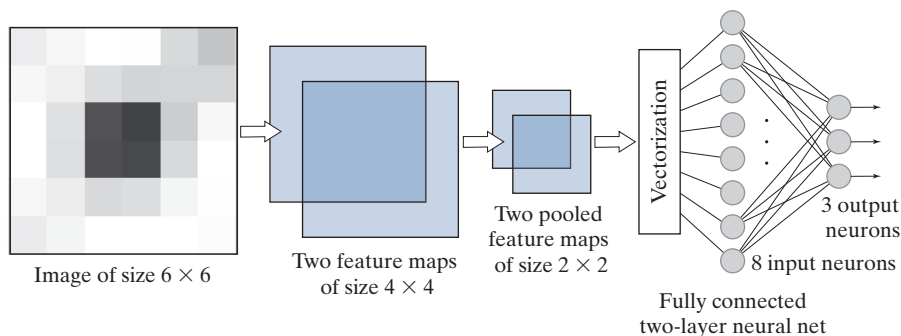
TABLE 12.6

The principal steps used to train a CNN. The network is initialized with a set of small random weights and biases. In backpropagation, a vector arriving (from the fully connected net) at the output pooling layer must be converted to 2-D arrays of the same size as the pooled feature maps in that layer. Each pooled feature map is upsampled to match the size of its corresponding feature map. The steps in the table are for one epoch of training.

Step	Description	Equations
Step 1	Input images	$a(0)$ = the set of image pixels in the input to layer 1
Step 2	Forward pass	For each neuron corresponding to location (x,y) in each feature map in layer ℓ compute: $z_{x,y}(\ell) = w(\ell) \star a_{x,y}(\ell - 1) + b(\ell)$ and $a_{x,y}(\ell) = h(z_{x,y}(\ell))$; $\ell = 1, 2, \dots, L_c$
Step 3	Backpropagation	For each neuron in each feature map in layer ℓ compute: $\delta_{x,y}(\ell) = h'(z_{x,y}(\ell)) [\delta_{x,y}(\ell + 1) \star \text{rot180}(w(\ell + 1))]$; $\ell = L_c - 1, L_c - 2, \dots, 1$
Step 4	Update parameters	Update the weights and bias for each feature map using $w_{l,k}(\ell) = w_{l,k}(\ell) - \alpha \delta_{l,k}(\ell) \star \text{rot180}(a(\ell - 1))$ and $b(\ell) = b(\ell) - \alpha \sum_x \sum_y \delta_{x,y}(\ell)$; $\ell = 1, 2, \dots, L_c$

FIGURE 12.45

CNN with one convolutional layer used to learn to recognize the images in Fig. 12.46.



epochs, or until the output error of the neural net reaches an acceptable value. The error is computed exactly as we did in Section 12.5. It can be the mean squared error, or the recognition error. Keep in mind that the weights in $w(\ell)$ and the bias value $b(\ell)$ are *different* for *each* feature map in layer ℓ .

EXAMPLE 12.16: Teaching a CNN to recognize some simple images.

We begin our illustrations of CNN performance by teaching the CNN in Fig. 12.45 to recognize the small 6×6 images in Fig. 12.46. As you can see on the left of this figure, there are three samples each of images of a horizontal stripe, a small centered square, and a vertical stripe. These images were used as the training set. On the right are noisy samples of images in these three categories. These were used as the test set.

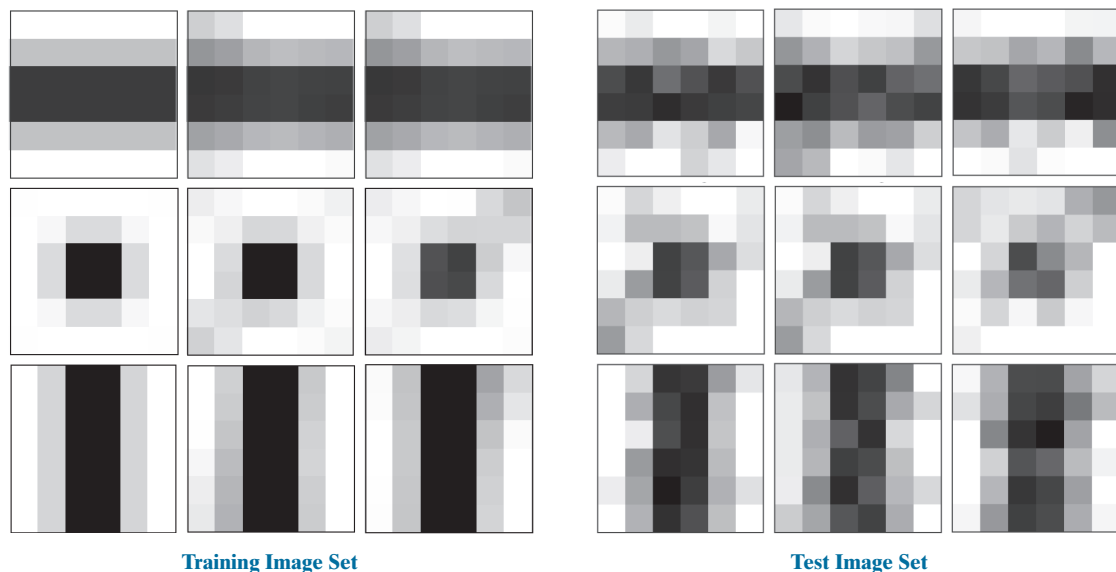
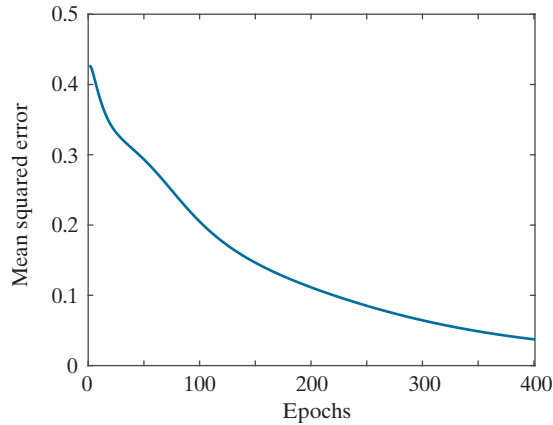


FIGURE 12.46 Left: Training images. Top row: Samples of a dark horizontal stripe. Center row: Samples of a centered dark square. Bottom row: Samples of a dark vertical stripe. Right: Noisy samples of the three categories on the left, created by adding Gaussian noise of zero mean and unit variance to the samples on the left. (All images are 8-bit grayscale images.)

FIGURE 12.47

Training MSE as a function of epoch for the images in Fig. 12.46. Perfect recognition of the training and test sets was achieved after approximately 100 epochs, despite the fact that the MSE was relatively high there.



As Fig. 12.45 shows, the inputs to our system are single images. We used a receptor field of size 3×3 , which resulted in feature maps of size 4×4 . There are two feature maps, which means we need two kernels of size 3×3 , and two biases. The pooled feature maps were generated using average pooling in neighborhoods of size 2×2 . This resulted in two pooled feature maps of size 2×2 , because the feature maps are of size 4×4 . The two pooled maps contain eight total elements which were organized as an 8-D column vector to vectorize the output of the last layer. (We used linear indexing of each image, then concatenated the two resulting 4-D vectors into a single 8-D vector.) This vector was then fed into the fully connected neural net on the right, which consists of the input layer and a three-neuron output layer, one neuron per class. Because this network has no hidden layers, it implements linear decision functions (see Problem 12.18). To train the system, we used $\alpha = 1.0$ and ran the system for 400 epochs. Figure 12.47 is a plot of the MSE as a function of epoch. Perfect recognition of the training set was achieved after approximately 100 epochs of training, despite the fact that the MSE was relatively high there. Recognition of the test set was 100% as well. The kernel and bias values learned by the system were:

$$\mathbf{w}_1 = \begin{bmatrix} 3.0132 & 1.1808 & -0.0945 \\ 0.9718 & 0.7087 & -0.9093 \\ 0.7193 & 0.0230 & -0.8833 \end{bmatrix}, b_1 = -0.2990 \quad \mathbf{w}_2 = \begin{bmatrix} -0.7388 & 1.8832 & 4.1077 \\ -1.0027 & 0.3908 & 2.0357 \\ -1.2164 & -1.1853 & -0.1987 \end{bmatrix}, b_2 = -0.2834$$

It is important that the CNN learned these parameters automatically from the raw training images. No features in the sense discussed in Chapter 11 were employed.

EXAMPLE 12.17: Using a large training set to teach a CNN to recognize handwritten numerals.

In this example, we look at a more practical application using a database containing 60,000 training and 10,000 test images of handwritten numeric characters. The content of this database, called the *MNIST database*, is similar to a database from NIST (National Institute of Standards and Technology). The former is a “cleaned up” version of the latter, in which the characters have been centered and formatted into grayscale images of size 28×28 pixels. Both databases are freely available online. Figure 12.48 shows examples of typical numeric characters available in the databases. As you can see, there is

FIGURE 12.48

Samples similar to those available in the NIST and MNIST databases. Each character subimage is of size 28×28 pixels. (Individual images courtesy of NIST.)



significant variability in the characters—and this is just a small sampling of the 70,000 characters available for experimentation.

Figure 12.49 shows the architecture of the CNN we trained to recognize the ten digits in the MNIST database. We trained the system for 200 epochs using $\alpha = 1.0$. Figure 12.50 shows the training MSE as a function of epoch for the 60,000 training images in the MNIST database.

Training was done using mini batches of 50 images at a time to improve the learning rate (see the discussion in Section 12.7). We also classified all images of the training set and all images of the test set after each epoch of training. The objective of doing this was to see how quickly the system was learning the characteristics of the data. Figure 12.51 shows the results. A high level of correct recognition performance was achieved after relatively few epochs for both data sets, with approximately 98% correct recognition achieved after about 40 epochs. This is consistent with the training MSE in Fig. 12.50, which dropped quickly, then began a slow descent after about 40 epochs. Another 160 epochs of training were required for the system to achieve recognition of about 99.9%. These are impressive results for such a small CNN.

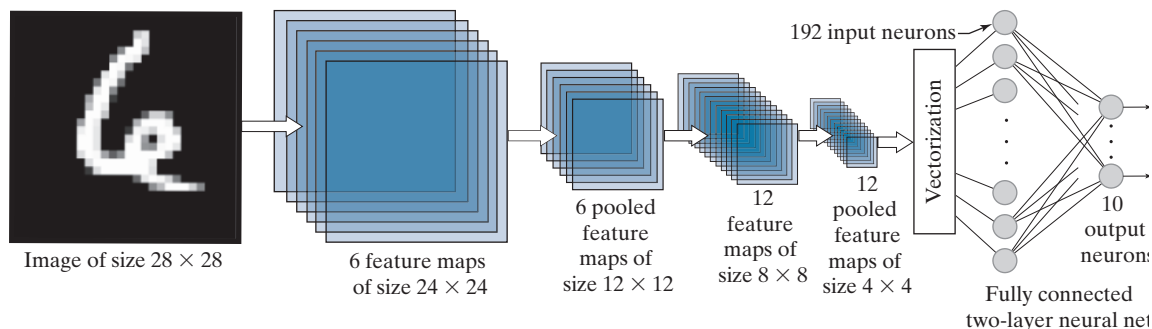


FIGURE 12.49 CNN used to recognize the ten digits in the MNIST database. The system was trained with 60,000 numerical character images of the same size as the image shown on the left. This architecture is the same as the architecture we used in Fig. 12.42. (Image courtesy of NIST.)

FIGURE 12.50

Training mean squared error as a function of epoch for the 60,000 training digit images in the MNIST database.

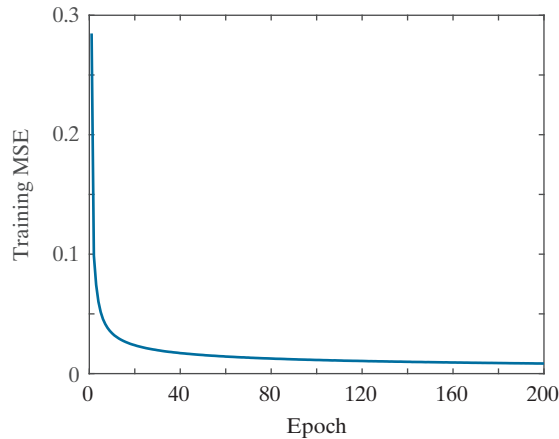


Figure 12.52 shows recognition performance on each digit class for both the training and test sets. The most revealing feature of these two graphs is that the CNN did equally as well on both sets of data. This is a good indication that the training was successful, and that it generalized well to digits it had not seen before. This is an example of the neural network not “over-fitting” the data in the training set.

Figure 12.53 shows the values of the kernels for the first feature map, displayed as intensities. There is one input image and six feature maps, so six kernels are required to generate the feature maps of the first layer. The dimensions of the kernels are the same as the receptive field, which we set at 5×5 . Thus, the first image on the left in Fig. 12.53 is the 5×5 kernel corresponding to the first feature map. Figure 12.54 shows the kernels for the second layer. In this layer, we have six inputs (which are the pooled maps of the first layer) and twelve feature maps, so we need a total of $6 \times 12 = 72$ kernels and biases to generate the twelve feature maps in the second layer. Each column of Fig. 12.54 shows the six

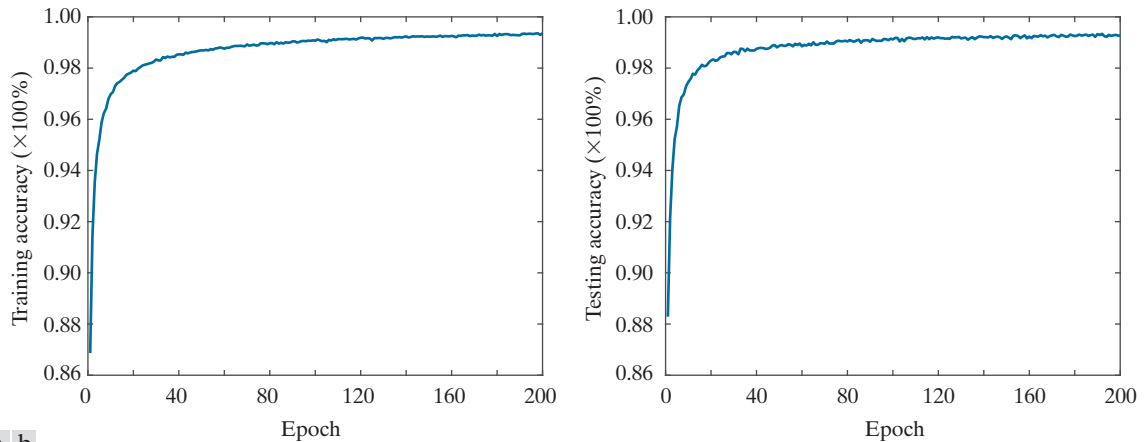
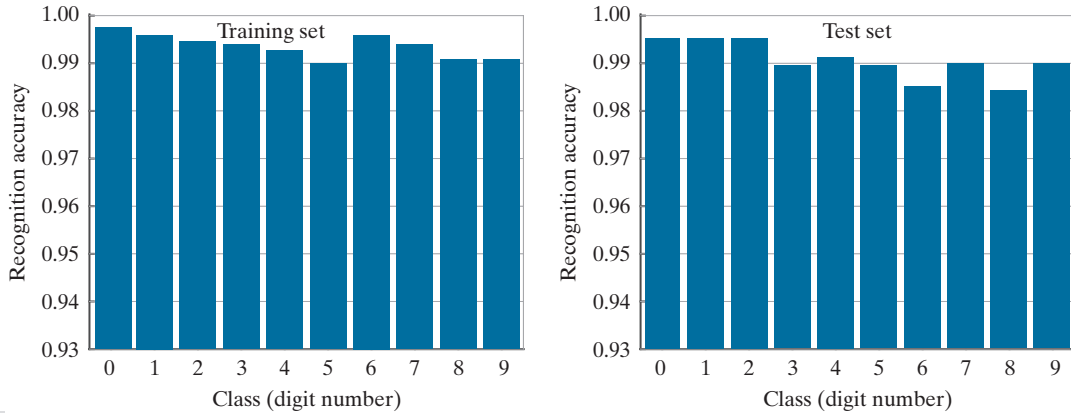


FIGURE 12.51 (a) Training accuracy (percent correct recognition of the training set) as a function of epoch for the 60,000 training images in the MNIST database. The maximum achieved was 99.36% correct recognition. (b) Accuracy as a function of epoch for the 10,000 test images in the MNIST database. The maximum correct recognition rate was 99.13%.



a b

FIGURE 12.52 (a) Recognition accuracy of training set by image class. Each bar shows a number between 0 and 1. When multiplied by 100%, these numbers give the correct recognition percentage for that class. (b) Recognition results per class in the test set. In both graphs the recognition rate is above 98%.

5×5 kernels corresponding to one of the feature maps in the second layer. We used 2×2 pooling in both layers, resulting in a 50% reduction of each of the two spatial dimensions of the feature maps.

Finally, it is of interest to visualize how one input image proceeds through the network, using the kernels learned during training. Figure 12.55 shows an input digit image from the test set, and the computations performed by the CNN at each layer. As before, we display numerical results as intensities.

Consider the results of convolution in the first layer. If you look at each resulting feature map carefully, you will notice that it highlights a different characteristic of the input. For example, the feature map on the top of the first column highlights the two vertical edges on the top of the character. The second highlights the edges of the entire inner region, and the third highlights a “blob-like” feature of the digit, as if it had been blurred by a lowpass kernel. The other three feature maps show other features. If you now look at the first two feature maps in the second layer, and compare them with the first feature map in the first layer, you can see that they could be interpreted as higher-level abstractions of the top of the character, in the sense that they show a dark area flanked on each side by white areas. Although these abstractions are not always easy to analyze visually, this example clearly demonstrates that they can be very effective. And, remember the important fact that our simple system learned these features automatically from 60,000 training images. This capability is what makes convolutional networks so powerful when it comes to image pattern classification. In the next example, we will consider even more complex images, and show some of the limitations of our simple CNN architecture.

EXAMPLE 12.18: Using a large image database to teach a CNN to recognize natural images.

In this example, we trained the same CNN architecture as in Fig. 12.49, but using the RGB color images in Fig. 12.56. These images are representative of those found in the CIFAR-10 database, a popular database used to test the performance of image classification systems. Our objective was to test the limitations of the CNN architecture in Fig. 12.49 by training it with data that is significantly more complex than the MNIST images in Example 12.17. The only difference between the architecture needed to

FIGURE 12.53

Kernels of the first layer after 200 epochs of training, shown as images.

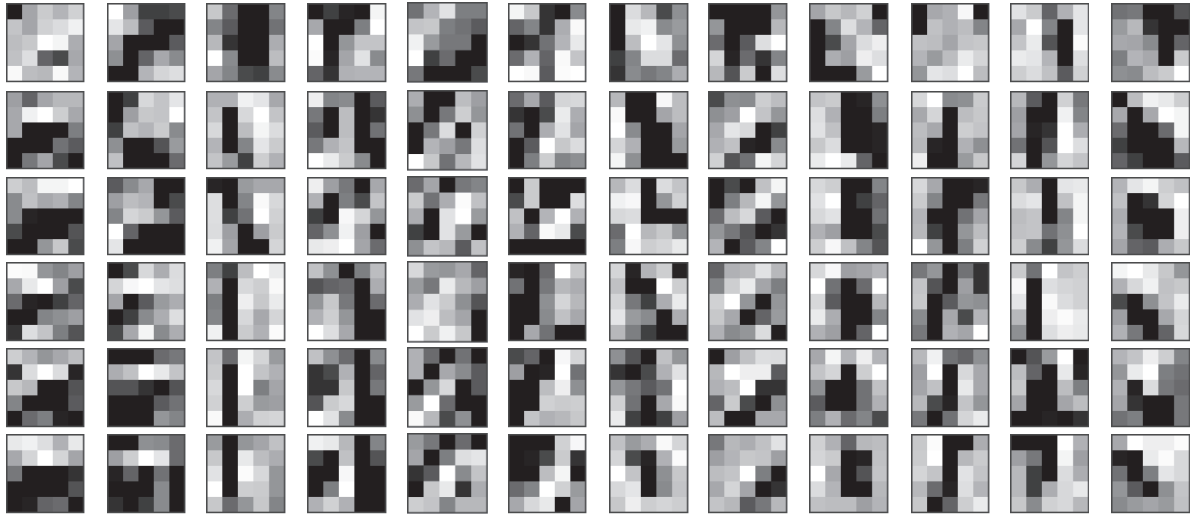


FIGURE 12.54 Kernels of the second layer after 200 epochs of training, displayed as images of size 5×5 . There are six inputs (pooled feature maps) into the second layer. Because there are twelve feature maps in the second layer, the CNN learned the weights of $6 \times 12 = 72$ kernels.

FIGURE 12.55

Results of a forward pass for one digit image through the CNN in Fig. 12.49 after training. The feature maps were generated using the kernels from Figs. 12.53 and 12.54, followed by pooling. The neural net is the two-layer neural network from Fig. 12.49. The output high value (in white) indicates that the CNN recognized the input properly. (This figure is the same as Fig. 12.44.)

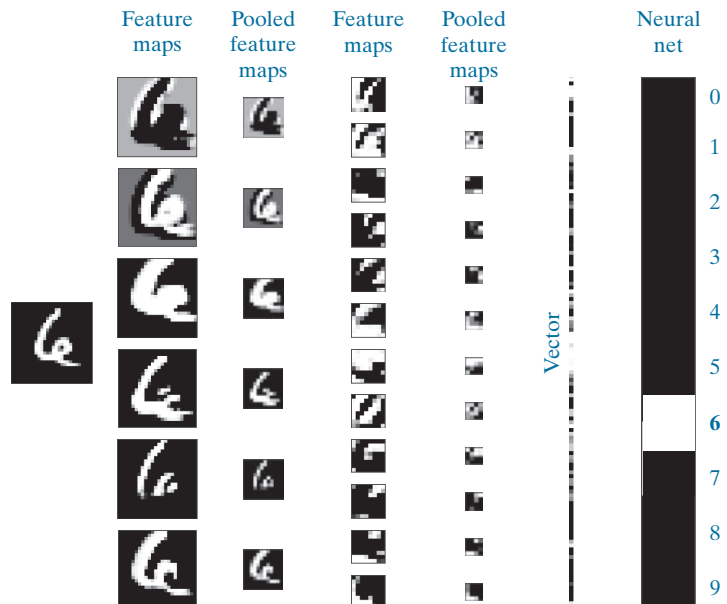


FIGURE 12.56 Mini images of size 32×32 pixels, representative of the 50,000 training and 10,000 test images in the CIFAR-10 database (the 10 stands for ten classes). The class names are shown on the right. (Images courtesy of Pearson Education.)

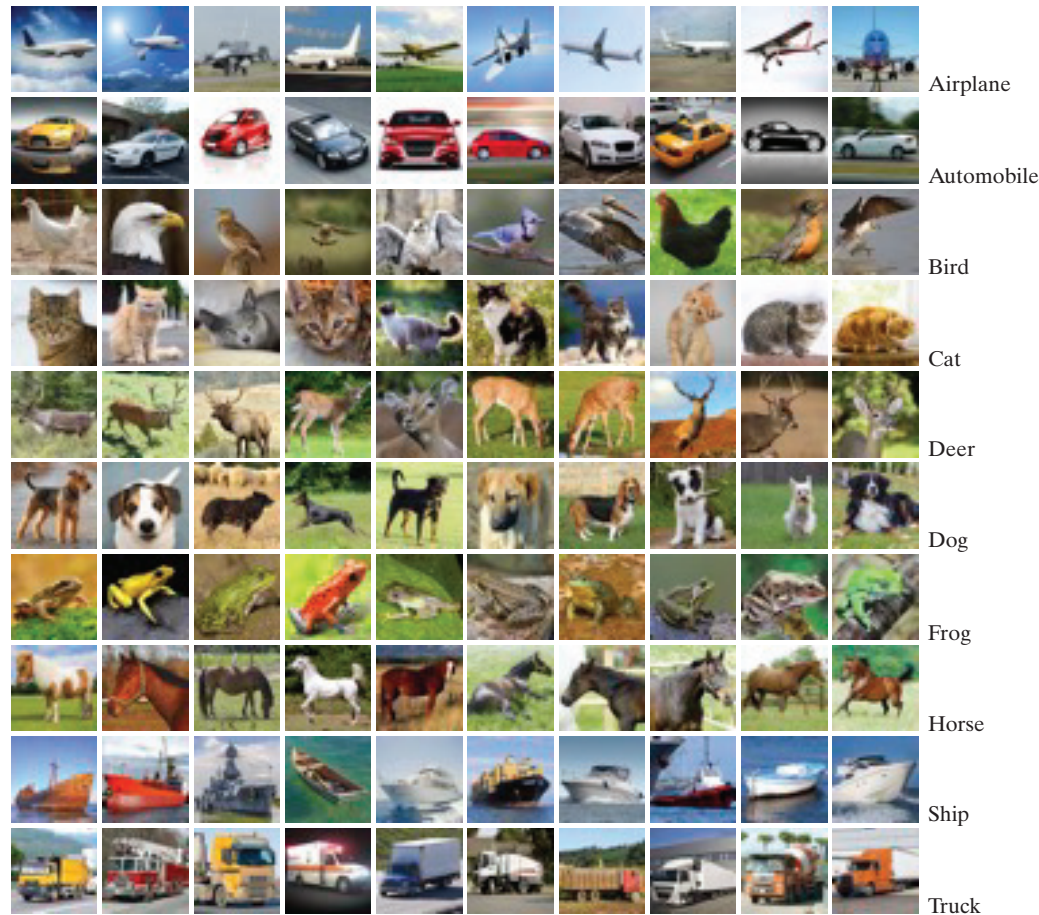
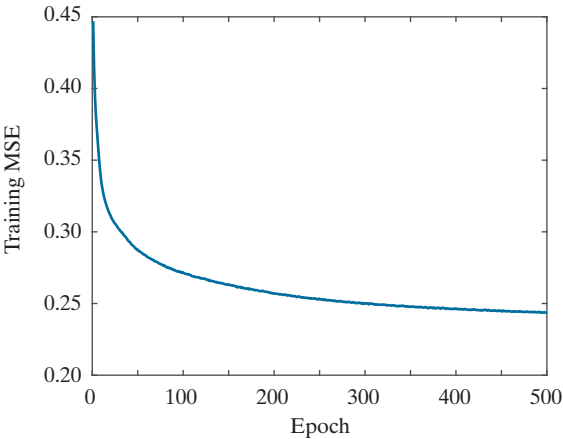


FIGURE 12.57 Training mean squared error as a function of the number of epochs for a training set of 50,000 CIFAR-10 images.



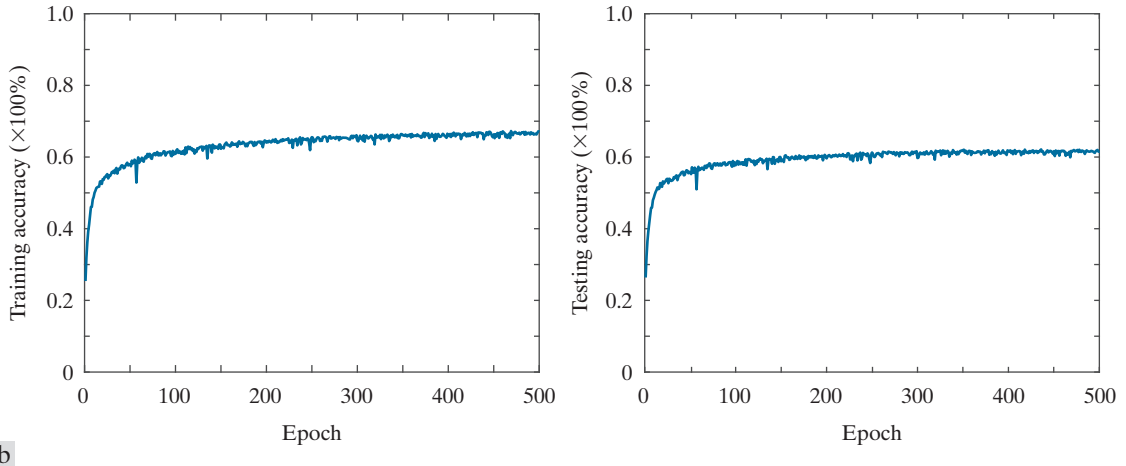


FIGURE 12.58 (a) Training accuracy (percent correct recognition of the training set) as a function of epoch for the 50,000 training images in the CIFAR-10 database. (b) Accuracy as a function of epoch for the 10,000 CIFAR-10 test images.

process the CIFAR-10 images, and the architecture in Fig. 12.49, is that the CIFAR-10 images are RGB color images, and hence have three channels. We worked with these input images using the approach explained in the subsection entitled Multiple Input Images, on page 973.

We trained the modified CNN for 500 epochs using the 50,000 training images of the CIFAR-10 database. Figure 12.57 is a plot of the mean squared error as a function of epoch during the training phase. Observe that the MSE begins to plateau at a value of approximately 0.25. In contrast, the MSE plot in Fig. 12.50 for the MNIST data achieved a much lower final value. This is not unexpected, given that the CIFAR-10 images are significantly more complex, both in the objects of interest as well as their backgrounds. The lower expected recognition performance of the training set is confirmed by the training-accuracy plotted in Fig. 12.58(a) as a function of epoch. The recognition rate leveled-off around 68% for the training data and about 61% for the test data. Although these results are not nearly as good as those obtained for the MNIST data, they are consistent with what we would expect from a very basic network. It is possible to achieve over 96% accuracy on this database (see Graham [2015]), but that requires a more complex network and a different pooling strategy.

Figure 12.59 shows the recognition accuracy per class for the training and test image sets. With a few exceptions, the highest recognition rate in both the training and test sets was achieved for engineered objects, and the lowest was for small animals. Frogs were an exception, caused most likely by the fact that frog size and shape are more consistent than they are, for example, in dogs and birds. As you can see in Fig. 12.59, if the small animals were removed from the list, recognition performance on the rest of the images would have been considerably higher.

Figures 12.60 and Fig. 12.61 show the kernels of the first and second layers. Note that each column in Fig. 12.60 has three 5×5 kernels. This is because there are three input channels to the CNN in this example. If you look carefully at the columns in Fig. 12.60, you can detect a similarity in the arrangement and values of the coefficients. Although it is not obvious what the kernels are detecting, it is clear that they are consistent in each column, and that all columns are quite different from each other, indicating a capability to detect different features in the input images. We show Fig. 12.61 for completeness only,

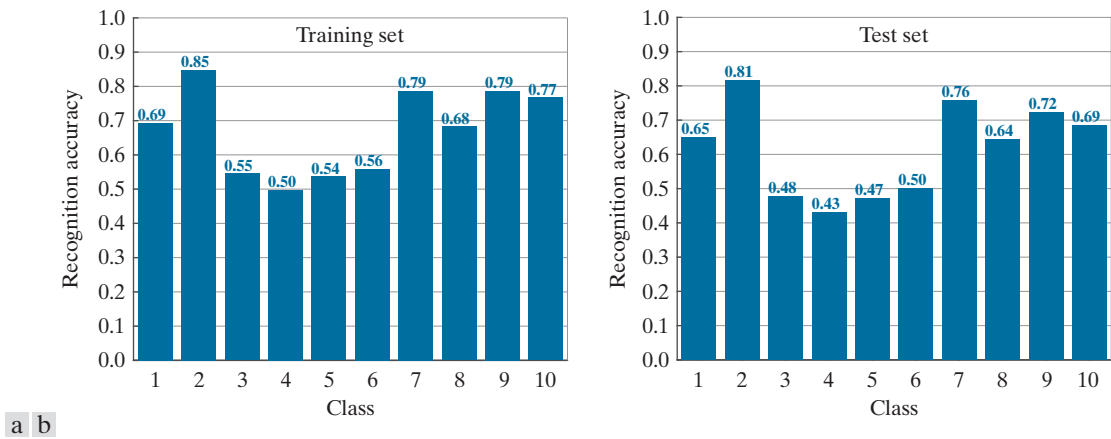
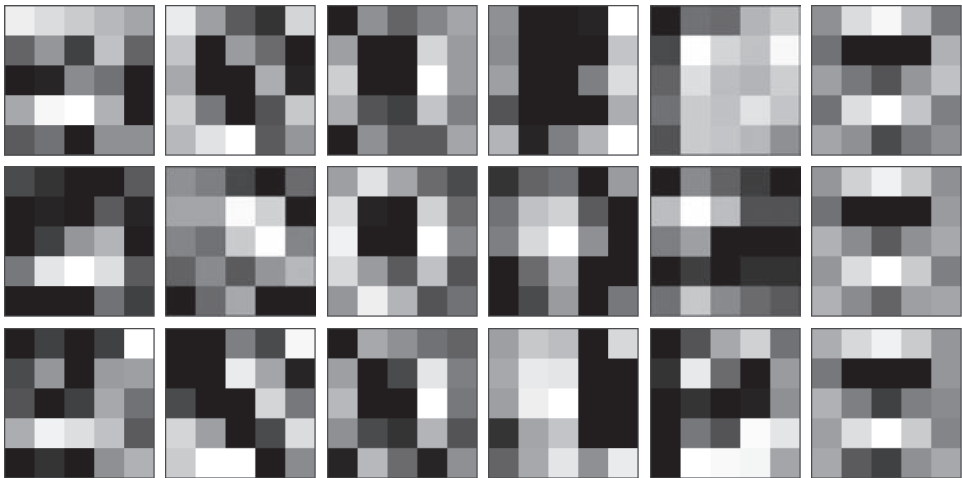


FIGURE 12.59 (a) CIFAR-10 recognition rate of training set by image class. Each bar shows a number between 0 and 1. When multiplied by 100%, these numbers give the correct recognition percentage for that class. (b) Recognition results per class in the test set.

as there is little we can infer that deep into the network, especially at this small scale, and considering the complexity of the images in the training set. Finally, Fig. 12.62 shows a complete recognition pass through the CNN using the weights in Figs. 12.60 and 12.61. The input shows the three color channels of the RGB image in the seventh column of the first row in Fig. 12.56. The feature maps in the first column, show the various features extracted from the input. The second column shows the pooling results, zoomed to the size of the features maps for clarity. The third and fourth columns show the results in the second layer, and the fifth column shows the vectorized output. Finally, the last column shows the result of recognition, with white representing a high output, and the others showing much smaller values. The input image was properly recognized as belonging to class 1.

FIGURE 12.60 Weights of the kernels of the first convolution layer after 500 epochs of training.



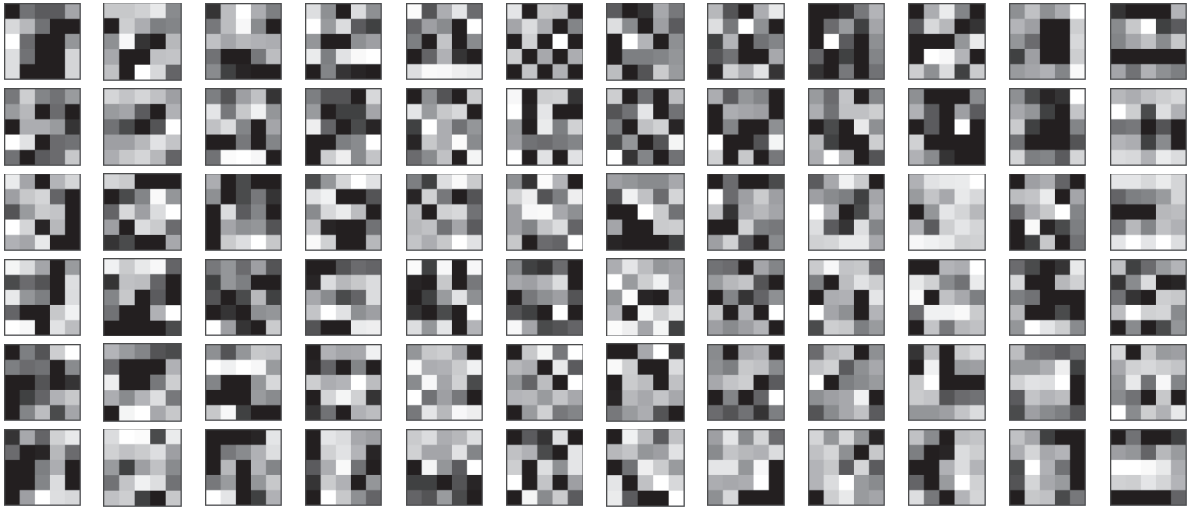


FIGURE 12.61 Weights of the kernels of the second convolution layer after 500 epochs of training. The interpretation of these kernels is the same as in Fig. 12.54.

12.7 SOME ADDITIONAL DETAILS OF IMPLEMENTATION

We mentioned in the previous section that neural (including convolutional) nets have the ability to learn features directly from training data, thus reducing the need for “engineered” features. While this is a significant advantage, it does not imply that the design of a neural network is free of human input. On the contrary, designing complex neural networks requires significant skill and experimentation.

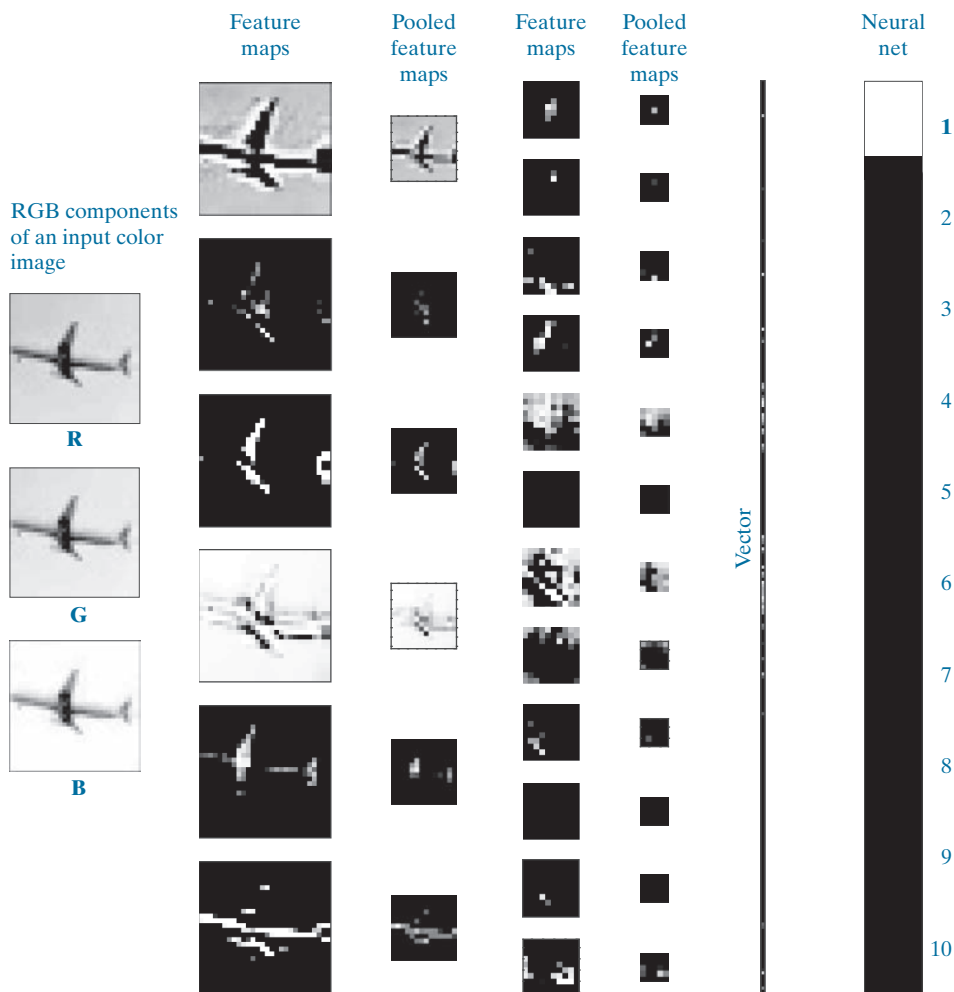
In the last two sections, our focus was on the development of fundamental concepts in neural nets, with an emphasis on the derivation of backpropagation for both fully connected and convolutional nets. Backpropagation is the backbone of neural net design, but there are other important considerations that influence how well a neural net learns, and then generalizes to patterns it has not seen before. In this section, we discuss briefly some important aspects in the design of fully connected and convolutional neural networks.

One of the first questions when designing a neural net architecture is how many layers to specify for the network. Theoretically, the *universality approximation theorem* (Cybenko [1989]) tells us that, under mild conditions, arbitrarily complex decision functions can be *approximated* by a continuous feedforward neural network with a single hidden layer. Although the theorem does not tell us how to compute the parameters of that single hidden layer, it does indicate that structurally simple neural nets can be very powerful. You have seen this in some of the examples in the last two sections. Experimental evidence suggests that deep neural nets (i.e., networks with two or more hidden layers) are better than a single hidden layer network at learning abstract representations, which typically is the main point of learning. There is no such thing as an algorithm to determine the “optimum” number of layers to use in a neural network. Therefore, specifying the number of layers generally

FIGURE 12.62

Graphical illustration of a forward pass through the trained CNN. The purpose was to recognize one input image from the set in Fig. 12.56. As the output shows, the image was recognized correctly as belonging to class 1, the class of airplanes.

(Original image courtesy of Pearson Education.)



is determined by a combination of experience and experimentation. “Starting small” is a logical approach to this problem. The more layers a network has, the higher the probability that backpropagation will run into problems such as so-called *vanishing gradients*, where gradient values are so small that gradient descent ceases to be effective. In convolutional networks, we have the added issue that the size of the inputs decreases as the images propagate through the network. There are two causes for this. The first is a natural size reduction caused by convolution itself, with the amount of reduction being proportional to the size of the receptive fields. One solution is to use *padding* prior to performing convolution operations, as we discussed in Section 3.4. The second (and most significant) cause of size reduction is pooling. The minimum pooling neighborhood is of size 2×2 , which reduces the size of feature maps by three-quarters at each layer. A solution that helps is to *upsample* the input

images, but this must be done with care because the relative sizes of features of interest would increase proportionally, thus influencing the size selected for receptive fields.

After the number of layers has been specified, the next task is to specify the number of neurons per layer. We always know how many neurons are needed in the first and last layers, but the number of neurons for internal layer is also an open question with no theoretical “best” answer. If the objective is to keep the number of layers as small as possible, the power of the network is increased to some degree by increasing the number of neurons per layer.

The main aspects of specifying the architecture of a neural network are completed by specifying the activation function. In this chapter, we worked with sigmoid functions for consistency between examples, but there are applications in which hyperbolic tangent and ReLU activation functions are superior in terms of improving training performance.

Once a network architecture has been specified, training is the central aspect of making the architecture useful. Although the networks we discussed in this chapter are relatively simple, networks applied to very large-scale problems can have millions of nodes and require large blocks of time to train. When available, the parameters of a *pretrained* network are an ideal starting point for further training, or for validating recognition performance. Another central theme in training neural nets is the use of GPUs to accelerate matrix operations.

An issue often encountered in training is *over-fitting*, in which recognition of the training set is acceptable, but the recognition rate on samples not used for training is much lower. That is, the net is not able to *generalize* what it learned and apply it to inputs it has not encountered before. When additional training data is not available, the most common approach is to artificially enlarge the training set using transformations such as geometric distortions and intensity variations. The transformations are carried out while preserving the class membership of the transformed patterns. Another major approach is to use *dropout*, a technique that randomly drops nodes with their connections from a neural network during training. The idea is to change the architecture slightly to prevent the net from adapting too much to a fixed set of parameters (see Srivastava et al. [2014]).

In addition to computational speed, another important aspect of training is efficiency. Simple things, such as shuffling the input patterns at the beginning of each training epoch can reduce or eliminate the possibility of “cycling,” in which parameter values repeat at regular intervals. *Stochastic gradient descent* is another important training refinement in which, instead of using the entire training set, samples are selected randomly and input into the network. You can think of this as dividing the training set into *mini-batches*, and then choosing a single sample from each mini-batch. This approach often results in speedier convergence during training.

In addition to the above topics, a paper by LeCun et al. [2012] is an excellent overview of the types of considerations introduced in the preceding discussion. In fact, the breath spanned by these topics is extensive enough to be the subject of an entire book (see Montavon et al. [2012]). The neural net architectures we discussed were by necessity limited in scope. You can get a good idea of the practical requirements of implementing practical networks by reading a paper by Krizhevsky, Sutskever, and Hinton [2012], which summarizes the design and implementation of a large-scale, deep convolutional neural network. There are a multitude of designs that have

been implemented over the past decade, including commercial and free implementations. A quick internet search will reveal a multitude of available architectures.

Summary, References, and Further Reading

Background material for Sections 12.1 through 12.4 are the books by Theodoridis and Koutroumbas [2006], by Duda, Hart, and Stork [2001], and by Tou and Gonzalez [1974]. For additional reading on the material on matching shape numbers see Bribiesca and Guzman [1980]. On string matching, see Sze and Yang [1981]. A significant portion of this chapter was devoted to neural networks. This is a reflection of the fact that neural nets, and in particular convolutional neural nets, have made significant strides in the past decade in solving image pattern classifications problems. As in the rest of the book, our presentation of this topic focused on fundamentals, but the topics covered were thoroughly developed. What you have learned in this chapter is a solid foundation for much of the work being conducted in this area. As we mentioned earlier, the literature on neural nets is vast, and quickly growing. As a starting point, a basic book by Nielsen [2015] provides an excellent introduction to the topic. The more advanced book by Goodfellow, Bengio, and Courville [2016] provides more depth into the mathematical underpinning of neural nets. Two classic papers worth reading are by Rumelhart, Hinton, and Williams [1986], and by LeCun, Bengio, and Haffner [1998]. The LeNet architecture we discussed in Section 12.6 was introduced in the latter reference, and it is still a foundation for image pattern classification. A recent survey article by LeCun, Bengio, and Hinton [2015] gives an interesting perspective on the scope of applicability of neural nets in general. The paper by Krizhevsky, Sutskever, and Hinton [2012] was one of the most important catalysts leading to the significant increase in the present interest on convolutional networks, and on their applicability to image pattern classification. This paper is also a good overview of the details and techniques involved in implementing a large-scale convolutional neural network. For details on the software aspects of many of the examples in this chapter, see Gonzalez, Woods, and Eddins [2009].

Problems

Solutions to the problems marked with an asterisk () are in the DIP4E Student Support Package (consult the book website: www.ImageProcessingPlace.com).*

12.1 Do the following:

- (a)* Compute the decision functions of a minimum distance classifier for the patterns in Fig. 12.10. You may obtain the required mean vectors by (careful) inspection.
 - (b) Sketch the decision boundary implemented by the decision functions in (a).
- 12.2*** Show that Eqs. (12-3) and (12-4) perform the same function in terms of pattern classification.
- 12.3** Show that the boundary given by Eq. (12-8) is the perpendicular bisector of the line joining the n -dimensional points $\mathbf{m}_i$ and $\mathbf{m}_j$.
- 12.4*** Show how the minimum distance classifier discussed in connection with Fig. 12.11 could be implemented by using N_c resistor banks (N_c is the number of classes), a summing junction at

each bank (for summing currents), and a maximum selector capable of selecting the maximum value of N_c decision functions in order to determine the class membership of a given input.

- 12.5*** Show that the correlation coefficient of Eq. (12-10) has values in the range $[-1, 1]$. (*Hint:* Express γ in vector form.)
- 12.6** Show that the distance measure $D(a, b)$ in Eq. (12-12) satisfies the properties in Eq. (12-13).
- 12.7*** Show that $\beta = \max(|a|, |b|) - \alpha$ in Eq. (12-14) is 0 if and only if a and b are identical strings.
- 12.8** Carry out the manual computations that resulted in the mean vector and covariance matrices in Example 12.5.
- 12.9*** The following pattern classes have Gaussian probability density functions:

$$c_1 : \{(0,0)^T, (2,0)^T, (2,2)^T, (0,2)^T\}$$

$$c_2 : \{(4,4)^T, (6,4)^T, (6,6)^T, (4,6)^T\}$$

- (a) Assume that $P(c_1) = P(c_2) = 1/2$ and obtain the equation of the Bayes decision boundary between these two classes.

- (b) Sketch the boundary.

- 12.10** Repeat Problem 12.9, but use the following pattern classes:

$$c_1 : \{(-1,0)^T, (0,-1)^T, (1,0)^T, (0,1)^T\}$$

$$c_2 : \{(-2,0)^T, (0,-2)^T, (2,0)^T, (0,2)^T\}$$

Note that the classes are not linearly separable.

- 12.11** With reference to the results in Table 12.1, compute the overall correct recognition rate for the patterns of the training set. Repeat for the patterns of the test set.

- 12.12*** We derived the Bayes decision functions

$$d_j(\mathbf{x}) = p(\mathbf{x}/c_j)P(c_j), j = 1, 2, \dots, N_c$$

using a 0-1 loss function. Prove that these decision functions minimize the probability of error. (*Hint:* The probability of error $p(e)$ is $1 - p(c)$, where $p(c)$ is the probability of being correct. For a pattern vector $\mathbf{x}$ belonging to class c_i , $p(c/\mathbf{x}) = p(c_i/\mathbf{x})$. Find $p(c)$ and show that $p(c)$ is maximum [$p(e)$ is minimum] when $p(\mathbf{x}/c_i)P(c_i)$ is maximum.)

- 12.13** Finish the computations started in Example 12.7.

- 12.14*** The perceptron algorithm given in Eqs. (12-44) through (12-46) can be expressed in a more concise form by multiplying the patterns of class c_2 by -1 , in which case the correction steps in the algorithm become $\mathbf{w}(k+1) = \mathbf{w}(k)$, if $\mathbf{w}^T(k)\mathbf{y}(k) > 0$, and $\mathbf{w}(k+1) = \mathbf{w}(k) + \alpha\mathbf{y}(k)$ otherwise, where we use $\mathbf{y}$ instead of $\mathbf{x}$ to make it clear that the patterns of class c_2 were multiplied by -1 . This is one of several perceptron algorithm formulations that can be derived starting from the general gradient descent equation

$$\mathbf{w}(k+1) = \mathbf{w}(k) - \alpha \left[\frac{\partial J(\mathbf{w}, \mathbf{y})}{\partial \mathbf{w}} \right]_{\mathbf{w}=\mathbf{w}(k)}$$

where $\alpha > 0$, $J(\mathbf{w}, \mathbf{y})$ is a criterion function, and the partial derivative is evaluated at $\mathbf{w} = \mathbf{w}(k)$. Show that the perceptron algorithm in the prob-

lem statement can be obtained from this general gradient descent procedure by using the criterion function

$$J(\mathbf{w}, \mathbf{y}) = \frac{1}{2} (\|\mathbf{w}^T \mathbf{y} - \mathbf{w}^T \mathbf{y}\|)$$

(*Hint:* The partial derivative of $\mathbf{w}^T \mathbf{y}$ with respect to $\mathbf{w}$ is $\mathbf{y}$.)

- 12.15*** Prove that the perceptron training algorithm given in Eqs. (12-44) through (12-46) converges in a finite number of steps if the training pattern sets are linearly separable. [*Hint:* Multiply the patterns of class c_2 by -1 and consider a non-negative threshold, T_0 so that the perceptron training algorithm (with $\alpha = 1$) is expressed in the form $\mathbf{w}(k+1) = \mathbf{w}(k)$, if $\mathbf{w}^T(k)\mathbf{y}(k) > T_0$, and $\mathbf{w}(k+1) = \mathbf{w}(k) + \alpha\mathbf{y}(k)$ otherwise. You may need to use the Cauchy-Schwartz inequality: $\|\mathbf{a}\|^2 \|\mathbf{b}\|^2 \geq (\mathbf{a}^T \mathbf{b})^2$.]

- 12.16** Derive equations of the derivatives of the following activation functions:

- (a) The sigmoid activation function in Fig. 12.30(a).
 (b) The hyperbolic tangent activation function in Fig. 12.30(b).
 (c)* The ReLU activation function in Fig. 12.30(c).

- 12.17*** Specify the structure, weights, and bias(es) of the smallest neural network capable of performing *exactly* the same function as a minimum distance classifier for two pattern classes in n -dimensional space. You may assume that the classes are tightly grouped and are linearly separable.

- 12.18** What is the decision boundary implemented by a neural network with n inputs, a single output neuron, and no hidden layers? Explain.

- 12.19** Specify the structure, weights, and bias of a neural network capable of performing exactly the same function as a Bayes classifier for two pattern classes in n -dimensional space. The classes are Gaussian with different means but equal covariance matrices.

- 12.20** Answer the following:

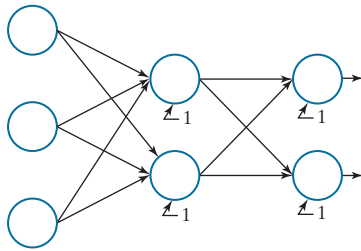
- (a)* Under what conditions are the neural networks in Problems 12.17 and 12.19 identical?
 (b) Suppose you specify a neural net architecture identical to the one in Problem 12.17. Would training by backpropagation yield the same

weights and bias as that network if trained with a sufficiently large number of samples? Explain.

12.21 Two pattern classes in two dimensions are distributed in such a way that the patterns of class c_1 lie randomly along a circle of radius r_1 . Similarly, the patterns of class c_2 lie randomly along a circle of radius r_2 , where $r_2 = 2r_1$. Specify the structure of a neural network with the minimum number of layers and nodes needed to classify properly the patterns of these two classes.

12.22* If two classes are linearly separable, we can train a perceptron starting with weights and a bias that are all zero, and we would still get a solution. Can you do the same when training a neural network by backpropagation? Explain.

12.23 Label the outputs, weights, and biases for every node in the following neural network using the general notation introduced in Fig. 12.31.



12.24 Answer the following:

- The last element of the input vector in Fig. 12.32 is 1. Is this vector augmented? Explain.
- Repeat the calculations in Fig. 12.32, but using weight matrices that are 100 times the values of those used in the figure.
- * What can you conclude in general from your results in (b)?

12.25 Answer the following:

- * The chain rule in Eq. (12-70) shows three terms. However, you are probably more familiar with chain rule expressions that have two terms. Show that if you start with the expression

$$\delta_j(\ell) = \frac{\partial E}{\partial z_j(\ell)} = \sum_i \frac{\partial E}{\partial z_i(\ell+1)} \frac{\partial z_i(\ell+1)}{\partial z_j(\ell)}$$

you can arrive at the result in Eq. (12-70).

- Show how the middle term in the third line of Eq. (12-70) follows from the middle term in the second.

12.26 Show the validity of Eq. (12-72). (*Hint*: Use the chain rule.)

12.27* Show that the dimensions of matrix $\mathbf{D}(\ell)$ in Eq. (12-79) are $n_\ell \times n_p$. (*Hint*: Some of the parameters in that equation are computed in forward propagation, so you already know their dimensions.)

12.28 With reference to the discussion following Eq. (12-82), explain why the error for one pattern is obtained by squaring the elements of one column of matrix $(\mathbf{A}(L) - \mathbf{R})$, adding them, and dividing the result by 2.

12.29* The matrix formulation in Table 12.3 contains all patterns as columns of a single matrix $\mathbf{X}$. This is ideal in terms of speed and economy of implementation. It is also well suited when training is done using mini-batches. However, there are applications in which the large number of training vectors is too large to hold in memory, and it becomes more practical to loop through each pattern using the vector formulation. Compose a table similar to Table 12.3, but using individual patterns, $\mathbf{x}$, instead of matrix $\mathbf{X}$.

12.30 Consider a CNN whose inputs are RGB color images of size 512×512 pixels. The network has two convolutional layers. Using this information, answer the following:

- * You are told that the spatial dimensions of the feature maps in the first layer are 504×504 , and that there are 12 feature maps in the first layer. Assuming that no padding is used, and that the kernels used are square, and of an odd size, what are the spatial dimensions of these kernels?
- If subsampling is done using neighborhoods of size 2×2 , what are the spatial dimensions of the pooled feature maps in the first layer?
- What is the depth (number) of the pooled feature maps in the first layer?
- The spatial dimensions of the convolution kernels in the second layer are 3×3 . Assuming no padding, what are the sizes of the feature maps in the second layer?
- You are told that the number of feature maps

in the second layer is 6, and that the size of the pooling neighborhoods is again 2×2 . What are the dimensions of the vectors that result from vectorizing the last layer of the CNN? Assume that vectorization is done using linear indexing.

- 12.31** Suppose the input images to a CNN are padded to compensate for the size reduction caused by convolution and subsampling (pooling). Let P denote the thickness of the padding border, let V denote the width of the (square) input images, let S denote the stride, and let F denote the width of the (square) receptive field.

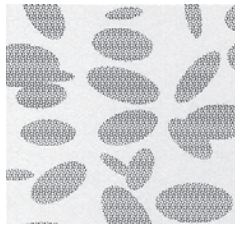
- (a) Show that the number, N , of neurons in each row in the resulting feature map is

$$N = \frac{V + 2P - F}{S} + 1$$

- (b)* How would you interpret a result using this equation that is not an integer?

- 12.32*** Show the validity of Eq. (12-106).

- 12.33** An experiment produces binary images of blobs that are nearly elliptical in shape, as the following example image shows. The blobs are of three sizes, with the average values of the principal axes of the ellipses being (1.3, 0.7), (1.0, 0.5), and (0.75, 0.25). The dimensions of these axes vary $\pm 10\%$ about their average values.



Develop an image processing system capable of rejecting incomplete or overlapping ellipses, then classifying the remaining single ellipses into one of the three given size classes. Show your solution in block diagram form, giving specific details regarding the operation of each block. Solve the classification problem using a minimum distance classifier, indicating clearly how you would go about obtaining training samples, and how you would use these samples to train the classifier.

- 12.34** A factory mass-produces small American flags for sporting events. The quality assurance team has observed that, during periods of peak production, some printing machines have a tendency to drop (randomly) between one and three stars and one or two entire stripes. Aside from these errors, the flags are perfect in every other way. Although the flags containing errors represent a small percentage of total production, the plant manager decides to solve the problem. After much investigation, she concludes that automatic inspection using image processing techniques is the most economical approach. The basic specifications are as follows: The flags are approximately 7.5 cm by 12.5 cm in size. They move lengthwise down the production line (individually, but with a $\pm 15\%$ variation in orientation) at approximately 50 cm/s, with a separation between flags of approximately 5 cm. In all cases, “approximately” means $\pm 5\%$. The plant manager employs you to design an image processing system for each production line. You are told that cost and simplicity are important parameters in determining the viability of your approach. Design a complete system based on the model of Fig. 1.23. Document your solution (including assumptions and specifications) in a brief (but clear) written report addressed to the plant manager. You can use any of the methods discussed in the book.